

**ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP**

**XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN
TRỰC TUYẾN TÍCH HỢP AI HỖ TRỢ
TÌM KIẾM VÀ GỢI Ý SÁCH**

Ngành: Khoa học Máy tính

MÃ ĐỀ TÀI: HK253-DATN-076

HỘI ĐỒNG: 5L Khoa học Máy tính

GVHD: TS. Trương Tuấn Anh

CN. Dương Huỳnh Anh Đức

GVPB: ThS. Trần Thị Quê Nguyệt

–o0o–

SVTH: Võ Quang Thắng - 2213214

TP. HỒ CHÍ MINH, 10/2026

LỜI CAM ĐOAN

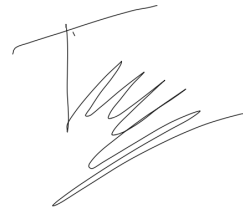
Em xin cam đoan đồ án tốt nghiệp với đề tài **“Xây dựng hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách”** là công trình do em thực hiện dưới sự hướng dẫn của **TS. Trương Tuấn Anh** và **CN. Dương Huỳnh Anh Đức**.

Các nội dung nghiên cứu, thiết kế, triển khai và đánh giá trong báo cáo được trình bày trung thực. Những tài liệu, số liệu, hình ảnh, công cụ và kết quả tham khảo từ các nguồn khác đều được trích dẫn hoặc ghi nhận trong phần tài liệu tham khảo và các phần liên quan của báo cáo.

Em xin chịu trách nhiệm trước Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa - Đại học Quốc gia TP.HCM về tính trung thực và chính xác của nội dung báo cáo này.

Tp. Hồ Chí Minh, tháng 10 năm 2026

Sinh viên thực hiện



Võ Quang Thắng

LỜI CẢM ƠN

Trong suốt thời gian học tập và nghiên cứu tại **Trường Đại học Bách Khoa - Đại học Quốc gia Thành phố Hồ Chí Minh**, em đã nhận được sự quan tâm, giảng dạy và hướng dẫn tận tình từ quý thầy cô trong khoa. Đồ án tốt nghiệp này là minh chứng cho sự nỗ lực không ngừng của em trong việc vận dụng những kiến thức đã được trang bị để giải quyết các vấn đề thực tiễn trong lĩnh vực công nghệ thông tin.

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **TS. Trương Tuấn Anh**, người thầy đã tận tâm hướng dẫn, định hướng và đồng hành cùng em trong suốt quá trình thực hiện đồ án. Thầy không chỉ truyền đạt kiến thức chuyên môn mà còn định hướng phương pháp nghiên cứu khoa học, giúp em có cái nhìn sâu sắc hơn về vấn đề đang giải quyết. Những lời khuyên, góp ý quý báu của thầy là động lực to lớn giúp em vượt qua mọi khó khăn, thử thách để hoàn thành tốt đồ án này.

Em cũng xin gửi lời cảm ơn chân thành đến **CN. Dương Huỳnh Anh Đức** đã đồng hành, hỗ trợ và đóng góp nhiều ý kiến quý báu trong quá trình thực hiện đồ án, giúp em có thêm góc nhìn thực tiễn và áp dụng hiệu quả các giải pháp kỹ thuật.

Em xin bày tỏ lòng biết ơn sâu sắc đến toàn thể quý thầy cô **Khoa Khoa học và Kỹ thuật Máy tính** đã tận tình giảng dạy, truyền đạt kiến thức và kinh nghiệm quý báu trong suốt những năm học vừa qua. Nền tảng kiến thức vững chắc mà các thầy cô đã trang bị là hành trang vô cùng quý giá để em tự tin bước vào môi trường làm việc thực tế.

Em cũng xin cảm ơn gia đình, bạn bè đã luôn bên cạnh, động viên và tạo điều kiện tốt nhất để em có thể hoàn thành tốt chương trình học tập cũng như đồ án tốt nghiệp này.

Do thời gian và kinh nghiệm có hạn, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, chỉ bảo của quý thầy cô để đồ án được hoàn thiện hơn và em có thêm kinh nghiệm quý báu cho công việc sau này.

Xin chân thành cảm ơn!

Tp. Hồ Chí Minh, tháng 10 năm 2026

Sinh viên thực hiện



Võ Quang Thắng

TÓM TẮT

Đồ án tốt nghiệp này tập trung vào việc nghiên cứu, thiết kế và xây dựng **Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI**, nhằm giải quyết các hạn chế của các hệ thống thư viện truyền thống như khả năng tìm kiếm kém hiệu quả và thiếu sự cá nhân hóa. Hệ thống được phát triển với mục tiêu số hóa toàn diện quy trình vận hành thư viện, đồng thời ứng dụng trí tuệ nhân tạo để nâng cao trải nghiệm đọc và khám phá tri thức của người dùng.

Về mặt kỹ thuật, hệ thống được xây dựng dựa trên kiến trúc **Modular Monolith** với Clean Architecture + Domain-Driven Design cho từng module, đảm bảo tính bảo mật và khả năng mở rộng. Backend sử dụng **Spring Boot (Java)** để xử lý các nghiệp vụ phức tạp, kết hợp với Frontend **React + Vite** mang lại giao diện hiện đại, tốc độ build nhanh và triển khai đơn giản qua NGINX. Các chức năng chính bao gồm: quản lý tài nguyên sách, quản lý độc giả, quy trình mượn trả tự động, kiểm soát phí phạt và hệ thống báo cáo thống kê trực quan cho thủ thư.

Điểm đột phá của đồ án là việc tích hợp sâu các công nghệ AI/NLP tiên tiến. Cụ thể, hệ thống ứng dụng mô hình **Sentence-BERT (SBERT)** để thực hiện tìm kiếm ngữ nghĩa (Semantic Search), giúp xử lý các truy vấn tiếng Việt phức tạp mà không phụ thuộc vào từ khóa chính xác. Song song đó, hệ thống gợi ý sách sử dụng phương pháp lai (**Hybrid Approach**), kết hợp giữa Lọc cộng tác (Collaborative Filtering) và Lọc dựa trên nội dung (Content-Based), giúp đưa ra các đề xuất sách phù hợp nhất dựa trên lịch sử và hành vi của từng độc giả.

Kết quả của đồ án là một hệ thống phần mềm hoàn chỉnh, có giao diện thân thiện, đáp ứng đầy đủ các yêu cầu nghiệp vụ của một thư viện đại học. Hệ thống đã chứng minh được tính khả thi trong việc kết hợp giữa quản lý truyền thống và công nghệ thông minh, mang lại giá trị thực tiễn cao trong việc thúc đẩy văn hóa đọc và chuyển đổi số trong môi trường giáo dục.

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Danh mục hình ảnh	vii
Danh mục bảng biểu	ix
1 Giới thiệu	1
1.1 Động cơ của đề tài	1
1.2 Mục tiêu	1
1.3 Phạm vi	2
1.4 Ý nghĩa	3
1.4.1 Ý nghĩa thực tiễn	3
1.4.2 Ý nghĩa khoa học	3
1.5 Cấu trúc báo cáo	3
2 Kiến thức và công nghệ nền tảng	5
2.1 Các Lĩnh vực Công nghệ Chính	5
2.2 Nền tảng Kiến thức Khoa học	5
3 Công trình liên quan	7
3.1 Các công trình nghiên cứu khoa học	7
3.2 Các hệ thống và phần mềm tương tự	7
3.3 Kết chương và xác định khoảng trống nghiên cứu	8
4 Hệ thống đề xuất	9
4.1 Mô tả hệ thống	9
4.2 Hướng tiếp cận thiết kế	9
4.3 Yêu cầu	10
4.3.1 Yêu cầu chức năng	10
4.3.2 Yêu cầu phi chức năng	11
4.3.3 Yêu cầu dữ liệu	12
5 Phân tích và Thiết kế	13

5.1	Phân tích	13
5.1.1	Mô hình hóa tương tác (Use-case Diagram)	13
5.1.2	Mô hình hóa luồng xử lý (Activity Diagram)	14
5.1.3	Mô hình lớp (Class diagram)	17
5.1.4	Công nghệ được sử dụng	18
5.2	Ứng dụng AI vào hệ thống	19
5.2.1	Cơ sở thiết kế dữ liệu AI	21
5.2.2	Module 1: Xử lý PDF file	22
5.2.3	Module 2: Hỗ trợ tìm kiếm ngữ nghĩa (Semantic Search)	24
5.2.4	Module 3: Gợi ý cá nhân hoá (Recommendation)	25
5.3	Thiết kế cơ sở dữ liệu	27
5.3.1	Lựa chọn Hệ Quản trị Cơ sở dữ liệu (HQTCSDL)	27
5.3.2	Mô hình Quan hệ Thực thể (ERD)	28
6	Hiện thực hệ thống	29
6.1	Hiện thực ứng dụng	29
6.1.1	Hiện thực Frontend	29
6.1.2	Hiện thực Backend	36
6.1.3	Hiện thực AI Service	40
6.1.4	Tổng kết kết quả hiện thực	44
6.2	Tổng kết hiện thực	45
7	Kiểm thử hệ thống	46
7.1	Chiến lược và môi trường kiểm thử	46
7.2	Kiểm thử tầng mã nguồn	47
7.2.1	Kiểm thử Frontend	47
7.2.2	Kiểm thử Backend	48
7.2.3	Kiểm thử AI Service	49
7.3	Kiểm thử trên môi trường triển khai	49
7.3.1	Kiểm thử tương tác hệ thống	50
7.3.2	Cấu trúc bảo mật định tuyến	51
7.3.3	Khả năng giám sát và đo lường	51
7.4	Đánh giá định lượng chất lượng AI	52
7.5	Kiểm thử chịu tải có kiểm soát	54
7.6	Kết luận kiểm thử	55
8	Triển khai hệ thống	56
8.1	Caddy — Reverse Proxy	56
8.1.1	Vai trò của Caddy	56
8.2	Môi trường sản xuất	56
8.2.1	Docker Compose sản xuất	56
8.2.2	Quản lý biến môi trường và bảo mật	57
8.2.3	Flyway Migration tự động	57

8.3	Quy trình khởi tạo hạ tầng ban đầu	58
8.4	Hệ thống Tích hợp và Triển khai liên tục (CI/CD Pipeline)	59
8.4.1	Đặt vấn đề và mục tiêu thiết kế	59
8.4.2	Kiến trúc tổng thể và luồng hoạt động	59
8.4.3	Chi tiết giai đoạn Tích hợp liên tục (CI)	60
8.4.4	Chi tiết giai đoạn Triển khai liên tục (CD)	61
8.4.5	Quản lý bảo mật và phân tách cấu hình bí mật	63
8.4.6	Đánh giá định lượng hiệu quả hệ thống	63
8.5	Hạ tầng mạng: Domain và HTTPS	64
8.5.1	Tên miền (Domain)	64
8.5.2	Tổng chi phí vận hành	64
9	Tổng kết	66
9.1	Tổng hợp kết quả hiện thực	66
9.1.1	Phân lớp Frontend	66
9.1.2	Phân lớp Backend	67
9.1.3	Phân lớp AI Service	67
9.1.4	Kiểm thử và Đảm bảo chất lượng	68
9.1.5	Đóng gói và Triển khai hạ tầng	68
9.2	Đánh giá tác động thực tiễn	68
9.3	Nhận định kiến trúc và giới hạn kỹ thuật	69
9.4	Định hướng phát triển	69
9.5	Kết luận chung	70

Tài liệu tham khảo

Tài nguyên và Mã nguồn dự án

Danh sách hình vẽ

5.1	Activity Diagram cho xác thực người dùng	14
5.2	Activity Diagram cho tra cứu và gợi ý tài liệu	15
5.3	Activity Diagram cho nghiệp vụ mượn–trả	15
5.4	Activity Diagram cho đặt trước và tương tác sau mượn	16
5.5	Activity Diagram cho nghiệp vụ thêm sách	16
5.6	Class diagram tổng quát của hệ thống	17
5.7	Trách nhiệm giữa frontend, backend và AI service	20
5.8	Module 1 – Luồng xử lý PDF file sau khi thủ thư upload	22
5.9	Module 2 – Luồng thực thi semantic search	25
5.10	Sơ đồ Quan hệ Thực thể (ERD) tổng thể của hệ thống	28
6.1	Kiến trúc tổng thể và cách thức tổ chức mã nguồn Frontend	30
6.2	Giao diện tổng quan và chức năng tìm kiếm thông minh dành cho bạn đọc	32
6.3	Trải nghiệm xem chi tiết và tương tác với ấn phẩm	33
6.4	Quy trình tích hợp thanh toán tự động	33
6.5	Nghiệp vụ nhập liệu và số hóa tài liệu thông minh	34
6.6	Nghiệp vụ quản lý vận hành và giám sát hệ thống của thủ thư	34
6.7	Luồng giao tiếp và hỗ trợ trực tuyến giữa các nhóm người dùng	35
6.8	Tính năng cá nhân hóa giao diện và đa ngôn ngữ (i18n)	35
6.9	Sơ đồ kiến trúc Modular Monolith tổng thể	37
6.10	Cấu trúc thư mục mã nguồn Backend thực tế	37
6.11	Tài liệu hóa API tự động bằng Swagger (Đại diện module Publication)	38
6.12	Kiến trúc tổng thể và luồng giao tiếp của AI Service	41
6.13	Luồng xử lý trích xuất, biến đổi và tải dữ liệu (ETL Pipeline) cho tài liệu PDF .	41
6.14	Các chức năng AI đã tích hợp vào hệ thống	43
6.15	Pipeline xử lý PDF và lưu vector	43
7.1	Kết quả chạy bộ kiểm thử Frontend	47
7.2	Kết quả chạy bộ kiểm thử Backend	48
7.3	Kết quả chạy bộ kiểm thử AI Service	49
7.4	Kết quả kiểm thử tương tác trên hệ thống library74.uk	50
7.5	Kết quả benchmark định lượng chất lượng AI	52
7.6	Kết quả kiểm thử tải có kiểm soát trên production	54
8.1	Kiến trúc tổng thể luồng CI/CD trong hệ thống Library74	60
8.2	Luồng kết nối từ tên miền Library74 đến hệ thống production	64

9.1 Nhận diện thương hiệu của nền tảng Library74	66
--	----

Danh sách bảng

4.1	Bảng đặc tả yêu cầu hệ thống	10
5.1	Đối chiếu nhóm Use Case với hiện thực hệ thống	13
5.2	Công nghệ được sử dụng trong hệ thống	18
5.3	Giá trị thiết kế của lớp AI trong hệ thống	20
5.4	Ranh giới trách nhiệm khi tích hợp AI	21
5.5	Nguồn dữ liệu dùng cho thiết kế AI	21
5.6	Các thành phần kỹ thuật trong Module 1	23
5.7	Thiết kế Module 1 – Xử lý PDF file	24
5.8	Trọng số xếp hạng trong Module 2	25
5.9	Thiết kế Module 2 – Hỗ trợ tìm kiếm ngữ nghĩa	25
5.10	Ba nguồn tín hiệu trong Module 3 – Recommendation	26
5.11	Trọng số hybrid recommendation trong Module 3	27
6.1	Các màn hình Frontend đã hiện thực theo nhóm người dùng	30
6.2	Các hạng mục kỹ thuật Frontend	35
6.3	Các điểm giá trị nổi bật ở Frontend	36
6.4	Các module Backend đã hiện thực	38
6.5	Các hạng mục kỹ thuật Backend	40
6.6	Các điểm giá trị nổi bật ở Backend	40
6.7	Các thành phần AI Service đã hiện thực	42
6.8	Các năng lực AI đã tích hợp vào hệ thống	43
6.9	Các điểm giá trị nổi bật ở AI Service	44
6.10	Các luồng giá trị được minh chứng trong phần hiện thực	45
6.11	Các luồng end-to-end đã hiện thực	45
7.1	Tổng quan chiến lược kiểm thử hệ thống	46
7.2	Kết quả kiểm thử Frontend	47
7.3	Phạm vi kiểm thử Backend theo module	48
7.4	Phạm vi kiểm thử AI Service	49
7.5	Kết quả API smoke test trên phân hệ máy chủ	50
7.6	Đánh giá năng lực bảo vệ trên môi trường thực tế	51
7.7	Đánh giá mạng lưới đo lường trên phân hệ production	52
7.8	Kết quả benchmark AI trên dữ liệu production hiện tại	53
7.9	Kết quả load probe theo bậc trên endpoint tìm kiếm catalog	54

8.1	Các nhóm biến môi trường của hệ thống	57
8.2	Chi tiết các workflow Tích hợp liên tục (CI)	61
8.3	Đánh giá hiệu quả vận hành trước và sau khi áp dụng CI/CD	63
8.4	Chi phí vận hành hệ thống trong cấu hình demo	65
1	Danh sách các tài nguyên trực tuyến của hệ thống	

Chương 1: Giới thiệu

Chương này trình bày tổng quan về đề tài, bao gồm lý do lựa chọn, mục tiêu cần đạt được, phạm vi nghiên cứu và phát triển, ý nghĩa khoa học–thực tiễn của dự án, đồng thời giới thiệu cấu trúc tổng thể của báo cáo.

1.1 Động cơ của đề tài

Trong bối cảnh chuyển đổi số giáo dục đang diễn ra mạnh mẽ, việc hiện đại hóa các hệ thống thư viện tại trường đại học là một yêu cầu cấp thiết. Các hệ thống thư viện hiện tại, như Koha hay OPAC-HCMUT, dù đáp ứng được nhu cầu quản lý cơ bản nhưng vẫn còn nhiều hạn chế. Cụ thể, các hệ thống này chủ yếu dựa trên tìm kiếm từ khóa đơn giản, chưa khai thác tốt nội dung số của tài liệu PDF, chưa có cơ chế gợi ý dựa trên hành vi người dùng, và giao diện chưa thật sự tối ưu cho trải nghiệm sử dụng trên nhiều thiết bị. Thêm vào đó, các nghiệp vụ như đặt trước, nhắc hạn trả, thông báo thời gian thực, theo dõi phí phạt và thống kê xu hướng đọc thường còn rời rạc, gây tốn nhiều thời gian cho thủ thư.

Từ thực tiễn đó, có một nhu cầu rõ rệt từ các bên liên quan:

- **Sinh viên và giảng viên:** Cần một công cụ tìm kiếm tài liệu nhanh chóng, chính xác, hỗ trợ cả truy vấn tiếng Việt/tiếng Anh và gợi ý tài liệu liên quan để phục vụ học tập, nghiên cứu.
- **Thủ thư và nhà quản lý:** Mong muốn có một hệ thống tự động hóa các công việc lặp lại như quản lý ấn phẩm, bản sao, mượn–trả, đặt trước, phí phạt, thông báo và báo cáo thống kê.
- **Nhà trường:** Hướng tới việc thúc đẩy văn hóa đọc và năng lực số của sinh viên thông qua một nền tảng thư viện hiện đại, có khả năng mở rộng và tích hợp với các dịch vụ số khác.

Vì vậy, việc phát triển một **Hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách** là cần thiết, không chỉ để giải quyết các hạn chế của hệ thống cũ mà còn để thử nghiệm cách kết hợp phần mềm quản lý thư viện truyền thống với AI service xử lý nội dung tài liệu.

1.2 Mục tiêu

Đề tài đặt ra mục tiêu tổng quát là xây dựng một hệ thống quản lý thư viện trực tuyến thông minh, vừa đảm bảo các chức năng quản lý cốt lõi, vừa tích hợp AI/NLP để nâng cao trải nghiệm tìm kiếm, khám phá và đọc tài liệu của người dùng.

Các mục tiêu cụ thể bao gồm:

- **Xây dựng hệ thống quản lý thư viện trực tuyến:** Cung cấp các chức năng chính như quản lý ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag, người dùng, mượn–trả, đặt trước, phí phạt và thông báo.
- **Ứng dụng AI/NLP để nâng cao trải nghiệm:**
 - Phát triển công cụ tìm kiếm kết hợp lexical search và semantic search, hỗ trợ truy vấn tiếng Việt/tiếng Anh thông qua tag canonical và alias dịch.
 - Xây dựng hệ thống gợi ý sách dựa trên hành vi người dùng, kết hợp content-based recommendation, collaborative filtering và fallback theo xu hướng.
 - Tích hợp pipeline xử lý PDF để sinh tóm tắt nội dung, audience và đúng 5 tag học thuật phục vụ tìm kiếm, gợi ý và hiển thị chi tiết sách.
- **Hỗ trợ công tác quản lý:** Tự động hóa thống kê, dashboard, nhắc hạn trả, xử lý quá hạn, hàng chờ đặt trước và thanh toán phí phạt, giúp thủ thư dễ dàng theo dõi vận hành thư viện.
- **Đảm bảo khả năng triển khai và mở rộng:** Xây dựng backend theo mô hình modular monolith, frontend SPA tách biệt và AI service độc lập để thuận tiện triển khai, kiểm thử và tách module trong tương lai.

1.3 Phạm vi

Đề tài sẽ tập trung vào các phạm vi nghiên cứu và phát triển như sau:

- **Về chức năng quản lý:** Hệ thống thực hiện các chức năng cốt lõi bao gồm quản lý ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag, người dùng, vai trò, mượn–trả, đặt trước, hàng chờ, hạn trả, phí phạt, thanh toán, thông báo và dashboard thống kê.
- **Về ứng dụng AI:**
 - Xây dựng hệ thống tìm kiếm kết hợp từ khóa, tag/category alias và vector semantic search; ưu tiên lexical search với truy vấn ngắn để giữ tính chính xác và dùng semantic search để mở rộng khả năng khám phá.
 - Tích hợp hệ thống gợi ý cá nhân hóa dựa trên lịch sử xem, wishlist, mượn sách, đánh giá và dữ liệu vector nội dung.
 - Phát triển mô-đun AI ETL xử lý PDF, làm sạch văn bản, chia chunk, sinh embedding, tóm tắt nội dung, audience và tag.
- **Về nền tảng công nghệ:** Hệ thống gồm frontend React/Vite, backend Spring Boot modular monolith, PostgreSQL/Flyway, Kafka/WebSocket, S3-compatible storage, PayOS và AI service Python/FastAPI/Celery dùng embedding model kết hợp Gemini ở bước reduce metadata.
- **Về phạm vi triển khai:** Sản phẩm của đề tài là hệ thống web thật sự hoàn chỉnh, được kiểm chứng bằng dữ liệu và kịch bản nghiệp vụ phù hợp với quy mô thư viện cấp trường;

một số tích hợp ngoài như PayOS, S3 và Gemini phụ thuộc cấu hình môi trường triển khai.

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

- **Nâng cao trải nghiệm người dùng:** Giúp sinh viên và giảng viên tìm kiếm tài liệu nhanh hơn, xem được tag/tóm tắt nội dung, nhận gợi ý tài liệu liên quan và theo dõi trạng thái mượn–trả/đặt trước trực tuyến.
- **Tối ưu hóa công tác quản lý:** Giảm tải công việc thủ công cho thủ thư thông qua dashboard, scheduler, notification, quản lý phí phạt, thanh toán và các công cụ tra cứu nghiệp vụ.
- **Thúc đẩy chuyển đổi số:** Góp phần xây dựng mô hình thư viện số hiện đại, phù hợp với chủ trương chuyển đổi số trong giáo dục và phát triển năng lực số cho sinh viên.

1.4.2 Ý nghĩa khoa học

- **Ứng dụng AI/NLP vào bài toán thực tế:** Đề tài áp dụng xử lý văn bản PDF, embedding, semantic search và hệ thống gợi ý vào bài toán tìm kiếm tài liệu trong thư viện đại học.
- **Kết hợp mô hình và hệ thống:** Dự án là cầu nối giữa kỹ thuật AI và hệ thống phần mềm hoàn chỉnh: AI service không đứng riêng lẻ mà được nối với catalog, search, recommendation, notification và giao diện người dùng.
- **Đề xuất giải pháp khả thi:** Hệ thống áp dụng hướng hybrid search và hybrid recommendation ở mức phù hợp với dữ liệu và quy mô thư viện theo hướng tối ưu nhất giữa độ chính xác và khả năng khám phá.

1.5 Cấu trúc báo cáo

Ngoài các phần đầu mục như Tóm tắt, Mục lục, Tài Nguyên và Mã Nguồn, Danh mục hình ảnh và bảng biểu, nội dung chính của báo cáo được cấu trúc thành 9 chương như sau:

- **Chương 1 - Giới thiệu:** Trình bày tổng quan về động cơ, mục tiêu, phạm vi và ý nghĩa của đề tài, đồng thời giới thiệu cấu trúc tổng thể của báo cáo.
- **Chương 2 - Kiến thức và công nghệ nền tảng:** Trình bày các cơ sở lý thuyết, kiến thức chuyên ngành và các công nghệ cốt lõi được sử dụng để phát triển dự án.
- **Chương 3 - Công trình liên quan:** Phân tích, so sánh các nghiên cứu và hệ thống tương tự để xác định khoảng trống nghiên cứu mà đề tài giải quyết.
- **Chương 4 - Hệ thống đề xuất:** Mô tả tổng quan hệ thống được đề xuất, bao gồm đối tượng người dùng, yêu cầu chức năng, phi chức năng, yêu cầu dữ liệu và các luồng nghiệp

vụ chính.

- **Chương 5 - Phân tích và Thiết kế:** Trình bày chi tiết quá trình phân tích và thiết kế hệ thống, bao gồm kiến trúc phần mềm, thiết kế cơ sở dữ liệu, thiết kế AI Service và thiết kế giao diện.
- **Chương 6 - Hiện thực hệ thống:** Mô tả quá trình hiện thực frontend, backend và AI service, bao gồm giao diện cho Độc giả/Thủ thư, API nghiệp vụ, pipeline xử lý dữ liệu AI và các tích hợp hạ tầng.
- **Chương 7 - Kiểm thử:** Trình bày chiến lược kiểm thử phân tầng cho frontend, backend và AI service, bao gồm unit/context test, controller/integration test, SQL contract test, AI metadata quality test và live Gemini artifact.
- **Chương 8 - Triển khai hệ thống:** Mô tả quá trình đóng gói Docker, cấu hình NGINX, CI/CD và triển khai lên môi trường VPS thực tế.
- **Chương 9 - Tổng kết:** Tóm tắt kết quả đạt được, nhận xét ưu điểm và hạn chế còn tồn tại của hệ thống.

Chương 2: Kiến thức và công nghệ nền tảng

Chương này trình bày tổng quan về các kiến thức và các lĩnh vực công nghệ nền tảng được vận dụng để xây dựng "Hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách".

2.1 Các Lĩnh vực Công nghệ Chính

Dự án được định hướng phát triển dựa trên kiến trúc phần mềm hiện đại, trong đó các thành phần chính được phân tách rõ ràng để thuận lợi cho việc bảo trì và mở rộng sau này.

- **Công nghệ phía Server (Back-End):** Đây là thành phần trung tâm, chịu trách nhiệm xử lý toàn bộ logic nghiệp vụ của hệ thống, cung cấp các giao diện lập trình ứng dụng (API), quản lý dữ liệu mượn-trả, và thực hiện các tác vụ thống kê.
- **Công nghệ phía Client (Front-End):** Thành phần này đảm nhiệm việc xây dựng giao diện người dùng, giúp người dùng cuối (thủ thư, bạn đọc) tương tác với hệ thống một cách trực quan và thân thiện trên nền tảng web.
- **Hệ quản trị Cơ sở dữ liệu:** Hệ thống sử dụng một hệ quản trị cơ sở dữ liệu quan hệ để tổ chức, lưu trữ và truy xuất toàn bộ dữ liệu của thư viện, bao gồm thông tin sách, người dùng, và lịch sử giao dịch.
- **Trí tuệ Nhân tạo (AI) và Học máy (Machine Learning):**
 - **Hệ thống gợi ý (Recommendation System):** Trọng tâm của dự án là áp dụng mô hình **Hybrid**, một phương pháp kết hợp ưu điểm của hai kỹ thuật chính:
 1. **Lọc dựa trên nội dung (Content-Based Filtering):** Gợi ý tài liệu dựa trên sự tương đồng về thuộc tính và nội dung của chúng.
 2. **Lọc cộng tác (Collaborative Filtering):** Đề xuất tài liệu dựa trên hành vi và sở thích của các nhóm người dùng tương tự nhau.
 - **Xử lý Ngôn ngữ Tự nhiên (NLP):** Các kỹ thuật NLP được ứng dụng để nâng cao trải nghiệm người dùng, bao gồm: phát triển công cụ **tìm kiếm ngữ nghĩa** cho tiếng Việt và tích hợp khả năng **tự động tóm tắt nội dung sách**.

2.2 Nền tảng Kiến thức Khoa học

Việc triển khai các lĩnh vực công nghệ trên đòi hỏi các nền tảng kiến thức sau:

- **Kiến trúc và Phát triển Hệ thống:**

- Hiểu biết về kiến trúc **Client-Server** và nguyên lý thiết kế **RESTful API** để đảm bảo sự giao tiếp hiệu quả giữa Front-End và Back-End.
- Kiến thức về thiết kế và mô hình hóa cơ sở dữ liệu quan hệ (ví dụ: ERD) để xây dựng một cấu trúc lưu trữ tối ưu.
- **Khoa học Dữ liệu và Trí tuệ Nhân tạo:**
 - **Nguyên lý các thuật toán gợi ý:** Nắm vững bản chất, ưu và nhược điểm của các phương pháp Content-Based, Collaborative Filtering, và cách chúng bổ trợ cho nhau trong một mô hình Hybrid.
 - **Các kỹ thuật xử lý ngôn ngữ tự nhiên:** Có kiến thức về các phương pháp tiền xử lý văn bản, mô hình hóa ngôn ngữ (word embedding), và các thuật toán đo lường độ tương đồng ngữ nghĩa (ví dụ: **cosine similarity**).

Kết chương

Nội dung chương này đã hệ thống hóa các lĩnh vực công nghệ và nền tảng kiến thức khoa học cần thiết cho dự án. Các thành phần công nghệ chính bao gồm **Back-End**, **Front-End**, **Cơ sở dữ liệu quan hệ** và các module **Trí tuệ nhân tạo**. Trọng tâm của dự án nằm ở việc áp dụng các kiến thức về **Hệ thống gợi ý Hybrid** và **Xử lý Ngôn ngữ Tự nhiên** để tạo ra một sản phẩm thông minh và khác biệt.

Các nền tảng kiến thức đã trình bày bao quát các yêu cầu lý thuyết cần thiết để triển khai dự án, từ việc xây dựng một ứng dụng web hoàn chỉnh đến tích hợp các tính năng AI nâng cao theo đúng mục tiêu đề ra.

Chương 3: Công trình liên quan

Chương này tập trung phân tích các công trình nghiên cứu khoa học và các hệ thống phần mềm đã có liên quan đến lĩnh vực thư viện số và hệ thống gợi ý. Từ đó, chương sẽ chỉ ra những khoảng trống nghiên cứu và định vị hướng đi của đề tài.

3.1 Các công trình nghiên cứu khoa học

Qua việc tổng hợp các công trình nghiên cứu, có thể thấy các hướng tiếp cận phổ biến trong việc xây dựng hệ thống gợi ý sách cho thư viện có những đặc điểm sau:

- **Ưu điểm và xu hướng chung:**

- Hầu hết các mô hình gợi ý đều theo hướng **hybrid**, kết hợp giữa Lọc dựa trên nội dung (Content-Based) và Lọc cộng tác (Collaborative Filtering) để cân bằng giữa độ cá nhân hóa và khả năng khai thác hành vi người dùng.
- Một số nghiên cứu cải tiến kết quả bằng cách thêm các cơ chế xếp hạng (ranking) hoặc gợi ý theo cụm người dùng (clustering), đặc biệt hiệu quả trong môi trường dữ liệu vừa và nhỏ.
- Việc đánh giá mô hình thường dựa trên dữ liệu thực tế từ các thư viện, giúp kiểm chứng rõ ràng hiệu quả của thuật toán.

- **Hạn chế phổ biến:**

- Nhiều công trình tập trung sâu vào phần mô hình thuật toán nhưng ít chú trọng đến việc triển khai thành một hệ thống hoàn chỉnh với đầy đủ backend và frontend.
- Các hướng đi hiện đại như tìm kiếm ngữ nghĩa (semantic search) hay các mô hình Transformer tuy tiềm năng nhưng đòi hỏi chi phí tính toán và nguồn lực lớn, vượt quá khả năng của nhiều dự án quy mô nhỏ.

3.2 Các hệ thống và phần mềm tương tự

Hiện nay, có nhiều hệ thống quản lý thư viện điện tử đã được triển khai rộng rãi, tiêu biểu có thể kể đến:

- **Opac-HCMUT-VNU:** Là hệ thống của Đại học Quốc Gia TP.HCM, đáp ứng tốt các nghiệp vụ tra cứu và mượn-trả cơ bản. Tuy nhiên, chức năng tìm kiếm vẫn chủ yếu dựa vào từ khóa, lọc và chưa tích hợp tính năng gợi ý thông minh, tìm kiếm theo ngữ nghĩa.
- **Koha:** Là một phần mềm quản lý thư viện mã nguồn mở rất phổ biến, cung cấp đầy đủ chức năng quản lý và báo cáo. Mặc dù vậy, giao diện của Koha đã cũ, chưa hỗ trợ gợi ý tự

động và chưa có cơ chế phân tích hành vi người dùng.

- **Aleph (Ex Libris):** Đây là một hệ thống thương mại quy mô lớn, tích hợp nhiều cơ sở dữ liệu học thuật. Nhược điểm chính của hệ thống này là chi phí triển khai cao và không phù hợp cho các thư viện quy mô nhỏ như cấp khoa hoặc trường đại học nhỏ.

3.3 Kết chương và xác định khoảng trống nghiên cứu

Từ việc phân tích các công trình khoa học và hệ thống thực tế, có thể nhận thấy một số **khoảng trống (gaps)** quan trọng mà đề tài này hướng đến giải quyết:

1. **Khoảng trống giữa Mô hình và Thực thi:** Nhiều nghiên cứu tập trung vào việc xây dựng các mô hình gợi ý phức tạp nhưng lại bỏ qua việc triển khai chúng thành một hệ thống phần mềm hoàn chỉnh, có tính ứng dụng thực tế. Đa số các mô hình hiện nay tập trung nhiều vào thuật toán gợi ý, trong khi ít chú trọng đến việc triển khai thành hệ thống hoàn chỉnh.
2. **Khoảng trống về Trải nghiệm Người dùng:** Các hệ thống hiện tại, kể cả mã nguồn mở như Koha, thường có giao diện cũ, chưa tối ưu cho các thiết bị hiện đại và thiếu các tính năng thông minh giúp nâng cao trải nghiệm tìm kiếm, khám phá tài liệu của người dùng.
3. **Khoảng trống về Tính khả thi:** Các giải pháp gợi ý tiên tiến thường yêu cầu nguồn dữ liệu lớn hoặc xử lý phức tạp, vượt quá phạm vi và nguồn lực của một sinh viên thực hiện.

Hướng đi của dự án: Đề tài này không đặt nặng việc phát triển một mô hình AI quá chuyên sâu. Thay vào đó, dự án tập trung vào việc **bắc cầu cho các khoảng trống trên** bằng cách:

- Xây dựng một **hệ thống hoàn chỉnh** với kiến trúc frontend-backend tách biệt, dễ bảo trì và mở rộng.
- Ưu tiên **trải nghiệm người dùng** với giao diện hiện đại và tích hợp các tính năng gợi ý, tìm kiếm thông minh một cách liền mạch.
- Áp dụng một **giải pháp AI khả thi** (mô hình hybrid cơ bản) để tạo ra một sản phẩm hoạt động ổn định, có giá trị thực tiễn và phù hợp với bối cảnh của một đồ án tốt nghiệp.

Chương 4: Hệ thống đề xuất

4.1 Mô tả hệ thống

Hệ thống được đề xuất là một **hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm, gợi ý và khai thác nội dung tài liệu PDF**. Mục tiêu là xây dựng một nền tảng có đủ nghiệp vụ thư viện: quản lý ấn phẩm, bản sao, người dùng, mượn-trả, đặt trước, phí phạt, thông báo và dashboard vận hành. Trên nền nghiệp vụ đó, AI được dùng ở những điểm thật sự có giá trị như làm giàu metadata từ PDF, tìm kiếm ngữ nghĩa và gợi ý tài liệu theo hành vi người dùng.

Hệ thống được thiết kế để phục vụ hai nhóm người dùng chính, mỗi nhóm có vai trò và nhu cầu riêng biệt:

- **Bạn đọc (Sinh viên, Giảng viên, Nghiên cứu viên):** Là nhóm người dùng trực tiếp khai thác tài nguyên thư viện. Họ cần tìm kiếm tài liệu nhanh, xem tình trạng bản sao, đọc tóm tắt AI, thêm sách vào wishlist, đánh giá sách, gửi yêu cầu mượn, theo dõi sách đang mượn, đặt trước sách, xem phí phạt và nhận thông báo kịp thời.
- **Thủ thư (Người quản lý thư viện):** Là nhóm người dùng vận hành hệ thống. Thủ thư quản lý ấn phẩm, tác giả, nhà xuất bản, danh mục, tag, bản sao vật lý, upload bìa và PDF, xử lý mượn-trả tại quầy, xác nhận nhận sách, quản lý đặt trước, ghi nhận sách hư/mất, xử lý phí phạt bằng tiền mặt hoặc PayOS và theo dõi dashboard thống kê.

4.2 Hướng tiếp cận thiết kế

Hệ thống Quản lý Thư viện được thiết kế theo phương pháp “từ dưới lên”. Backend được tổ chức theo modular monolith, frontend là React SPA, còn AI service là một service riêng xử lý PDF, semantic search và recommendation.

Quá trình thiết kế được thực hiện theo các bước sau:

1. Xác định các module cốt lõi của hệ thống như auth, user, catalog, circulation, recommendation và AI service.
2. Xây dựng và kiểm thử từng module theo trách nhiệm riêng: xác thực, quản lý ấn phẩm, quản lý bản sao, giao dịch mượn-trả, thông báo, wishlist, rating, tìm kiếm và gợi ý.
3. Tích hợp các module thành các luồng nghiệp vụ đầy đủ, ví dụ: tạo ấn phẩm, upload PDF, AI xử lý metadata, người dùng tìm kiếm, gửi yêu cầu mượn, thủ thư xác nhận và hệ thống gửi thông báo.
4. Lặp lại quy trình này để hoàn thiện frontend, backend, AI service và các tích hợp ngoài như S3-compatible storage, Kafka/WebSocket và PayOS.

4.3 Yêu cầu

4.3.1 Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống được đặc tả chi tiết trong bảng dưới đây, phân loại theo từng nhóm chức năng và người dùng tương tác.

Bảng 4.1: Bảng đặc tả yêu cầu hệ thống

ID	Tên yêu cầu	Loại	Mô tả chi tiết
R1	Quản lý thực thể dữ liệu	Dữ liệu	Hệ thống phải lưu trữ và quản lý các nhóm dữ liệu cốt lõi: ấn phẩm, bản sao, người dùng, giao dịch mượn-trả, đặt trước, phí phạt, đánh giá, wishlist, thông báo và dữ liệu phục vụ AI.
R2	Phân biệt Ấn phẩm và Bản sao	Dữ liệu	Hệ thống phải phân biệt giữa ấn phẩm và bản sao vật lý . Mỗi bản sao có barcode riêng, trạng thái riêng, tình trạng sách và vị trí kệ để thủ thư có thể quản lý lưu thông chính xác.
R3	Thuộc tính Ấn phẩm	Dữ liệu	Mỗi ấn phẩm phải có metadata cơ bản như ISBN, tiêu đề, tác giả, nhà xuất bản, danh mục, tag, ngôn ngữ, năm xuất bản, mô tả, ảnh bìa và file PDF nếu có. Hệ thống cũng lưu summary, nhóm đọc giả mục tiêu và tag do AI sinh ra sau khi xử lý tài liệu.
R4	Quản lý vai trò người dùng	Chức năng	Hệ thống phải hỗ trợ xác thực và phân quyền theo vai trò, tối thiểu gồm Bạn đọc và Thủ thư . Bạn đọc được đăng ký, đăng nhập, quản lý hồ sơ cá nhân; thủ thư được truy cập các màn hình quản lý nghiệp vụ.
R5	Nghiệp vụ mượn-trả và phí phạt	Nghiệp vụ	Hệ thống phải hỗ trợ bạn đọc gửi yêu cầu mượn, thủ thư cho mượn trực tiếp hoặc xác nhận nhận sách, trả sách bằng barcode, ghi nhận sách hư/mất, tự tính phí quá hạn hoặc phí sự cố và xử lý thanh toán bằng tiền mặt hoặc PayOS.

Bảng 4.1 – tiếp theo

ID	Tên yêu cầu	Loại	Mô tả chi tiết
R6	Tìm kiếm hybrid	Chức năng	Hệ thống phải cung cấp tìm kiếm catalog theo keyword, tác giả, danh mục, tag, ngôn ngữ, năm xuất bản, chi nhánh và tình trạng bản sao; đồng thời hỗ trợ tìm kiếm ngữ nghĩa trên nội dung PDF thông qua AI service và pgvector.
R7	Chức năng Đặt trước	Chức năng	Bạn đọc phải có khả năng đặt trước sách khi cần. Hệ thống lưu hàng chờ đặt trước, cho phép bạn đọc hủy đặt trước, thủ thư tra cứu yêu cầu và xác nhận nhận sách khi bản sao sẵn sàng.
R8	Gợi ý, wishlist và đánh giá	Chức năng	Hệ thống phải hỗ trợ gợi ý sách cá nhân hóa dựa trên hành vi người dùng, có fallback khi dữ liệu còn ít; đồng thời cho phép bạn đọc thêm sách vào wishlist, đánh giá, bình luận và tương tác với review.
R9	Hệ thống thông báo tự động	Chức năng	Hệ thống phải gửi thông báo cho người dùng về các sự kiện quan trọng như yêu cầu mượn, xác nhận nhận sách, sách sắp đến hạn, sách quá hạn, phí phạt, đặt trước và phản hồi review. Thông báo được lưu trong hệ thống và đẩy realtime qua WebSocket.

4.3.2 Yêu cầu phi chức năng

Hệ thống cần đảm bảo các yêu cầu phi chức năng trọng yếu sau:

- **Tính khả dụng (Usability):** Giao diện người dùng phải nhất quán, trực quan và dễ học cho cả bạn đọc và thủ thư. Frontend cần hỗ trợ các luồng chính như tìm kiếm, xem chi tiết sách, đặt trước, theo dõi sách đang mượn, phí phạt, thông báo, quản lý ấn phẩm và xử lý mượn–trả trên các kích thước màn hình phổ biến.
- **Hiệu năng (Performance):** Các chức năng tương tác phải có thời gian phản hồi nhanh trong điều kiện tải thông thường. Các truy vấn catalog cần được phân trang; semantic search và recommendation cần giới hạn số lượng kết quả, dùng connection pool và timeout để tránh ảnh hưởng đến backend chính.
- **Độ tin cậy (Reliability):** Hệ thống phải hoạt động ổn định ngay cả khi một dịch vụ phụ trợ gặp lỗi. Khi AI service hoặc Gemini lỗi, hệ thống cần ghi nhận trạng thái xử lý, có fallback cho metadata/gợi ý và không làm nghẽn các nghiệp vụ chính như tra cứu, mượn–trả hoặc thanh toán.

- **Bảo mật (Security):** Hệ thống cần có cơ chế xác thực và phân quyền rõ ràng bằng JWT và kiểm tra vai trò ở backend. Các API quản trị chỉ dành cho thủ thư; callback từ AI service và webhook thanh toán cần được kiểm tra chữ ký hoặc cơ chế xác thực tương ứng; upload file phải đi qua đường dẫn có kiểm soát như presigned URL.
- **Khả năng bảo trì và mở rộng (Maintainability & Scalability):** Mã nguồn phải được tổ chức rõ ràng theo frontend, backend và AI service. Backend theo modular monolith giúp tách ranh giới nghiệp vụ; AI service tách riêng giúp mở rộng pipeline PDF, semantic search và recommendation mà không làm phức tạp phần nghiệp vụ thư viện.

4.3.3 Yêu cầu dữ liệu

Hệ thống yêu cầu và quản lý các nhóm dữ liệu chính sau:

- **Dữ liệu lõi (Core Data):** Bao gồm thông tin ấn phẩm, bản sao, barcode, tác giả, nhà xuất bản, danh mục, tag, file bìa, file PDF và thông tin tài khoản người dùng.
- **Dữ liệu nghiệp vụ (Transactional Data):** Gồm các bản ghi yêu cầu mượn, xác nhận nhận sách, trả sách, đặt trước, phí phạt, thanh toán tiền mặt, thanh toán PayOS, trạng thái bản sao và thông báo.
- **Dữ liệu hành vi và AI (Behavioral/AI Data):** Gồm lịch sử tìm kiếm, lượt xem chi tiết sách, wishlist, rating, lịch sử mượn, vector embedding của chunk PDF, summary AI, audience, tag AI và trạng thái ETL. Đây là nguồn dữ liệu phục vụ semantic search và recommendation.

Kết chương

Chương này đã trình bày tổng quan hệ thống quản lý thư viện trực tuyến tích hợp AI. Hệ thống được định hướng như một sản phẩm hoàn chỉnh, trong đó nghiệp vụ thư viện vẫn là phần lõi, còn AI được dùng để làm giàu dữ liệu, hỗ trợ tìm kiếm ngữ nghĩa và gợi ý tài liệu.

Phần trọng tâm của chương đã đặc tả các yêu cầu cốt lõi của hệ thống, từ quản lý ấn phẩm, bản sao, người dùng, mượn–trả, đặt trước, phí phạt và thông báo cho đến các chức năng thông minh như semantic search, recommendation, wishlist và rating. Đồng thời, các **yêu cầu phi chức năng** như tính khả dụng, hiệu năng, độ tin cậy, bảo mật và khả năng bảo trì cũng được xác định theo đúng kiến trúc frontend–backend–AI service hiện tại.

Những yêu cầu này tạo thành một nền tảng vững chắc, là cơ sở để tiến hành quá trình phân tích và thiết kế chi tiết hệ thống sẽ được trình bày ở Chương 5.

Chương 5: Phân tích và Thiết kế

5.1 Phân tích

5.1.1 Mô hình hóa tương tác (Use-case Diagram)

Phần phân tích use case được xây dựng theo phạm vi hệ thống đã hiện thực. Hệ thống có các tác nhân chính gồm **Khách**, **Bạn đọc**, **Thủ thư** và **Quản trị viên**; ngoài ra còn có các hệ thống phụ trợ như **Google OAuth**, **PayOS**, **Dịch vụ Email** và **Gemini API**. Để giữ báo cáo trong giới hạn số trang, phần này trình bày use case dưới dạng bảng đối chiếu thay cho các sơ đồ module chi tiết. Ranh giới chức năng chính của hệ thống gồm nhóm xác thực/tài khoản, nhóm tra cứu tài liệu, nhóm nghiệp vụ bạn đọc, nhóm nghiệp vụ thủ thư, nhóm quản trị và nhóm AI tự động. Các quan hệ nghiệp vụ chính gồm: đăng ký bao gồm xác thực email; tìm kiếm có thể mở rộng sang semantic search; mượn/đặt trước bao gồm kiểm tra điều kiện mượn; upload PDF bao gồm xử lý AI; trả sách và báo hư/mất bao gồm tính phí và gửi thông báo.

Bảng 5.1: Đối chiếu nhóm Use Case với hiện thực hệ thống

Nhóm	Use case chính	Hiện thực tương ứng
Xác thực và tài khoản	Đăng ký, xác thực email, đăng nhập thường, đăng nhập Google, refresh token, quên/đặt lại mật khẩu, hồ sơ, avatar, đổi mật khẩu, thông báo	Auth/User modules, OAuth2 Google, JWT/refresh token, email verification/reset password, profile API và notification API.
Tra cứu và khám phá tài liệu	Trang chủ, tìm kiếm catalog, semantic search, lọc danh mục/tag, chi tiết tài liệu, bản sao, đánh giá, sách tương tự, thống kê công khai	Public pages và catalog/recommendation API: /api/v1/publications/search, /api/v1/ai/semantic-search, /api/v1/publications/{id}/items, /ratings, /similar.
Chức năng bạn đọc	Mượn, đặt trước, hủy đặt trước, theo dõi sách đang mượn, lịch sử mượn trả, phí phạt, PayOS, wishlist, rating, gợi ý cá nhân hóa	Reader pages, circulation/reservation/fine APIs, PayOS payment, wishlist/rating APIs và recommendation API.

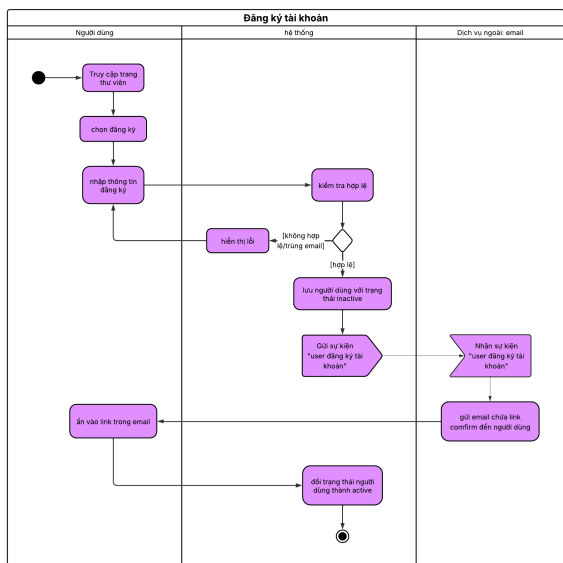
Tiếp tục ở trang sau

Bảng 5.1 – tiếp tục từ trang trước

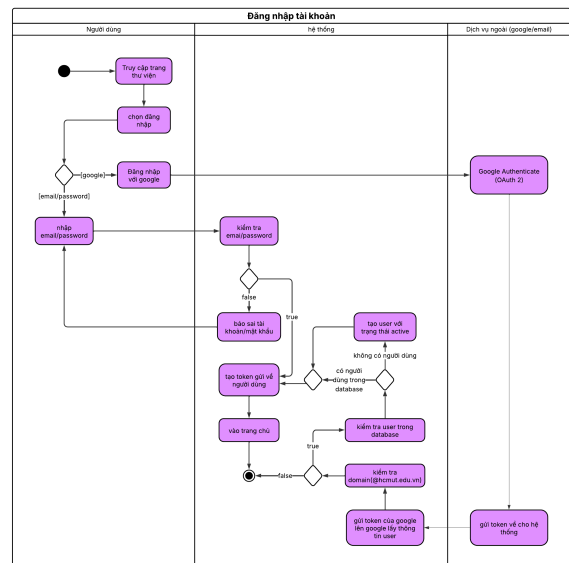
Nhóm	Use case chính	Hiện thực tương ứng
Chức năng thủ thư	Quản lý ấn phẩm, bản sao, upload bìa/PDF, AI ETL, xử lý yêu cầu mượn, mượn trực tiếp, trả sách, báo hư/mất, ghi chú giao dịch, thu tiền mặt	Librarian pages, catalog/item APIs, presigned upload, backend AI gateway, circulation transaction APIs, fine cash payment và notification/email events.
Chức năng quản trị	Quản lý thủ thư, quản lý người dùng, khóa/mở khóa, xác minh tài khoản, chính sách mượn trả, nhật ký hệ thống	Admin pages và admin APIs: librarian/user management, circulation policy, audit log và account verification.
AI và xử lý tự động	Semantic search, recommendation, sách tương tự, xử lý PDF, trích xuất text, embedding, sinh summary/tag/audience, ghi nhận tương tác, realtime/email notification	AI service FastAPI/Celery, RabbitMQ, Gemini API, PostgreSQL pgvector, interaction tracking, Kafka/email/WebSocket notification.

5.1.2 Mô hình hóa luồng xử lý (Activity Diagram)

Activity Diagram được dùng để mô tả các luồng xử lý chính sau khi đã xác định use case. Phần mô hình hóa tập trung vào các nghiệp vụ có nhiều bước và có tương tác giữa người dùng, giao diện, backend và dịch vụ phụ trợ: xác thực, tra cứu, mượn–trả, quản lý sách, đánh giá, xử lý hàng chờ và gợi ý sách.

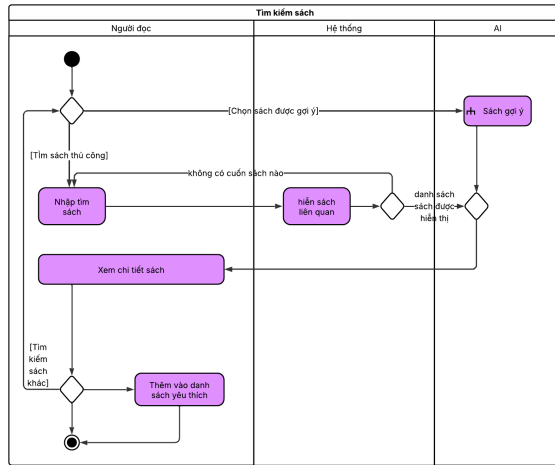


(a) Đăng ký tài khoản

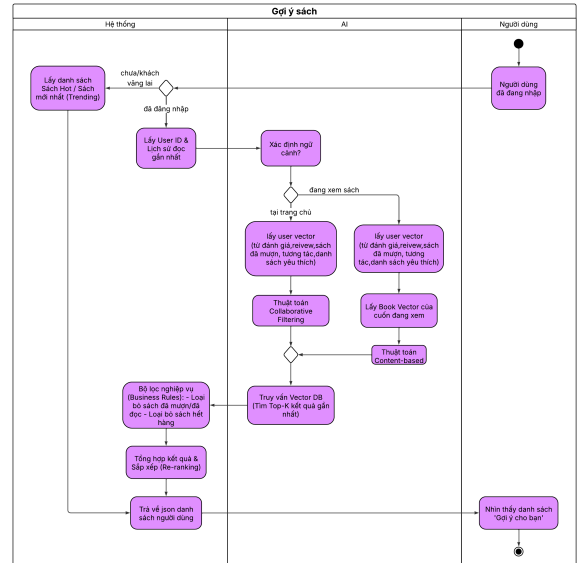


(b) Đăng nhập

Hình 5.1: Activity Diagram cho xác thực người dùng

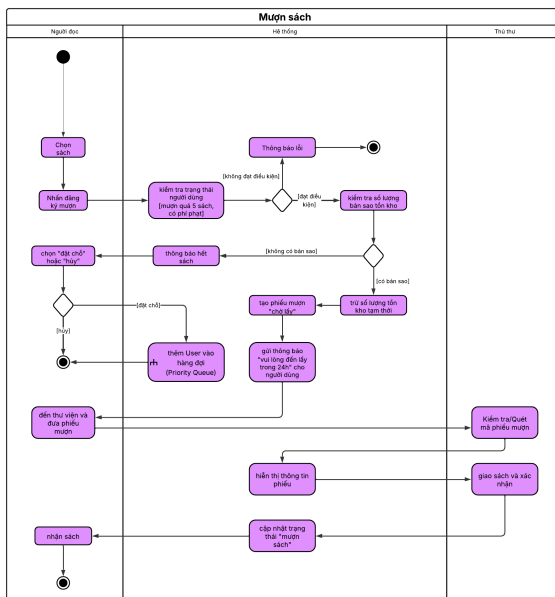


(a) Tìm kiếm sách

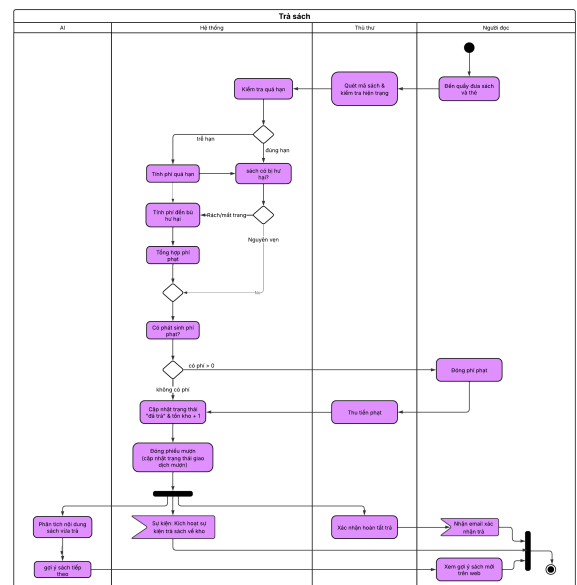


(b) Gợi ý sách

Hình 5.2: Activity Diagram cho tra cứu và gợi ý tài liệu

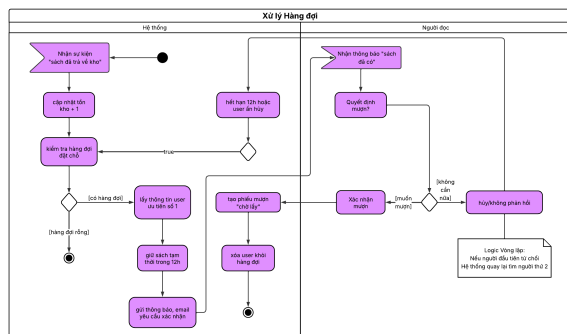


(a) Mượn sách

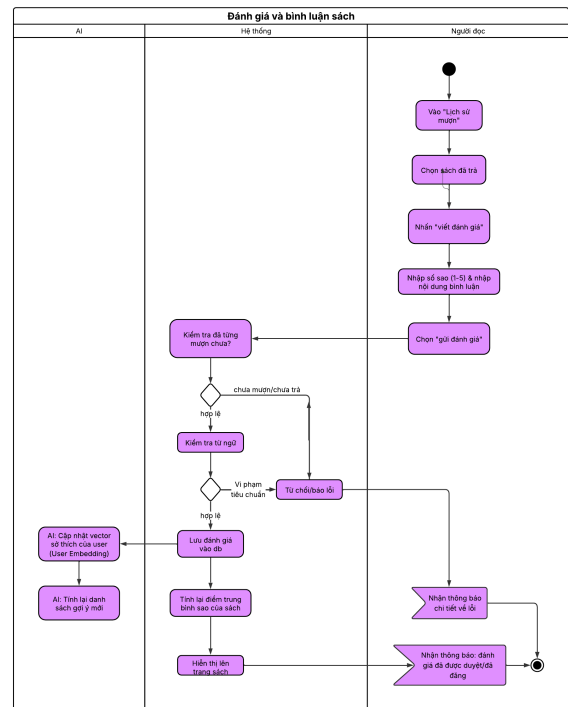


(b) Trả sách

Hình 5.3: Activity Diagram cho nghiệp vụ mượn-trả

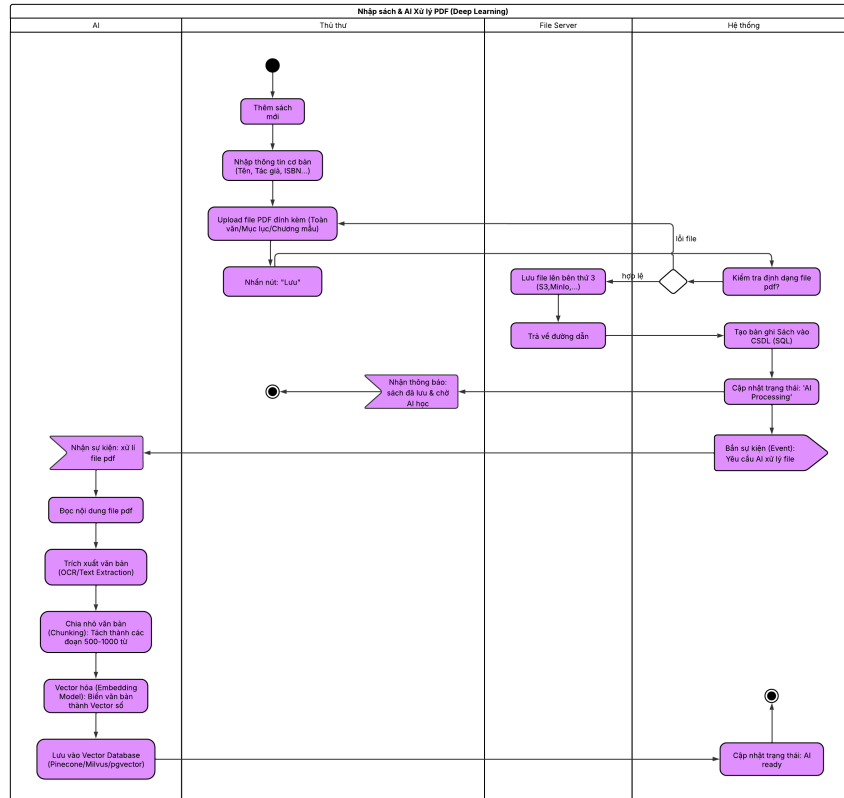


(a) Xử lý hàng chờ đặt trước



(b) Đánh giá và bình luận

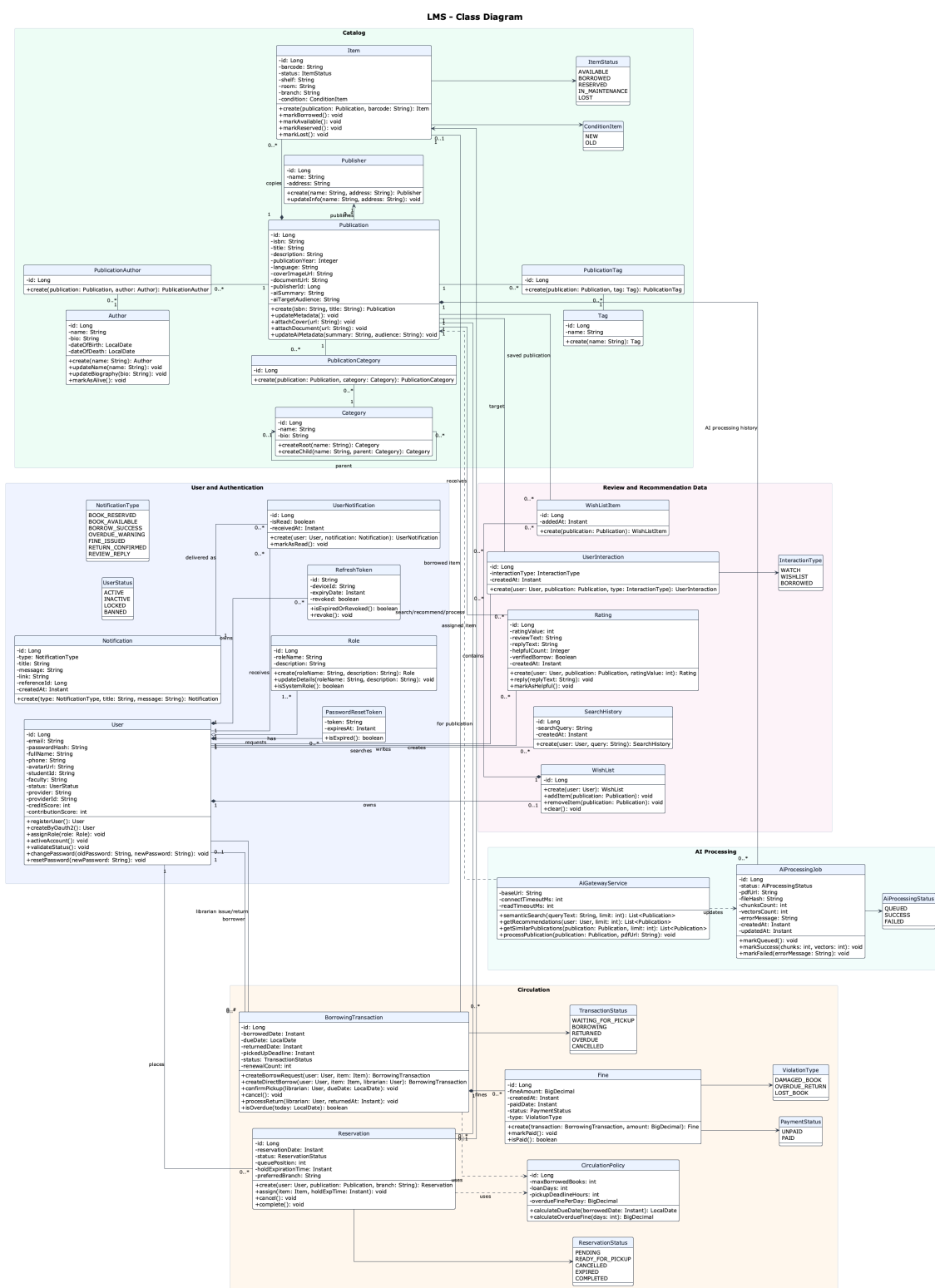
Hình 5.4: Activity Diagram cho đặt trước và tương tác sau mượn



Hình 5.5: Activity Diagram cho nghiệp vụ thêm sách

5.1.3 Mô hình lớp (Class diagram)

Class diagram trong Hình 5.6 mô tả các lớp domain chính của hệ thống theo hai cụm lớn: Core Domain và Extended Services. Trong đó, Core Domain bao gồm User, Catalog và Circulation; Extended Services bao gồm Reader Data, AI và Support. Sơ đồ tập trung vào entity, thuộc tính nghiệp vụ quan trọng và quan hệ giữa các lớp, giúp liên kết phân phân tích use case với thiết kế dữ liệu và cấu trúc backend.



Hình 5.6: Class diagram tổng quát của hệ thống

5.1.4 Công nghệ được sử dụng

Bảng 5.2: Công nghệ được sử dụng trong hệ thống

Phần	Công nghệ	Vai trò trong hệ thống
Frontend	React, TypeScript, Vite	Xây dựng SPA, chia trang theo vai trò và phát triển giao diện có kiểm soát kiểu dữ liệu.
Frontend	Tailwind CSS, lucide-react, sonner	Thiết kế giao diện, icon, toast notification và trạng thái tương tác.
Frontend	React Router/HashRouter	Điều hướng public/user/librarian/admin page và bảo vệ route theo vai trò.
Frontend	Context API	Quản lý trạng thái dùng chung như đăng nhập, ngôn ngữ, theme, notification và upload.
Frontend	Axios, STOMP/SockJS	Gọi REST API, gắn JWT/refresh token và nhận thông báo realtime qua WebSocket.
Frontend	Jest, Testing Library, Ng-inx	Kiểm thử frontend và phục vụ bản build static trong container production.
Backend	Java 21, Spring Boot	Hiện thực API nghiệp vụ theo mô hình modular monolith.
Backend	Spring Security, JWT, OAuth2	Xác thực, phân quyền, refresh token và đăng nhập Google.
Backend	Spring Data JPA, Flyway	Truy cập dữ liệu, quản lý entity và migration cơ sở dữ liệu.
Backend	PostgreSQL, pgvector	Lưu dữ liệu nghiệp vụ, metadata AI và vector phục vụ semantic search.
Backend	Kafka, WebSocket	Phát sự kiện và đẩy thông báo realtime đến người dùng.
Backend	RabbitMQ, S3-compatible Storage	Gửi job xử lý AI và lưu trữ ảnh bìa/PDF thông qua pre-signed URL.
Backend	PayOS, Gmail SMTP, OpenAPI/Swagger	Thanh toán phí phạt, gửi email và tài liệu hóa API.
Backend	JUnit, Mockito, Testcontainers	Kiểm thử unit/controller và integration với PostgreSQL thật.
Backend	Docker, Docker Compose	Đóng gói backend và triển khai cùng các dịch vụ phụ trợ.
AI Service	Python, FastAPI	Xây dựng API cho semantic search, similar books, recommendation và health check.
AI Service	Celery, RabbitMQ	Xử lý job AI bất đồng bộ sau khi thủ thư upload PDF.
AI Service	PyMuPDF	Trích xuất văn bản từ PDF trước khi làm sạch và chia chunk.
AI Service	SentenceTransformer/SBERT	Sinh embedding tiếng Việt cho chunk tài liệu và truy vấn tìm kiếm.
AI Service	Gemini API	Sinh summary, audience và tag học thuật ở bước metadata enrichment.

Phần	Công nghệ	Vai trò trong hệ thống
AI Service	PostgreSQL, pgvector	Lưu vector, ETL run, search log và dữ liệu phục vụ truy vấn nearest-neighbor.
AI Service	ALS implicit feedback, content-based, trending fallback	Tạo gợi ý sách cá nhân hóa và fallback khi dữ liệu tương tác còn ít.
AI Service	pytest, Docker	Kiểm thử AI Service và đóng gói FastAPI/Celery Worker để chạy tách biệt trong production.

Các công nghệ trong hệ thống được chọn theo đúng phạm vi hiện thực: frontend phục vụ trải nghiệm người dùng, backend xử lý nghiệp vụ thư viện và AI Service xử lý các tác vụ tìm kiếm/gợi ý thông minh. Bảng dưới đây gom các công nghệ chính đã sử dụng trong toàn hệ thống.

5.2 Ứng dụng AI vào hệ thống

Trong hệ thống LMS, AI không được đặt vào như một chức năng phụ để “làm đẹp” sản phẩm. AI được thiết kế như một **lớp năng lực hỗ trợ khám phá tri thức**: đọc nội dung PDF, hiểu ý định tìm kiếm của bạn đọc và đề xuất tài liệu phù hợp với hành vi sử dụng thật.

Điểm quan trọng nhất trong thiết kế là **AI không thay thế nghiệp vụ thư viện**. Các nghiệp vụ như mượn-trả, đặt trước, quản lý bản sao, phí phạt, thanh toán và phân quyền vẫn do backend Spring Boot kiểm soát. AI Service chỉ xử lý các bài toán phù hợp với ML/NLP, sau đó trả về **metadata bổ trợ**, **danh sách publication_id** hoặc **chiến lược gợi ý**; backend luôn là nơi lấy dữ liệu cuối cùng trước khi trả về frontend.

- **Module 1 – Xử lý PDF file**: biến file PDF do thủ thư upload thành dữ liệu có thể tìm kiếm và gợi ý.
- **Module 2 – Hỗ trợ tìm kiếm ngữ nghĩa**: giúp bạn đọc tìm theo ý định, không chỉ theo từ khóa chính xác.
- **Module 3 – Gợi ý cá nhân hoá**: kết hợp nội dung sách, hành vi người dùng và fallback nghiệp vụ để đề xuất sách có thể mượn.

Dự án chọn mô hình **AI Satellite Service**: backend Java giữ nghiệp vụ và giao dịch; AI Service Python/FastAPI xử lý workload nặng như đọc PDF, sinh embedding, truy vấn pgvector và recommendation. Cách tách này giúp hệ thống vừa **giữ lõi nghiệp vụ ổn định**, vừa **tận dụng hệ sinh thái AI của Python**.

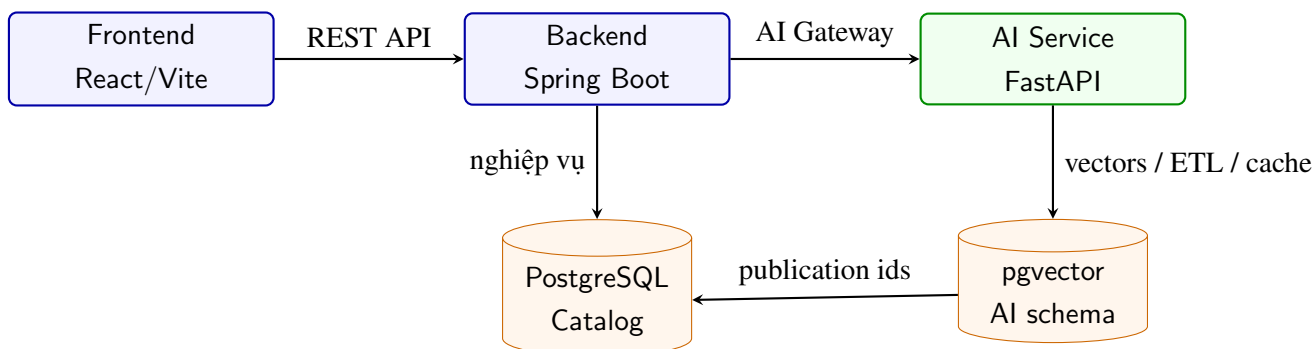
Về mặt giá trị sử dụng, AI giải quyết ba điểm yếu thường gặp của hệ thống thư viện truyền thống. Thứ nhất, nhiều tài liệu chỉ được mô tả bằng tiêu đề và vài trường metadata, trong khi nội dung thật nằm sâu trong PDF; pipeline AI giúp biến phần nội dung đó thành dữ liệu có thể tìm kiếm. Thứ hai, người đọc thường tìm bằng nhu cầu học tập thay vì từ khóa chính xác; semantic search giúp hệ thống hiểu gần đúng ý định truy vấn. Thứ ba, danh mục thư viện càng lớn thì

người dùng càng khó chọn sách phù hợp; recommendation dùng hành vi đọc, wishlist, mượn và rating để rút ngắn quá trình khám phá tài liệu.

Vấn đề	Thiết kế AI trong hệ thống	Giá trị đạt được
Tài liệu PDF khó khai thác	Trích xuất nội dung, làm sạch, chia chunk, sinh embedding và summary/tag.	Sách không chỉ được tìm bằng metadata nhập tay mà còn bằng nội dung thật.
Truy vấn người dùng không chính xác	Mã hóa query thành vector và so khớp với vector chunk trong pgvector.	Người đọc có thể tìm theo ý định như “sách nhập môn để hiểu” hoặc “tài liệu về chuẩn hóa dữ liệu”.
Khó chọn sách phù hợp	Kết hợp content-based, ALS implicit feedback và trending fallback.	Gợi ý vẫn hoạt động khi người dùng mới, sách mới hoặc dữ liệu tương tác còn ít.
AI có thể lỗi hoặc hết quota	Backend giữ nghiệp vụ chính; AI có timeout, retry, fallback và trạng thái xử lý rõ ràng.	Lỗi AI không làm hỏng catalog, mượn-trả, đặt trước hoặc thanh toán.

Bảng 5.3: Giá trị thiết kế của lớp AI trong hệ thống

Một nguyên tắc quan trọng là **AI chỉ làm giàu dữ liệu, không tự quyết định nghiệp vụ**. Ví dụ, semantic search có thể trả danh sách `publication_id` phù hợp, nhưng backend vẫn kiểm tra sách còn bản sao hay không, người dùng có quyền mượn hay không và dữ liệu nào được phép hiển thị. Tương tự, recommendation chỉ đề xuất ứng viên; trạng thái cuối cùng của sách, phí phạt, đặt trước và phân quyền vẫn nằm trong các module nghiệp vụ của backend. Nhờ vậy, hệ thống tận dụng được AI nhưng không đánh đổi tính nhất quán dữ liệu.



Hình 5.7: Trách nhiệm giữa frontend, backend và AI service

Thành phần	Trách nhiệm chính	Điểm nhấn thiết kế
Frontend	Upload tài liệu, gửi truy vấn, hiển thị trạng thái AI và kết quả đã chuẩn hóa.	Không gọi trực tiếp model AI; mọi request đi qua backend.
Backend	Xác thực, phân quyền, lưu catalog, quản lý bản sao/giao dịch và lấy metadata cuối cùng.	Là nguồn sự thật nghiệp vụ; AI không được quyết định trạng thái mượn-trả.
AI Service	Xử lý PDF, sinh embedding, tìm kiếm ngữ nghĩa, tạo metadata AI và recommendation.	Chỉ trả kết quả suy luận; backend quyết định dữ liệu hiển thị cuối cùng.

Bảng 5.4: Ranh giới trách nhiệm khi tích hợp AI

5.2.1 Cơ sở thiết kế dữ liệu AI

Trước khi hiện thực, thiết kế xác định rõ AI cần ba nhóm dữ liệu đầu vào. Cách thiết kế này giúp tránh tình trạng “có model trước rồi mới tìm dữ liệu”, đồng thời bảo đảm mỗi thuật toán đều có nguồn tín hiệu khả thi trong hệ thống thư viện.

Nhóm dữ liệu	Nguồn trong hệ thống	Dùng cho bài toán AI
Nội dung tài liệu	PDF, tiêu đề, phụ đề, mô tả, mục lục, tác giả, danh mục, tag.	Embedding, semantic search, sách tương tự, content-based recommendation.
Hành vi người dùng	Lướt xem, wishlist, mượn sách, rating, lịch sử tìm kiếm.	Collaborative filtering, ALS implicit feedback, trending fallback.
Trạng thái nghiệp vụ	Sách còn bản sao hay không, chi nhánh, trạng thái bản sao, quyền truy cập.	Lọc kết quả cuối cùng để gợi ý/tìm kiếm có giá trị sử dụng thật.

Bảng 5.5: Nguồn dữ liệu dùng cho thiết kế AI

Về mặt lưu trữ, thiết kế chọn hướng **vector theo chunk** thay vì một vector duy nhất cho cả sách. Một sách có thể bao gồm nhiều chủ đề; nếu chỉ dùng một vector trung bình cho toàn bộ tài liệu, các chương nhỏ nhưng quan trọng dễ bị loãng. Vì vậy, mỗi chunk có một embedding riêng và liên kết về `publication_id`.

Listing 5.1: Thiết kế bảng vector phục vụ semantic search

```
CREATE TABLE ai_engine.publication_vectors (
  id BIGSERIAL PRIMARY KEY,
  publication_id BIGINT NOT NULL,
  chunk_index INT NOT NULL,
  chunk_text TEXT NOT NULL,
  embedding VECTOR(768) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
```

```
);
```

```
CREATE INDEX idx_publication_vectors_embedding
ON ai_engine.publication_vectors
USING hns w (embedding vector_cosine_ops);
```

5.2.2 Module 1: Xử lý PDF file

Mục tiêu của module này là biến một file PDF thô thành dữ liệu có giá trị cho hệ thống: **summary**, **audience**, **tag**, **vector embedding** và **trạng thái xử lý AI**. Đây là module nền tảng: nếu PDF không được xử lý tốt, semantic search và recommendation phía sau sẽ không có đủ dữ liệu để hoạt động.

Luồng xử lý được thiết kế theo hướng **không chặn thao tác của thủ thư**. Thủ thư chỉ cần upload tài liệu; backend lưu URL tài liệu và kích hoạt AI xử lý ở phía sau.

Phần xử lý PDF được chia thành hai nhánh rõ ràng: **ETL cục bộ** và **LLM reduce**. ETL cục bộ chịu trách nhiệm đọc file, làm sạch văn bản, chia chunk và sinh embedding. LLM chỉ được dùng ở bước tổng hợp metadata cuối cùng, không tham gia vào mọi truy vấn tìm kiếm. Cách thiết kế này giúp hệ thống giảm chi phí gọi LLM, tránh phụ thuộc quá mạnh vào quota Gemini và vẫn giữ được dữ liệu vector ổn định để phục vụ semantic search.

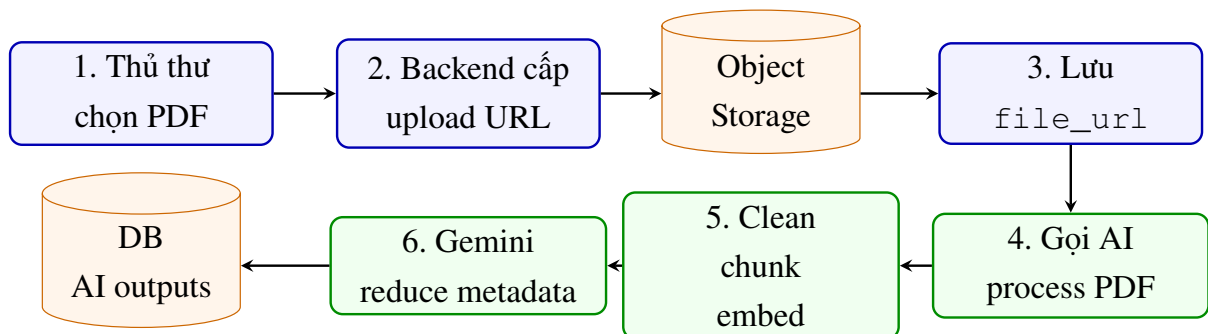
Với một tài liệu sau khi trích xuất được chuỗi văn bản D , hệ thống chia tài liệu thành các chunk c_i theo cửa sổ trượt:

$$C = \{c_1, c_2, \dots, c_n\}, \quad |c_i| \leq L, \quad \text{overlap}(c_i, c_{i+1}) = O$$

Trong đó L là kích thước chunk tối đa và O là phần chồng lấp giữa hai chunk liên kế. Overlap giúp giảm rủi ro một ý quan trọng bị cắt giữa hai đoạn. Mỗi chunk được mã hóa thành vector:

$$v_i = \text{Encoder}(c_i), \quad v_i \in \mathbb{R}^{768}$$

Các vector này được lưu trong PostgreSQL/pgvector cùng với `publication_id` và `chunk_index`. Nhờ vậy, hệ thống có thể truy vấn theo nội dung thật của sách thay vì chỉ tìm trên tiêu đề, tác giả hoặc mô tả ngắn.



Hình 5.8: Module 1 – Luồng xử lý PDF file sau khi thủ thư upload

Thành phần	Cách sử dụng trong hệ thống	Lý do thiết kế
PyMuPDF	Đọc PDF, trích xuất text theo trang, bỏ qua ảnh bìa hoặc phần không có text.	Nhẹ, chạy được trong AI container và phù hợp với tài liệu thư viện dạng PDF.
Text cleaning	Chuẩn hóa khoảng trắng, bỏ số trang/header/footer lặp, giới hạn đoạn quá ngắn.	Giảm nhiễu trước khi sinh embedding và trước khi gửi dữ liệu cho LLM.
Sliding window chunking	Chia văn bản thành các chunk có overlap để giữ ngữ cảnh liên tục.	Tăng khả năng tìm trùng đoạn liên quan khi truy vấn theo ý nghĩa.
SentenceTransformer /SBERT	Sinh embedding 768 chiều cho từng chunk và query.	Phù hợp mô hình bi-encoder, vector được tính trước nên tìm kiếm nhanh.
Gemini reduce	Nhận một phần nội dung đại diện, sinh summary, audience và tag theo JSON.	LLM chỉ dùng để làm giàu metadata, không nằm trên đường tìm kiếm chính.
Fallback metadata	Nếu Gemini lỗi/quota hết, dùng title, author, category và tag sẵn có để tạo metadata tối thiểu.	AI lỗi không làm hỏng catalog và không chặn thủ thư tiếp tục vận hành.

Bảng 5.6: Các thành phần kỹ thuật trong Module 1

Listing 5.2: Prompt rút gọn metadata bằng LLM

```
You are an assistant for a Vietnamese library management system.
Read the book content excerpts and return ONLY valid JSON.

Required JSON schema:
{
  "summary": "3-5 concise Vietnamese sentences",
  "audience": "target readers, e.g. beginner/intermediate/advanced",
  "tags_vi": ["3-8 Vietnamese subject tags"],
  "tags_en": ["English aliases for search expansion"]
}

Rules:
- Do not invent authors, publisher, ISBN or facts not present in the input.
- Prefer academic/library subject terms.
- Keep tags short, reusable and searchable.
- If the content is insufficient, infer only from title/category.
```

Prompt trên được thiết kế có ràng buộc đầu ra JSON để backend và AI Service dễ parse kết quả. Các trường tags_vi và tags_en được giữ song song nhằm hỗ trợ tìm kiếm song ngữ: người dùng có thể nhập từ khóa tiếng Việt hoặc tiếng Anh nhưng vẫn tìm được cùng một nhóm tài liệu.

Giai đoạn	Thiết kế	Điểm nhấn báo cáo
Upload và lưu file	Frontend xin presigned upload URL ; file PDF được đẩy lên object storage; backend lưu lại <code>file_url/s3Key</code> cho publication.	File được lưu trước, AI xử lý sau ; thao tác tạo/cập nhật sách không bị khóa bởi pipeline AI.
ETL nội dung	AI Service dùng PyMuPDF để trích xuất text, làm sạch header/footer/số trang, sau đó chia chunk bằng sliding window.	Index nội dung thật của PDF , không chỉ index tiêu đề hay mô tả thủ công.
Embedding và meta-data	Chunk được vector hóa và lưu vào <code>ai_engine.publication_vectors</code> ; Gemini chỉ được gọi ở bước reduce để tạo summary, audience và tag.	Không gọi LLM trong mỗi lần tìm kiếm ; giảm độ trễ, giảm quota và tăng độ ổn định.
Theo dõi trạng thái	Kết quả xử lý được lưu với trạng thái <code>RUNNING, SUCCESS, FAILED</code> ; nếu lỗi, catalog vẫn giữ ấn phẩm.	AI lỗi không làm hỏng nghiệp vụ thư viện .

Bảng 5.7: Thiết kế Module 1 – Xử lý PDF file

5.2.3 Module 2: Hỗ trợ tìm kiếm ngữ nghĩa (Semantic Search)

Mục tiêu của module này là giúp bạn đọc tìm được tài liệu theo **ý định học tập**, không bị phụ thuộc hoàn toàn vào việc nhớ đúng tên sách, tác giả hoặc tag. Ví dụ, truy vấn “sách nhập môn cơ sở dữ liệu dễ hiểu” vẫn có thể tìm đến các tài liệu liên quan, dù cụm từ đó không xuất hiện nguyên văn trong tiêu đề.

Về mặt thiết kế, hệ thống dùng hướng **Bi-Encoder** của Sentence-BERT [1]: vector của chunk PDF được tính trước trong Module 1, còn vector của query được tính tại thời điểm tìm kiếm. Việc so khớp vector được thực hiện bằng **PostgreSQL + pgvector** [2].

Khi người dùng nhập truy vấn q , AI Service sinh vector truy vấn:

$$v_q = \text{Encoder}(q)$$

Độ phù hợp giữa truy vấn và một chunk được tính bằng cosine similarity:

$$\text{sim}(q, c_i) = \frac{v_q \cdot v_i}{\|v_q\| \|v_i\|}$$

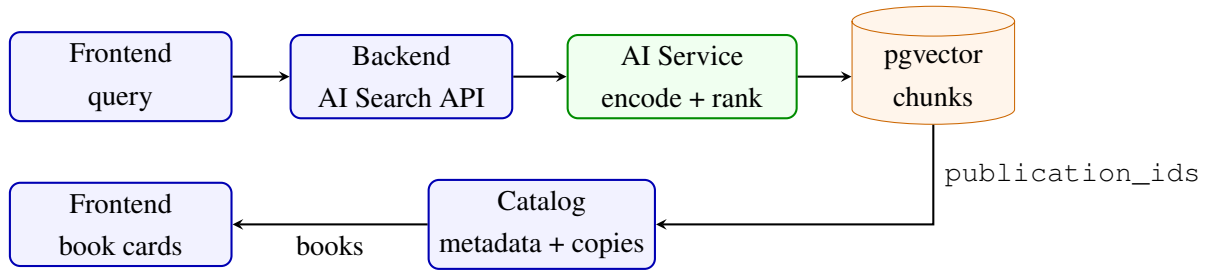
Vì mỗi sách có nhiều chunk, điểm semantic của một ấn phẩm không lấy trung bình toàn bộ chunk mà lấy điểm tốt nhất trong các chunk liên quan:

$$S_{\text{semantic}}(q, p) = \max_{c_i \in p} \text{sim}(q, c_i)$$

Điểm cuối cùng được chuẩn hóa và kết hợp thêm tín hiệu keyword để tránh trường hợp truy vấn rất rõ ràng như tên sách hoặc tên tác giả bị semantic search làm lệch kết quả:

$$S_{search}(q, p) = 0.70 \times S_{semantic}(q, p) + 0.20 \times S_{keyword}(q, p) + 0.10 \times S_{freshness}(p)$$

Trong đó $S_{keyword}$ đến từ tiêu đề, tác giả, category, tag; còn $S_{freshness}$ là điểm ưu tiên nhẹ cho tài liệu mới hoặc vừa được cập nhật. Trọng số này được chọn theo hướng ưu tiên ý nghĩa nội dung nhưng vẫn giữ kết quả ổn định với các truy vấn định danh.



Hình 5.9: Module 2 – Luồng thực thi semantic search

Tín hiệu	Trọng số	Ý nghĩa
Semantic score	0.70	Tín hiệu chính, đo mức gần nghĩa giữa câu hỏi của người dùng và nội dung PDF.
Keyword score	0.20	Giữ độ chính xác với truy vấn có tên sách, tác giả, ISBN, category hoặc tag rõ ràng.
Freshness/availability	0.10	Ưu tiên nhẹ cho tài liệu mới/còn khả dụng, tránh đề xuất kết quả đúng nghĩa nhưng kém hữu ích.

Bảng 5.8: Trọng số xếp hạng trong Module 2

Điểm thiết kế	Ý nghĩa
Query đi qua backend	Backend giữ format response, phân quyền và thống nhất cách trả dữ liệu cho frontend.
AI chỉ trả ID	AI Service không tự trả toàn bộ thông tin sách; backend lấy lại metadata, rating và tình trạng bản sao từ catalog.
Kết hợp keyword fallback	Với truy vấn ngắn như tên sách/tác giả, hệ thống vẫn ưu tiên keyword để tránh semantic search làm lệch kết quả rõ ràng.
Tìm trên nội dung thật	Vector được sinh từ chunk PDF, nên hệ thống có thể tìm theo nội dung bên trong sách, không chỉ theo metadata nhập tay.

Bảng 5.9: Thiết kế Module 2 – Hỗ trợ tìm kiếm ngữ nghĩa

5.2.4 Module 3: Gợi ý cá nhân hoá (Recommendation)

Mục tiêu của module này là biến dữ liệu tương tác rời rạc thành danh sách sách gợi ý có ý nghĩa. Trong bối cảnh thư viện, dữ liệu thường không dày như các nền tảng thương mại: có người dùng mới, sách mới, ít rating và hành vi không đều. Vì vậy, hệ thống chọn hướng **hybrid**

recommendation [3], kết hợp nhiều nguồn tín hiệu thay vì phụ thuộc vào một thuật toán duy nhất.

Hệ thống xem mỗi hành vi của bạn đọc là một tín hiệu quan tâm ngầm (*implicit feedback*). Các hành vi không có cùng mức độ quan trọng nên được quy đổi thành trọng số:

$$r_{u,p} = 1.0 \times \text{view} + 2.0 \times \text{wishlist} + 3.0 \times \text{borrow} + 2.5 \times \text{rating} + 1.5 \times \text{searchClick}$$

Trong đó u là người dùng và p là ấn phẩm. Trọng số mượn sách cao nhất vì đây là tín hiệu mạnh nhất cho thấy người dùng thật sự quan tâm. Wishlist và rating được xem là tín hiệu trung bình-cao; lượt xem chỉ là tín hiệu yếu vì người dùng có thể mở trang chi tiết nhưng không có nhu cầu mượn.

Điểm gợi ý cuối cùng được kết hợp từ ba nhóm tín hiệu:

$$S_{rec}(u, p) = 0.45 \times S_{content}(u, p) + 0.35 \times S_{als}(u, p) + 0.20 \times S_{trending}(p)$$

Content-based dùng vector của các sách người dùng từng tương tác để tạo hồ sơ sở thích. **ALS implicit** học mẫu tương tác giữa nhiều người dùng. **Trending fallback** bảo đảm vẫn có gợi ý khi tài khoản mới, sách mới hoặc dữ liệu hành vi còn ít.

Hồ sơ nội dung của người dùng được tính bằng trung bình có trọng số các vector sách đã tương tác:

$$profile(u) = \frac{\sum_{p \in I_u} r_{u,p} \times v_p}{\sum_{p \in I_u} r_{u,p}}$$

Sau đó điểm content-based giữa người dùng và sách ứng viên được tính bằng:

$$S_{content}(u, p) = \cos(profile(u), v_p)$$

Nguồn gợi ý	Cách hoạt động	Vai trò
Content-based	Tạo hồ sơ nội dung từ vector của các sách người dùng đã xem, wishlist hoặc mượn.	Phản ứng nhanh với hành vi mới.
ALS implicit feedback	Học quan hệ người dùng–sách từ WATCH, WISHLIST, BORROWED [4].	Khai thác mẫu hành vi tập thể.
Trending fallback	Xếp hạng theo tương tác, lượt mượn, rating và thời gian tạo sách.	Chống cold-start và lỗi AI.

Bảng 5.10: Ba nguồn tín hiệu trong Module 3 – Recommendation

Nguồn điểm	Trọng số	Vai trò trong hệ thống
Content-based	0.45	Bám sát nội dung sách người dùng từng quan tâm, hoạt động tốt ngay cả khi dữ liệu cộng đồng còn ít.
ALS implicit feedback	0.35	Khai thác mẫu hành vi giữa nhiều độc giả, giúp phát hiện sách liên quan nhưng không quá giống về metadata.
Trending fallback	0.20	Bảo đảm luôn có gợi ý hợp lý cho người dùng mới, sách mới hoặc khi pipeline AI chưa đủ dữ liệu.

Bảng 5.11: Trọng số hybrid recommendation trong Module 3

Để tránh gợi ý không phù hợp trong bối cảnh thư viện, danh sách ứng viên còn được lọc qua các ràng buộc nghiệp vụ: sách đang bị ẩn không được đề xuất, bản sao mất hoặc hư không được ưu tiên, sách người dùng đang mượn không lặp lại trong danh sách gợi ý, và các ấn phẩm không còn khả dụng sẽ bị giảm điểm. Nhờ vậy kết quả recommendation không chỉ “đúng về mặt thuật toán” mà còn dùng được trong vận hành thực tế.

5.3 Thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu là quá trình chuyển đổi từ mô hình phân tích yêu cầu và các thực thể đã được đặc tả ở các phần trước thành một cấu trúc lưu trữ logic và vật lý. Mục tiêu là xây dựng một nền tảng dữ liệu nhất quán, toàn vẹn, hiệu quả và có khả năng mở rộng để hỗ trợ toàn bộ nghiệp vụ của hệ thống, từ các giao dịch mượn trả cơ bản đến các tính năng AI phức tạp.

5.3.1 Lựa chọn Hệ Quản trị Cơ sở dữ liệu (HQTCSDL)

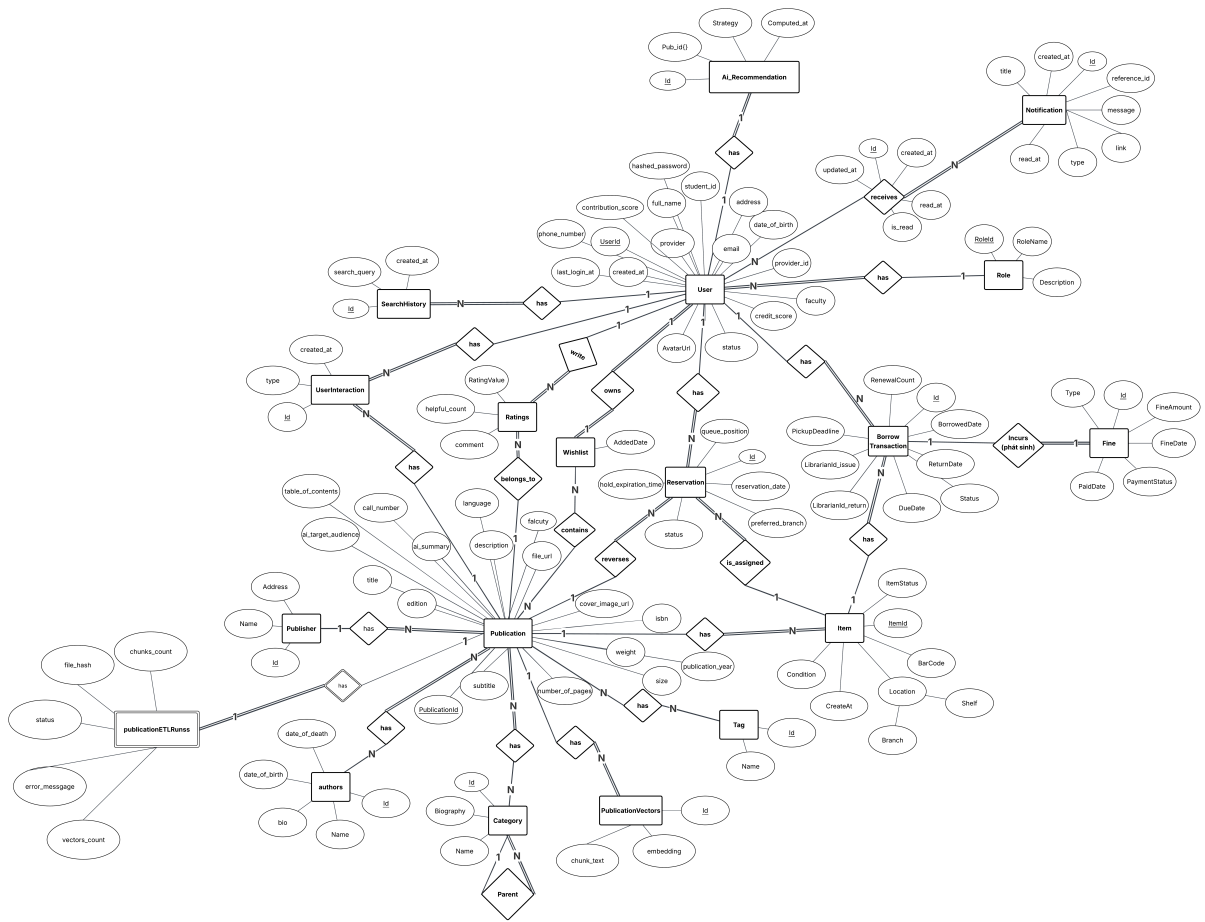
Dựa trên bản chất quan hệ chặt chẽ của dữ liệu (người dùng, sách, lượt mượn), hệ thống lựa chọn **PostgreSQL** làm HQTCSDL chính.

- **Tính tin cậy và toàn vẹn dữ liệu:** PostgreSQL nổi tiếng với sự tuân thủ nghiêm ngặt các tiêu chuẩn ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo các giao dịch nghiệp vụ quan trọng như mượn, trả, và tính phí phạt luôn được thực hiện một cách an toàn.
- **Hỗ trợ các kiểu dữ liệu nâng cao:** PostgreSQL hỗ trợ các kiểu dữ liệu phức tạp như JSONB, rất hữu ích để lưu trữ các metadata linh hoạt. Đặc biệt, thông qua các extension như `pgvector`, nó có khả năng lưu trữ và truy vấn các vector embedding hiệu quả, phục vụ trực tiếp cho tính năng tìm kiếm ngữ nghĩa.
- **Khả năng mở rộng:** Đây là một hệ quản trị CSDL mã nguồn mở mạnh mẽ, đã được chứng minh về hiệu năng trong các hệ thống quy mô lớn.

5.3.2 Mô hình Quan hệ Thực thể (ERD)

Bên dưới là sơ đồ ERD tổng quan của toàn bộ hệ thống. Mô hình dữ liệu được thiết kế xoay quanh thực thể trung tâm là **Publication** và **User**. **Publication** biểu diễn đầu sách hoặc tài liệu thư viện, liên kết với tác giả, nhà xuất bản, danh mục, thẻ chủ đề, bản sao vật lý (**Item**) và dữ liệu vector phục vụ tìm kiếm ngữ nghĩa. **User** liên kết với vai trò, lịch sử tìm kiếm, wishlist, đánh giá, tương tác, thông báo và các nghiệp vụ mượn/đặt trước.

Các quan hệ nghiệp vụ chính được thể hiện qua **borrowing_transactions**, **reservations** và **finest**. Một người dùng có thể tạo nhiều phiếu mượn hoặc đặt trước; mỗi phiếu mượn gắn với một bản sao sách cụ thể và có thể phát sinh phí phạt nếu quá hạn, mất sách hoặc hư hỏng. Các bảng AI như **PublicationVector**, **Ai_Recommendation** và **user_interactions** được tách ra nhưng vẫn liên kết với dữ liệu lõi để hỗ trợ gợi ý cá nhân hóa, tìm kiếm ngữ nghĩa và phân tích hành vi đọc.



Hình 5.10: Sơ đồ Quan hệ Thực thể (ERD) tổng thể của hệ thống

Chương 6: Hiện thực hệ thống

6.1 Hiện thực ứng dụng

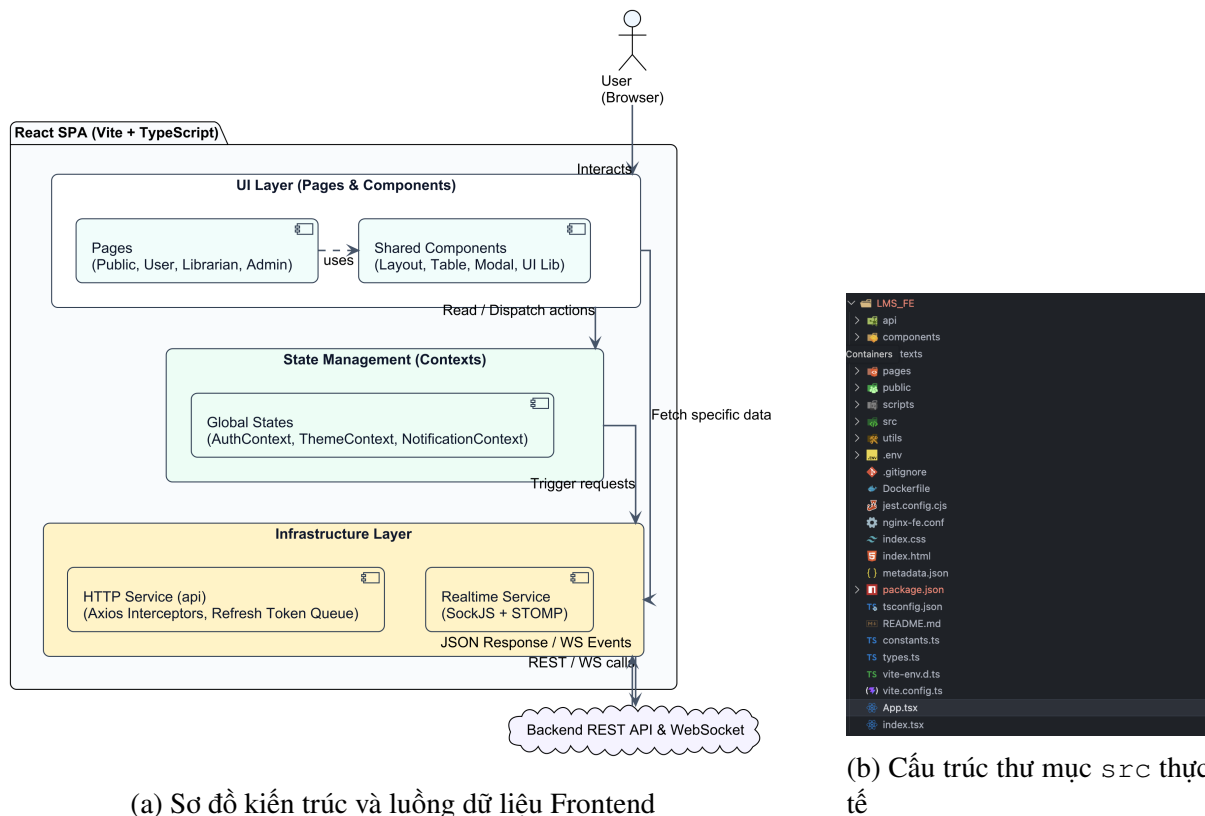
Phần hiện thực được tổ chức theo ba lớp triển khai chính của hệ thống: frontend, backend và AI service. Để tránh báo cáo bị dàn trải, chương này trình bày theo dạng tổng hợp những phần đã hiện thực, thay vì mô tả tuần tự từng màn hình hoặc từng hàm. Cách trình bày này giúp thể hiện đầy đủ phạm vi công việc đã làm nhưng vẫn giữ báo cáo ngắn gọn.

6.1.1 Hiện thực Frontend

Frontend được hiện thực bằng React, TypeScript và Vite theo mô hình Single Page Application. Frontend đảm nhiệm hiển thị dữ liệu, điều phối trạng thái đăng nhập, phân vùng giao diện theo vai trò, gọi API nghiệp vụ, nhận thông báo thời gian thực, upload tài liệu, hiển thị QR nhận sách và hỗ trợ thanh toán phí phạt qua PayOS. Mã nguồn được chia theo trách nhiệm: `pages` chứa màn hình nghiệp vụ, `components` chứa thành phần tái sử dụng, `contexts` chứa state dùng chung, `api` đóng gói HTTP service và `utils` chứa các hàm xử lý thuần.

Để đáp ứng độ phức tạp của một hệ thống nghiệp vụ tích hợp AI và thanh toán, phân hệ Frontend được thiết kế tuân thủ nghiêm ngặt nguyên lý phân tách trách nhiệm (Separation of Concerns). Hình 6.1 minh họa chi tiết sơ đồ kiến trúc luồng dữ liệu (Data Flow) cùng với cấu trúc thư mục thực tế của dự án.

Cụ thể, lớp `api` được đóng gói độc lập để xử lý giao tiếp HTTP và refresh token tự động. Dữ liệu sau đó được đẩy lên lớp `contexts` để quản lý trạng thái tập trung (Global State), và cuối cùng mới được phân phối xuống các trang (`pages`) và thành phần giao diện (`components`) để hiển thị cho người dùng.



Hình 6.1: Kiến trúc tổng thể và cách thức tổ chức mã nguồn Frontend

Dựa trên nền tảng kiến trúc vững chắc này, hệ thống đã triển khai thành công các luồng nghiệp vụ phức tạp và phân định rạch ròi quyền truy cập thông qua cơ chế Route Protection. Bảng 6.1 liệt kê chi tiết danh sách các màn hình chức năng đã được hiện thực và phân quyền cụ thể cho từng nhóm người dùng (Bạn đọc, Thủ thư, Quản trị viên).

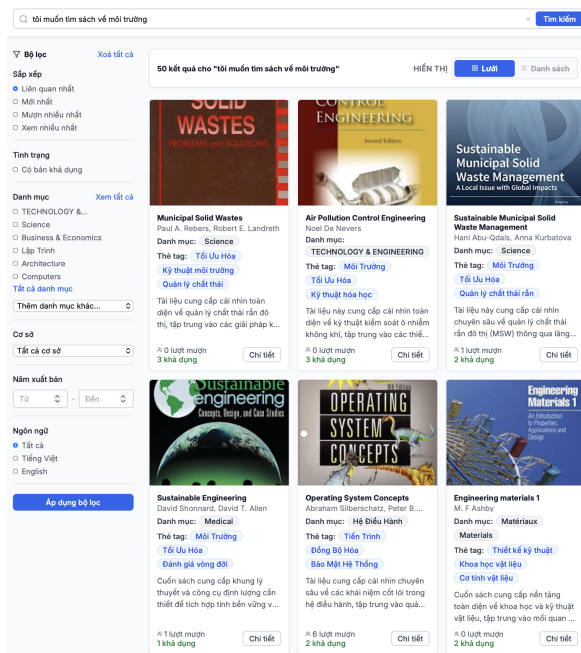
Bảng 6.1: Các màn hình Frontend đã hiện thực theo nhóm người dùng

Nhóm trang	Màn hình	Chức năng đã hiện thực
/publicpage	Trang chủ	Hiển thị sách mới, sách được mượn nhiều, sách được đánh giá cao, thống kê công khai và ô tìm kiếm nhanh.
/publicpage	Tìm kiếm sách	Tìm kiếm theo từ khóa, lọc danh mục/tag/ngôn ngữ/năm xuất bản/trạng thái bản sao, sắp xếp, phân trang và chuyển sang semantic search khi cần.
/publicpage	Chi tiết sách	Hiển thị metadata, bản sao, trạng thái khả dụng, summary/tag AI, rating, review, sách tương tự, nút mượn hoặc đặt trước.
/publicpage	Auth pages	Đăng ký, đăng nhập, Google OAuth callback, xác thực email, quên mật khẩu và đặt lại mật khẩu.

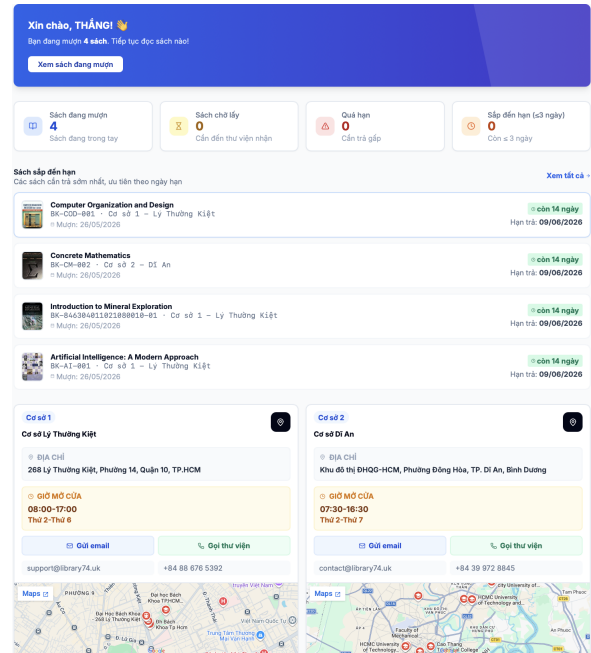
Nhóm trang	Màn hình	Chức năng đã hiện thực
/publicpage	Liên hệ, hướng dẫn, FAQ	Hiển thị thông tin thư viện, form liên hệ, iframe Google Maps lazy loading và nội dung hướng dẫn sử dụng.
/userpage	Dashboard bạn đọc	Tổng hợp sách đang mượn, đặt trước, phí phạt, thông báo, gợi ý sách và thao tác nhanh.
/userpage	Sách đang mượn/ lịch sử	Theo dõi giao dịch mượn, hạn trả, trạng thái quá hạn, lịch sử mượn trả và chi tiết giao dịch.
/userpage	Đặt trước	Xem danh sách phiếu đặt trước, vị trí hàng chờ, trạng thái sẵn sàng nhận sách, deadline và hủy đặt trước.
/userpage	Phí phạt	Xem phí quá hạn/hư/mất, trạng thái thanh toán, tạo QR PayOS và cập nhật kết quả thanh toán.
/userpage	Wishlist, rating, hồ sơ	Lưu sách yêu thích, đánh giá sách sau khi mượn, phản hồi review, cập nhật hồ sơ, avatar, mật khẩu, ngôn ngữ và theme.
/userpage	Thông báo	Nhận thông báo realtime, đánh dấu đã đọc, xem thông báo về mượn-trả, đặt trước, phí phạt và phản hồi review.
/librarianpage	Dashboard thủ thư	Thống kê vận hành, sách mượn/trả, yêu cầu chờ xử lý, phí phạt, cảnh báo quá hạn và các thao tác nhanh.
/librarianpage/ reports	Báo cáo điều hành	Tổng hợp báo cáo vận hành cho thủ thư, gồm các chỉ số mượn-trả, đặt trước, phí phạt, xu hướng sử dụng tài liệu và dữ liệu hỗ trợ ra quyết định.
/librarianpage	Quản lý ấn phẩm	Tạo/sửa/xóa ấn phẩm, tác giả, nhà xuất bản, danh mục, tag, ảnh bìa, ISBN auto fill, upload PDF và theo dõi trạng thái xử lý AI.
/librarianpage	Quản lý bản sao	Tạo bản sao vật lý, barcode, vị trí kệ, chi nhánh, trạng thái khả dụng, hư/mất và lịch sử lưu thông.
/librarianpage	Mượn-trả tại quầy	Tra cứu bạn đọc, quét/nhập barcode, cho mượn trực tiếp, xác nhận nhận sách, trả sách, ghi chú giao dịch và xử lý sách hư/mất.

Nhóm trang	Màn hình	Chức năng đã hiện thực
/librarianpage	Yêu cầu và đặt trước	Xem yêu cầu mượn, duyệt/từ chối, quản lý hàng chờ đặt trước, xác nhận nhận sách và gửi thông báo cho bạn đọc.
/librarianpage	Phí phạt và thanh toán	Tính phí, thu tiền mặt, tạo QR PayOS, polling trạng thái chuyển khoản và cập nhật trạng thái fine.
/adminpage	Quản lý tài khoản	Quản lý tài khoản độc giả/thủ thư, tạo thủ thư, khóa/mở khóa, xác minh tài khoản và cập nhật trạng thái người dùng.
/adminpage	Chính sách và audit	Cấu hình quy định mượn trả, xem nhật ký thao tác, lọc audit log và theo dõi thay đổi vận hành hệ thống.

Để mang lại trải nghiệm liền mạch cho bạn đọc, phân hệ Frontend được thiết kế tối ưu từ khâu khám phá tài liệu đến các thao tác tương tác chi tiết. Quá trình này bắt đầu tại màn hình Dashboard và công cụ tìm kiếm ngữ nghĩa, nơi người dùng có thể tra cứu ấn phẩm bằng ngôn ngữ tự nhiên thay vì chỉ dựa vào từ khóa rập khuôn.



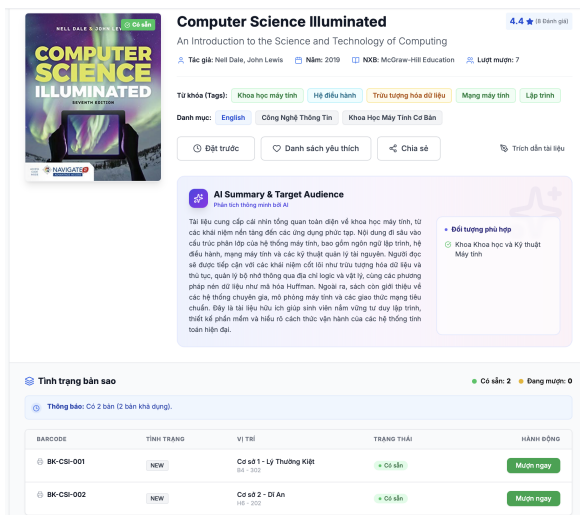
(a) Tìm kiếm ngữ nghĩa và lọc sách



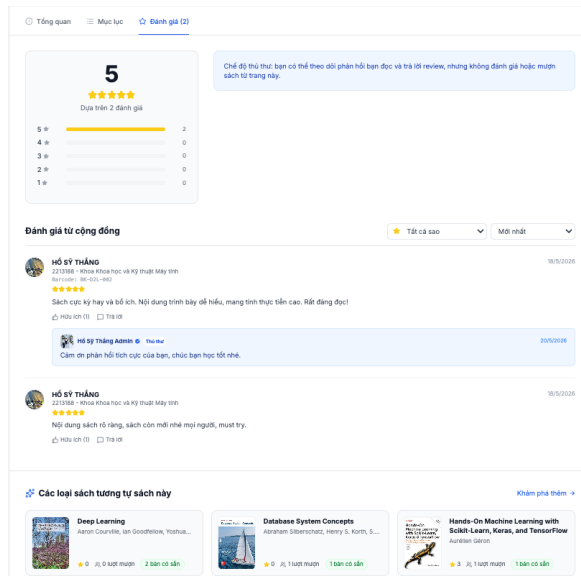
(b) Dashboard bạn đọc

Hình 6.2: Giao diện tổng quan và chức năng tìm kiếm thông minh dành cho bạn đọc

Sau khi tìm được tài liệu mong muốn, bạn đọc có thể xem thông tin chi tiết với các siêu dữ liệu (metadata) được AI tự động trích xuất. Đồng thời, hệ thống cung cấp đầy đủ các tính năng tương tác như đánh giá, phản hồi và thanh toán trực tuyến tự động thông qua mã QR PayOS.

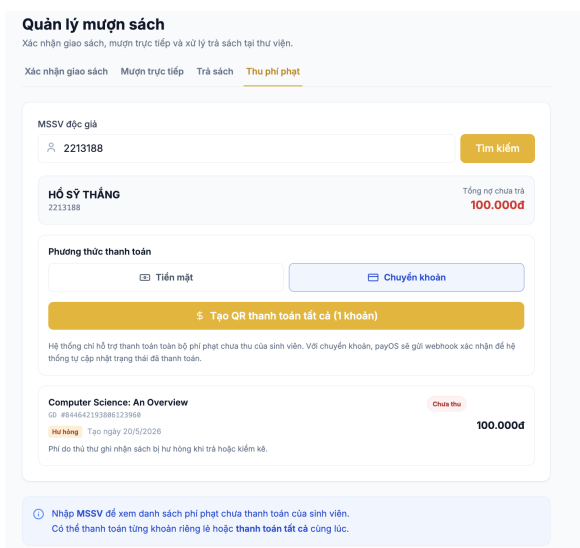


(a) Chi tiết sách và siêu dữ liệu (metadata) AI

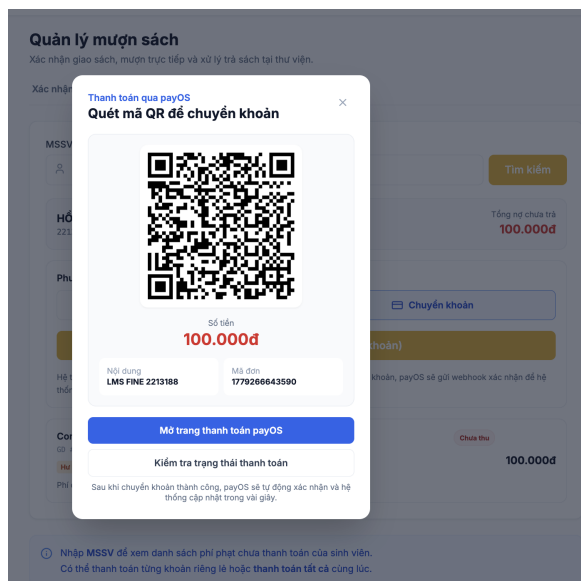


(b) Đánh giá và phản hồi

Hình 6.3: Trải nghiệm xem chi tiết và tương tác với ấn phẩm



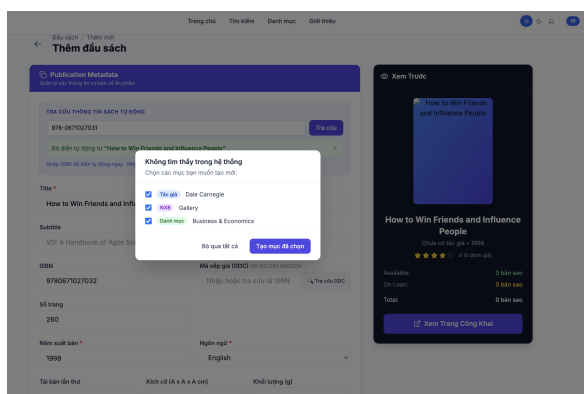
(a) Khởi tạo thanh toán



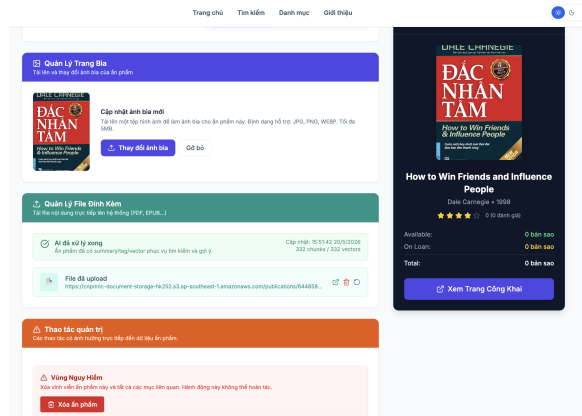
(b) Hiển thị mã QR PayOS

Hình 6.4: Quy trình tích hợp thanh toán tự động

Đối với nghiệp vụ quản trị, hệ thống tập trung giải quyết bài toán giảm tải thao tác thủ công cho thủ thư bằng cách tự động hóa luồng số hóa tài liệu. Các tính năng nổi bật bao gồm tự động điền thông tin từ mã ISBN và giám sát tiến trình AI xử lý tài liệu vừa tải lên.



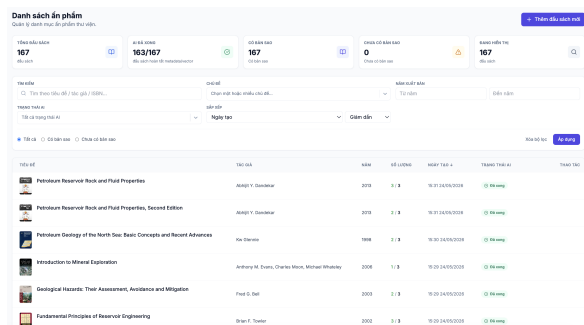
(a) Tự động trích xuất metadata từ ISBN



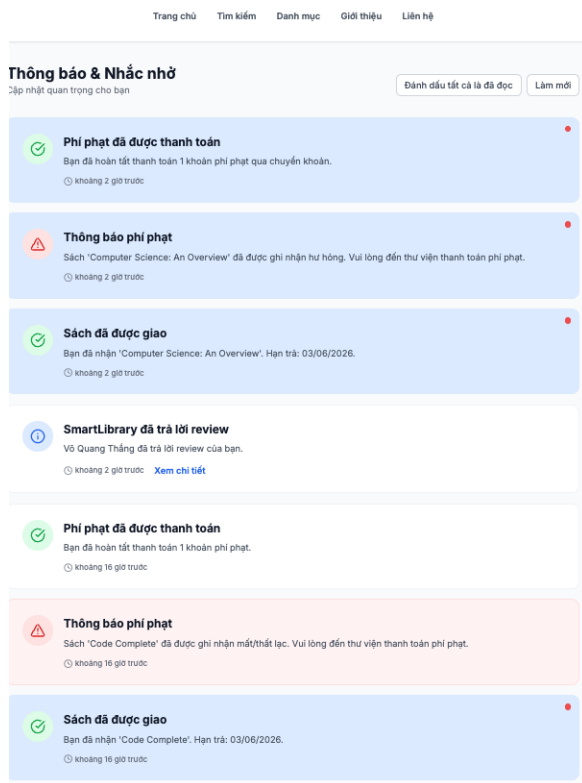
(b) Tải lên tài liệu và giám sát tiến trình AI

Hình 6.5: Nghiệp vụ nhập liệu và số hóa tài liệu thông minh

Bên cạnh khâu nhập liệu, công tác giám sát và quản lý danh mục ấn phẩm cũng được trực quan hóa, kết hợp cùng hệ thống thông báo thời gian thực (real-time notification) để thủ thư nắm bắt và xử lý kịp thời các sự kiện phát sinh.



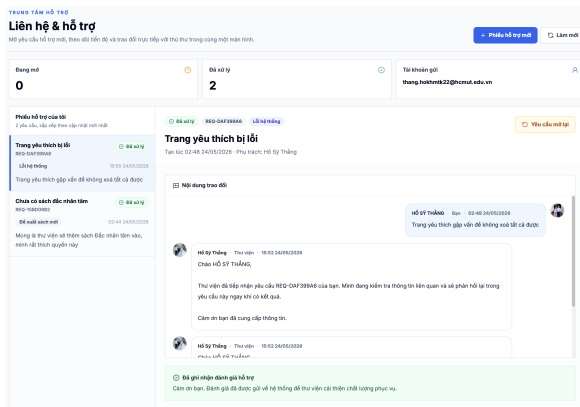
(a) Quản lý danh mục ấn phẩm



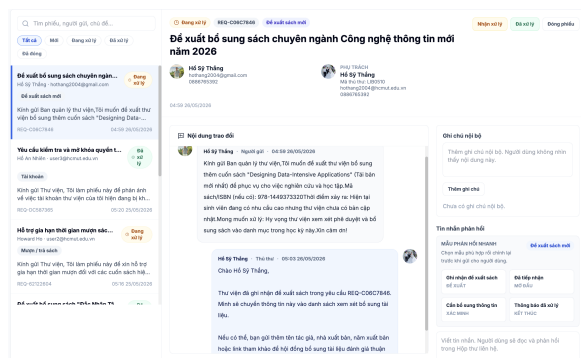
(b) Hệ thống thông báo thời gian thực

Hình 6.6: Nghiệp vụ quản lý vận hành và giám sát hệ thống của thủ thư

Cuối cùng, để đảm bảo tính kết nối và đáp ứng đa dạng nhu cầu sử dụng, hệ thống cung cấp luồng giao tiếp hỗ trợ trực tuyến giữa các nhóm người dùng, đi kèm các tùy chọn cá nhân hóa sâu về giao diện (Light/Dark mode) và ngôn ngữ (Việt - Anh).

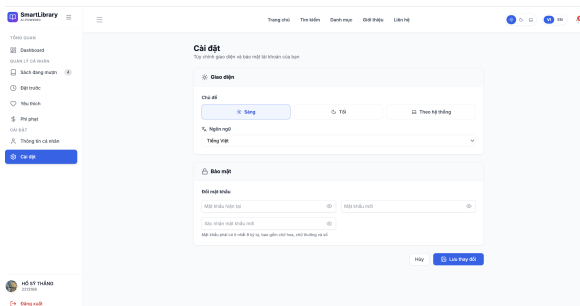


(a) Gửi yêu cầu hỗ trợ (Bạn đọc)

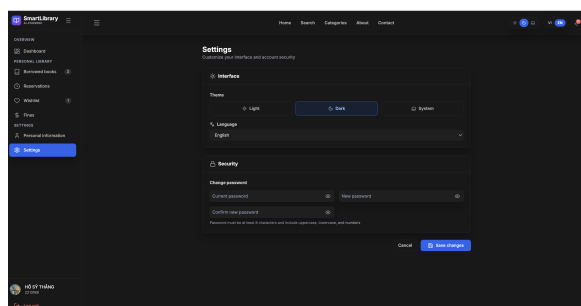


(b) Tiếp nhận và phản hồi (Thủ thư)

Hình 6.7: Luồng giao tiếp và hỗ trợ trực tuyến giữa các nhóm người dùng



(a) Giao diện sáng (Light) - Tiếng Việt



(b) Giao diện tối (Dark) - Tiếng Anh

Hình 6.8: Tính năng cá nhân hóa giao diện và đa ngôn ngữ (i18n)

Đằng sau giao diện trực quan và các trải nghiệm cá nhân hóa linh hoạt, phân hệ Frontend được xây dựng dựa trên một kiến trúc kỹ thuật vững chắc nhằm đảm bảo tính mở rộng và hiệu năng cao. Bảng 6.2 tổng hợp các giải pháp kỹ thuật trọng tâm đã được áp dụng để giải quyết các bài toán cốt lõi: từ quản lý trạng thái tập trung, tối ưu giao tiếp API, đến xử lý luồng sự kiện thời gian thực (real-time) và tích hợp an toàn các dịch vụ bên thứ ba.

Bảng 6.2: Các hạng mục kỹ thuật Frontend

Hạng mục Frontend	Nội dung đã hiện thực
Routing và phân quyền	HashRouter, lazy loading theo page, ProtectedRoute, redirect theo role student, librarian, admin.
State dùng chung	AuthContext, LanguageContext, ThemeContext, NotificationContext, UploadContext, AppDialogContext.
Giao tiếp API	Axios instance, gắn JWT và Accept-Language, unwrap response, xử lý lỗi nghiệp vụ, refresh token bằng queue để tránh gọi refresh nhiều lần.

Tiếp tục ở trang sau

Bảng 6.2 – tiếp tục từ trang trước

Hạng mục Frontend	Nội dung đã hiện thực
Realtime và notification	STOMP/SockJS, unread count, toast realtime, đánh dấu đã đọc và đồng bộ danh sách thông báo.
Upload và tài liệu	Upload bìa/PDF bằng presigned URL, hiển thị progress, cảnh báo khi đóng tab lúc upload đang chạy, gửi yêu cầu AI xử lý tài liệu.
Thanh toán	Khởi tạo thanh toán PayOS, hiển thị QR, polling trạng thái, cập nhật UI cho thủ thư và bạn đọc sau khi thanh toán.
Cá nhân hóa	Light/dark/system theme, song ngữ Việt-Anh, lưu cấu hình vào localStorage, cập nhật document.lang.

Tất cả các quyết định về mặt kiến trúc và kỹ thuật nêu trên đều hướng tới một mục tiêu duy nhất: chuyển hóa công nghệ thành giá trị sử dụng. Bảng 6.3 đúc kết lại những điểm sáng nổi bật của phân hệ, minh chứng cho việc ứng dụng công nghệ không chỉ dừng lại ở mặt hiển thị mà đã thực sự giải quyết triệt để các bài toán vận hành, chuyển đổi toàn diện trải nghiệm của bạn đọc và thủ thư.

Giá trị hiện thực	Cách hệ thống đã làm	Tác động với người dùng
Tìm kiếm đa hướng	Kết hợp keyword search, bộ lọc catalog, tag/category alias và semantic search.	Bạn đọc không cần nhớ chính xác tên sách vẫn có thể tìm tài liệu phù hợp.
Vận hành tại quầy	Màn hình thủ thư hỗ trợ mượn trực tiếp, xác nhận nhận sách, trả sách, ghi chú và xử lý phí.	Nghiệp vụ mượn-trả không còn chỉ nằm ở phần demo giao diện.
Báo cáo điều hành	Trang <code>/librarianpage/reports</code> gom dữ liệu vận hành thành các báo cáo theo dõi mượn-trả, phí phạt, đặt trước và xu hướng sử dụng tài liệu.	Thủ thư có thêm góc nhìn tổng hợp để theo dõi tình hình thư viện và ra quyết định vận hành dựa trên dữ liệu.
Tài liệu số	Upload bìa/PDF, progress upload, presigned URL và kích hoạt AI xử lý nền.	Thủ thư có thể đưa tài liệu mới vào hệ thống và theo dõi kết quả xử lý.
Phản hồi tức thời	Toast, notification realtime, unread count và đồng bộ trạng thái thanh toán/đặt trước.	Người dùng biết ngay khi có thay đổi quan trọng thay vì phải tải lại trang.

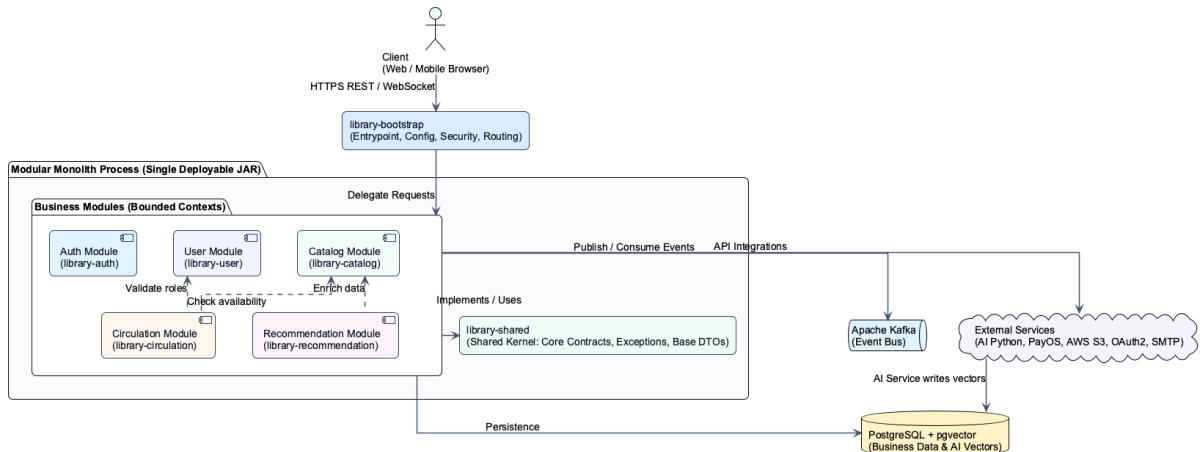
Bảng 6.3: Các điểm giá trị nổi bật ở Frontend

6.1.2 Hiện thực Backend

Backend được hiện thực bằng Spring Boot 3.3.5 và Java 21 theo mô hình modular monolith. Backend là lớp kiểm soát chính của hệ thống: xác thực, phân quyền, giao dịch mượn-trả, trạng thái bản sao, phí phạt, thanh toán, thông báo, audit và gateway sang AI service đều đi qua backend. Cấu trúc module được chia theo miền nghiệp vụ để dễ bảo trì nhưng vẫn triển khai

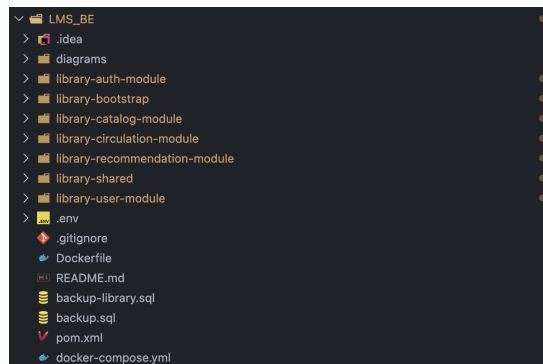
trong một ứng dụng thống nhất.

Để hiện thực hóa tư tưởng thiết kế này, hệ thống được phân rã thành các miền nghiệp vụ độc lập (Bounded Contexts) nhưng vẫn chia sẻ chung một lõi nền tảng (*library-shared*). Hình 6.9 minh họa chi tiết sơ đồ kiến trúc tổng thể của hệ thống. Trong đó, các module giao tiếp với nhau thông qua cơ chế hướng sự kiện (Event-driven) bằng Apache Kafka hoặc qua các giao diện hợp đồng, giúp giảm thiểu tối đa sự phụ thuộc chéo (tight-coupling).



Hình 6.9: Sơ đồ kiến trúc Modular Monolith tổng thể

Tương ứng với sơ đồ kiến trúc, mã nguồn vật lý cũng được tổ chức phân rã một cách chặt chẽ. Hình 6.10 thể hiện cấu trúc thư mục thực tế của dự án, minh chứng cho việc áp dụng triệt để mô hình Modular Monolith vào khâu triển khai mã nguồn.



Hình 6.10: Cấu trúc thư mục mã nguồn Backend thực tế

Nhờ cấu trúc phân mảnh rõ ràng, toàn bộ các điểm cuối giao tiếp (API endpoints) của hệ thống đều được quy chuẩn hóa và tự động sinh tài liệu thông qua OpenAPI/Swagger. Do quy mô hệ thống lớn với hàng trăm API, Hình 6.11 trích xuất giao diện API của module đại diện (Publication Controller) nhằm minh chứng cho mức độ chi tiết và tính chuẩn mực của các API Contract, đảm bảo sự minh bạch khi tích hợp với Frontend và AI Service.

publication-controller			^
GET	/api/v1/publications/{id}		🔒
PUT	/api/v1/publications/{id}		🔒
DELETE	/api/v1/publications/{id}		🔒
PUT	/api/v1/publications/{id}/document-url		🔒
POST	/api/v1/publications		🔒
POST	/api/v1/publications/{id}/view		🔒
POST	/api/v1/publications/{id}/cover		🔒
POST	/api/v1/publications/{id}/ai/vectorize		🔒
POST	/api/v1/publications/{id}/ai/reprocess		🔒
POST	/api/v1/publications/{id}/ai/metadata		🔒
GET	/api/v1/publications/{id}/items		🔒
GET	/api/v1/publications/{id}/document-upload-url		🔒
GET	/api/v1/publications/testimonials		🔒
GET	/api/v1/publications/search		🔒
GET	/api/v1/publications/public-stats		🔒
GET	/api/v1/publications/newest		🔒
GET	/api/v1/publications/most-borrowed		🔒
GET	/api/v1/publications/librarian		🔒
GET	/api/v1/publications/book-lookup		🔒

Hình 6.11: Tài liệu hóa API tự động bằng Swagger (Đại diện module Publication)

Chi tiết nhiệm vụ và nội dung hiện thực của toàn bộ các phân hệ con trong kiến trúc Backend được tổng hợp cụ thể tại Bảng 6.4.

Bảng 6.4: Các module Backend đã hiện thực

Module	Nhóm chức năng	Nội dung đã hiện thực
library-bootstrap	Khởi động hệ thống	Cấu hình Spring Boot, datasource, JPA, Flyway, Kafka, mail, OpenAPI, async, security, CORS và biến môi trường triển khai.
library-shared	Thành phần dùng chung	Response wrapper, exception, error code, base entity, DTO dùng chung, shared port, Kafka event, S3 storage port và audit logging aspect.
library-auth-module	Xác thực	Đăng ký, đăng nhập, refresh token, logout, xác thực email, quên mật khẩu, đặt lại mật khẩu, Google OAuth2, JWT decoder và WebSocket authentication.
library-user-module	Người dùng và thông báo	Hồ sơ người dùng, avatar, đổi mật khẩu, vai trò, trạng thái tài khoản, danh sách notification, Kafka consumer và WebSocket push.

Module	Nhóm chức năng	Nội dung đã hiện thực
library-admin-module	Quản trị	Quản lý tài khoản độc giả/thủ thư, tạo thủ thư, khóa/mở khóa, xác minh tài khoản, cấu hình chính sách mượn trả và audit log.
library-catalog-module	Catalog thư viện	Ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag, tìm kiếm công khai, chi tiết sách, trạng thái bản sao, upload bìa và presigned URL cho PDF.
library-circulation-module	Mượn-trả	Yêu cầu mượn, mượn trực tiếp, xác nhận nhận sách, trả sách, đặt trước, hàng chờ, gia hạn trạng thái, báo hư/mất và cập nhật trạng thái bản sao.
library-circulation-module	Phí phạt và scheduler	Tính phí quá hạn/hư/mất, thanh toán tiền mặt, PayOS, nhắc sắp đến hạn, xử lý quá hạn, hủy đặt trước hết hạn và dashboard vận hành.
library-recommendation-module	Tương tác bạn đọc	Wishlist, lịch sử tìm kiếm, lượt xem, rating, comment, phản hồi rating của thủ thư, đánh giá hệ thống và thống kê tương tác.
library-recommendation-module	AI Gateway	Gọi semantic search, sách tương tự, recommendation, gửi yêu cầu xử lý PDF, nhận AI callback có HMAC và cập nhật metadata AI về publication.

Để đảm bảo các miền nghiệp vụ nêu trên phối hợp nhịp nhàng và chịu tải tốt, toàn bộ hệ thống được xây dựng trên một bộ tiêu chuẩn kỹ thuật cốt lõi nghiêm ngặt. Bảng 6.5 trình bày chi tiết các giải pháp nền tảng đã được triển khai, bao gồm chuẩn hóa giao tiếp API, cơ chế bảo mật không trạng thái (stateless), quản trị cơ sở dữ liệu tập trung và các luồng xử lý tác vụ nền bất đồng bộ.

Hạng mục Backend	Nội dung đã hiện thực
Chuẩn API	ApiResponseApp, PageResponse, mã lỗi nghiệp vụ, message theo Accept-Language, GlobalExceptionHandler.
Bảo mật	Spring Security stateless, JWT, refresh token, Google OAuth2, kiểm tra role bằng annotation và AOP, khóa tài khoản có hiệu lực ngay khi đăng nhập.
Dữ liệu	PostgreSQL, Flyway migration, entity audit, phân biệt ẩn phẩm và bản sao, lưu trạng thái giao dịch, fine, reservation, notification và dữ liệu AI.
Tác vụ nền	Scheduler nhắc hạn, xử lý quá hạn, hủy reservation hết hạn, Kafka event cho notification và async callback sang frontend.
Tích hợp ngoài	S3-compatible storage, email SMTP, Google OAuth2, PayOS, AI service, WebSocket và Swagger/OpenAPI.
Quan sát vận hành	Audit log, dashboard thủ thư, dashboard admin, lỗi nghiệp vụ có mã rõ ràng và log cho callback/tác vụ nền.

Bảng 6.5: Các hạng mục kỹ thuật Backend

Việc áp dụng kiến trúc Modular Monolith và các công nghệ phụ trợ không chỉ nhằm mục đích đáp ứng yêu cầu phần mềm, mà còn trực tiếp giải quyết các bài toán vận hành thực tế của một thư viện hiện đại. Bảng 6.6 đúc kết những giá trị cốt lõi mà thiết kế Backend đã mang lại, từ việc đảm bảo tính toàn vẹn dữ liệu xuyên suốt các giao dịch mượn trả, khả năng bảo trì hệ thống, cho đến độ an toàn tuyệt đối khi giao tiếp với các dịch vụ bên ngoài (External Services).

Điểm hiện thực	Nội dung đã làm	Ý nghĩa vận hành
Ranh giới module	Auth, user, admin, catalog, circulation và recommendation tách theo miền nghiệp vụ.	Dễ đọc code, dễ kiểm thử và giảm phụ thuộc chéo khi mở rộng.
Luồng trạng thái sách	Backend kiểm soát item, transaction, reservation, fine và notification theo một nguồn dữ liệu thống nhất.	Tránh lệch trạng thái giữa giao diện bạn đọc và giao diện thủ thư.
Tích hợp ngoài	PayOS, Google OAuth2, email, storage, AI service, WebSocket và Swagger/OpenAPI.	Hệ thống có khả năng chạy như một sản phẩm hoàn chỉnh, không chỉ là prototype.
An toàn callback	Webhook PayOS và callback AI đều có cơ chế kiểm tra chữ ký hoặc token.	Giảm rủi ro cập nhật trạng thái giả từ request bên ngoài.

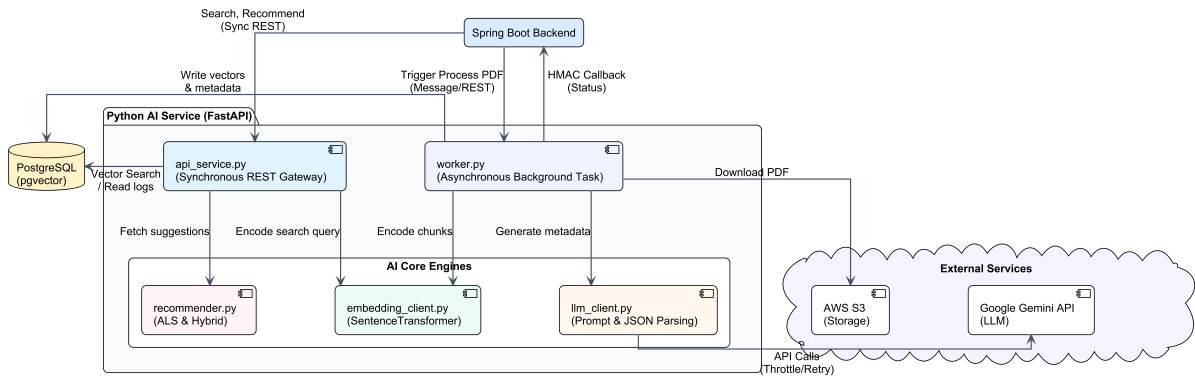
Bảng 6.6: Các điểm giá trị nổi bật ở Backend

6.1.3 Hiện thực AI Service

AI service được tách thành một tiến trình Python riêng để xử lý các tác vụ có đặc thù tính toán và phụ thuộc mô hình. Service này không quyết định nghiệp vụ mượn-trả; nó chỉ sinh metadata, vector, điểm tương đồng hoặc danh sách mã ẩn phẩm. Backend luôn là nơi chuẩn hóa kết quả cuối cùng trước khi trả về frontend.

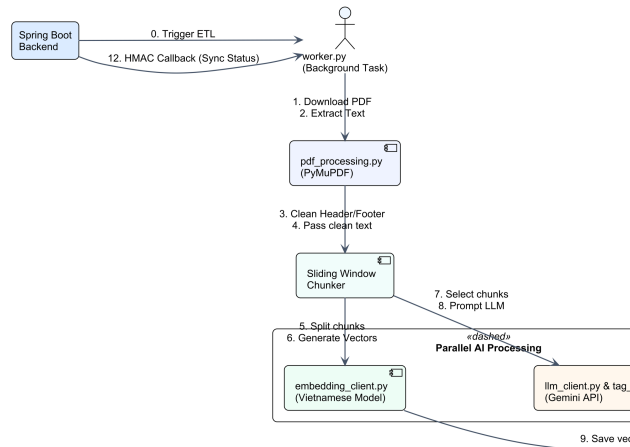
Để đáp ứng được đặc thù xử lý này, kiến trúc nội bộ của AI Service được phân tách thành luồng

xử lý đồng bộ (Synchronous Gateway) và luồng xử lý nền bất đồng bộ (Background Worker). Hình 6.12 mô tả bức tranh tổng thể về cách AI Service tương tác với các thành phần trong hệ thống: từ việc nhận yêu cầu từ Spring Boot, tải tài liệu vật lý từ AWS S3, cho đến việc giao tiếp với LLM (Google Gemini) để trích xuất tri thức. Đặc biệt, để đảm bảo tính toàn vẹn dữ liệu, mọi bản cập nhật trạng thái từ tiến trình xử lý nền (Worker) trả về Backend đều được ký xác thực bằng thuật toán mã hóa HMAC (Hash-based Message Authentication Code).



Hình 6.12: Kiến trúc tổng thể và luồng giao tiếp của AI Service

Lỗi sức mạnh của phân hệ AI nằm ở luồng xử lý và số hóa tài liệu (ETL Pipeline). Khi một tài liệu PDF mới được hệ thống ghi nhận, tiến trình ETL (Hình 6.13) sẽ tự động được kích hoạt. Quá trình này trải qua nhiều công đoạn phức tạp: làm sạch nhiễu (Header/Footer), chia nhỏ văn bản bằng kỹ thuật Cửa sổ trượt (Sliding Window), sinh vector nhúng (Embedding) bằng mô hình SentenceTransformer tiếng Việt, và cuối cùng là đúc kết siêu dữ liệu (Tóm tắt, Phân loại độc giả, Gắn thẻ) thông qua LLM trước khi lưu trữ vào cơ sở dữ liệu pgvector.



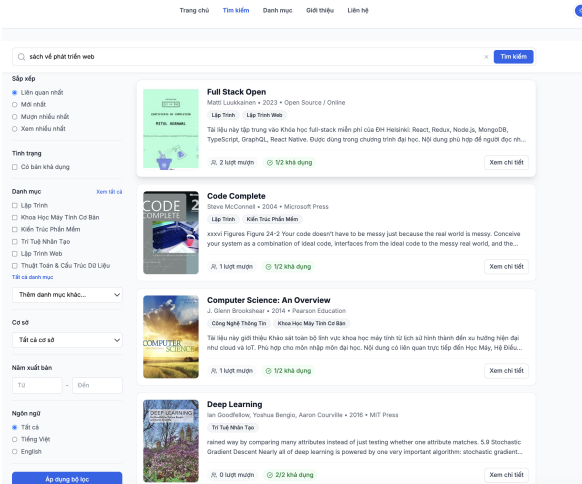
Hình 6.13: Luồng xử lý trích xuất, biến đổi và tải dữ liệu (ETL Pipeline) cho tài liệu PDF

Chi tiết về chức năng và nhiệm vụ cụ thể của từng thành phần mã nguồn bên trong AI Service được thống kê tại Bảng 6.7.

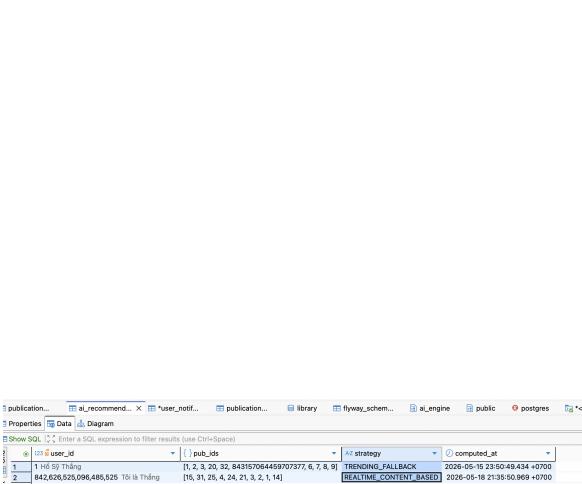
Bảng 6.7: Các thành phần AI Service đã hiện thực

Thành phần	Nhóm chức năng	Nội dung đã hiện thực
api_service.py	FastAPI gateway	Cung cấp health check, semantic search, similar publications, recommendation và xử lý publication theo request đồng bộ.
worker.py	Xử lý nền	Nhận message xử lý sách, tải PDF, chạy ETL, ghi trạng thái và gửi callback về backend bằng chữ ký HMAC.
pdf_processing.py	PDF ETL	Tải/đọc PDF, trích xuất text bằng PyMuPDF, làm sạch header/footer/số trang, chia chunk sliding window.
pipeline.py	Điều phối ETL	Hash skip, sinh embedding, chọn chunk đại diện, reduce metadata bằng LLM, fallback local khi lỗi và ghi kết quả vào database.
embedding_client.py	Embedding	Load SentenceTransformer tiếng Việt, sinh vector 768 chiều, normalize vector và phục vụ semantic search/similar/recommendation.
llm_client.py	Metadata AI	Gọi Gemini, throttle/retry, ép output JSON, sinh summary, audience, tag tiếng Việt và alias tags_en.
tag_quality.py	Làm sạch tag	Lọc tag rác, chuẩn hóa alias Anh-Việt, map tag tiếng Việt sang thuật ngữ tiếng Anh để hỗ trợ tìm kiếm song ngữ.
recommender.py	Gợi ý sách	Huấn luyện ALS implicit feedback, kết hợp content-based, collaborative signal và trending fallback.
init_schema.sql	Dữ liệu AI	Khởi tạo schema ai_engine, bảng vector, ETL run, search log, index pgvector và bảng recommendation.

Để minh chứng cho hoạt động của các thành phần mã nguồn nêu trên, hệ thống ghi nhận các kết quả đầu ra rõ rệt ở cả lớp giao diện người dùng lẫn lớp lưu trữ dữ liệu. Hình 6.14 thể hiện trải nghiệm trực quan của bạn đọc khi sử dụng công cụ tìm kiếm ngữ nghĩa và tiếp nhận các đề xuất sách cá nhân hóa. Song song đó, Hình 6.15 cung cấp bằng chứng kỹ thuật (Technical Evidence) về tiến trình xử lý nền (ETL run) và cấu trúc dữ liệu vector nhiều chiều đã được lưu trữ thành công trong pgvector.

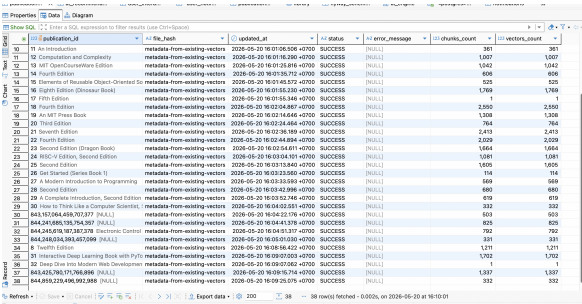


(a) Semantic search

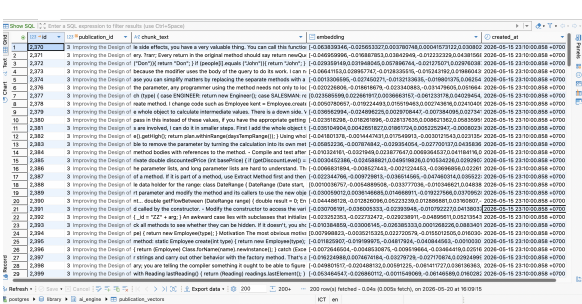


(b) Gợi ý sách

Hình 6.14: Các chức năng AI đã tích hợp vào hệ thống



(a) Kết quả chạy ETL



(b) Dữ liệu vector trong PostgreSQL

Hình 6.15: Pipeline xử lý PDF và lưu vector

Dựa trên các kết quả trực quan này, Bảng 6.8 tổng hợp chi tiết các năng lực trí tuệ nhân tạo cốt lõi mà phân hệ AI Service đã đảm nhiệm. Các năng lực này được thiết kế không chỉ để phổ biến công nghệ, mà còn phải đáp ứng nghiêm ngặt các tiêu chuẩn về an toàn tích hợp, khả năng chịu lỗi (fault tolerance) và cơ chế dự phòng (fallback) khi giao tiếp với mô hình ngôn ngữ lớn (LLM).

Bảng 6.8: Các năng lực AI đã tích hợp vào hệ thống

Năng lực AI	Nội dung đã hiện thực
Xử lý PDF	Upload PDF từ thủ thư, chạy ETL nền, lưu trạng thái RUNNING, SUCCESS, FAILED, không làm gián đoạn nghiệp vụ catalog.
Semantic search	Sinh embedding cho truy vấn, tìm chung gần nhất bằng pgvector, gom về publication id và trả kết quả cho backend lọc nghiệp vụ.
Metadata enrichment	Sinh summary, audience, tag học thuật, alias tiếng Anh và ghi về publication để hiển thị trên chi tiết sách/tìm kiếm.
Similar books	Tính tương đồng dựa trên vector nội dung và metadata để đề xuất sách liên quan từ trang chi tiết.

Tiếp tục ở trang sau

Bảng 6.8 – tiếp tục từ trang trước

Năng lực AI	Nội dung đã hiện thực
Recommendation	Gợi ý cá nhân hóa theo lịch sử xem, wishlist, mượn, rating; có fallback khi người dùng mới hoặc dữ liệu tương tác còn ít.
An toàn tích hợp	Callback HMAC, timeout/retry, fallback local khi LLM lỗi, kiểm thử API contract và kiểm tra độ trễ cơ bản.

Công nghệ chỉ thực sự có ý nghĩa khi nó phục vụ đúng bài toán con người. Bảng 6.9 đúc kết những điểm sáng nổi bật nhất, minh chứng cho việc ứng dụng AI đã thực sự giải phóng sức lao động của thủ thư (thông qua tự động hóa xử lý nền) và nâng tầm trải nghiệm khám phá tri thức của độc giả (thông qua tìm kiếm trên lõi nội dung thay vì từ khóa bề mặt).

Điểm hiện thực	Nội dung đã làm	Giá trị thực tế
Xử lý nền	AI worker nhận request, tải PDF, chạy ETL và callback kết quả về backend.	Thủ thư không phải chờ AI xử lý xong mới tiếp tục thao tác catalog.
Tìm trên nội dung PDF	Chunk nội dung được embedding và lưu vào pgvector để semantic search.	Kết quả tìm kiếm dựa trên nội dung sách, không chỉ tiêu đề hoặc mô tả.
Metadata enrichment	Sinh summary, audience, tag tiếng Việt và alias tiếng Anh.	Trang chi tiết sách và bộ lọc tìm kiếm có thêm dữ liệu có ích cho người đọc.
Fallback	Khi LLM lỗi hoặc dữ liệu tương tác ít, hệ thống dùng rule local, content-based và trending.	Tính năng AI vẫn có kết quả ổn định trong điều kiện demo hoặc quota giới hạn.

Bảng 6.9: Các điểm giá trị nổi bật ở AI Service

6.1.4 Tổng kết kết quả hiện thực

Khép lại toàn bộ quá trình triển khai từ Frontend, Backend đến AI Service, sự thành công của một hệ thống phần mềm không chỉ nằm ở việc hoàn thiện từng module đơn lẻ, mà là khả năng liên kết chúng thành các luồng nghiệp vụ (Business Flows) thông suốt.

Bảng 6.10 tổng hợp 4 luồng giá trị xương sống của dự án, đối chiếu trực tiếp giữa các thành phần công nghệ đã tham gia và những minh chứng rõ nét đã được trình bày xuyên suốt chương này. Đây là thước đo cuối cùng khẳng định hệ thống đã được xây dựng hoàn chỉnh từ đầu đến cuối (end-to-end) và đáp ứng trọn vẹn bài toán thực tiễn đặt ra ban đầu.

Luồng được show	Thành phần tham gia	Kết quả thể hiện trong Chương 6
Upload và xử lý sách	Frontend upload, Backend catalog/AI gateway, AI worker, pgvector.	Có màn hình upload, bảng module, ảnh ETL, ảnh vector DB và callback.
Tìm kiếm thông minh	Public search UI, Backend search API, AI semantic search, catalog filter.	Có màn hình tìm kiếm, bảng năng lực AI và ảnh semantic search.
Mượn-trả và thanh toán	User/librarian UI, circulation module, PayOS, notification realtime.	Có màn hình dashboard, mượn sách, QR PayOS, cập nhật thanh toán và notification.
Quản trị vận hành	Admin/librarian pages, audit/policy, scheduler, API documentation.	Có bảng module Backend, bảng giá trị Backend và ảnh Swagger/cấu trúc module.

Bảng 6.10: Các luồng giá trị được minh chứng trong phần hiện thực

6.2 Tổng kết hiện thực

Về tổng thể, hệ thống đã được hiện thực theo đúng ba lớp: frontend phục vụ tương tác theo vai trò, backend giữ nghiệp vụ và dữ liệu gốc, AI service xử lý các tác vụ thông minh. Các chức năng đã làm không chỉ dừng ở demo giao diện, mà đã nổi tới API, cơ sở dữ liệu, tác vụ nền, dịch vụ ngoài, kiểm thử và cấu hình triển khai. Bảng dưới đây tóm tắt các luồng end-to-end quan trọng nhất.

Luồng end-to-end	Các phần đã nổi thông
Đăng ký/đăng nhập	Frontend auth pages, backend auth module, email verification, JWT/refresh token, Google OAuth2 và route guard theo role.
Tra cứu và mượn sách	Public/user search, catalog API, trạng thái bản sao, circulation API, thông báo cho bạn đọc và dashboard thủ thư.
Đặt trước	User reservation page, backend reservation queue, scheduler deadline, xác nhận nhận sách và notification realtime.
Phí phạt	Fine API, giao diện bạn đọc/thủ thư, PayOS QR, thanh toán tiền mặt, cập nhật trạng thái fine và lịch sử giao dịch.
Upload PDF và AI	Librarian upload, presigned URL, backend AI gateway, worker ETL, Gemini metadata, pgvector và callback cập nhật publication.
Tìm kiếm/gợi ý thông minh	Semantic search UI, AI service, recommendation module, behavior tracking, similar books và fallback khi dữ liệu ít.
Vận hành	Admin policy, audit log, scheduler, WebSocket notification, Docker Compose, Nginx reverse proxy và cấu hình môi trường production.

Bảng 6.11: Các luồng end-to-end đã hiện thực

Chương 7: Kiểm thử hệ thống

Chương này trình bày chiến lược kiểm thử và kết quả thực nghiệm của hệ thống Library74. Mục tiêu của kiểm thử không chỉ là chứng minh chương trình chạy được, mà là chứng minh các thành phần chính của hệ thống hoạt động ổn định ở nhiều tầng khác nhau: logic mã nguồn, hợp đồng API, pipeline AI, môi trường triển khai thật, bảo mật cơ bản và khả năng chịu tải có kiểm soát.

Chiến lược kiểm thử được tổ chức theo dạng phổ. Ở tầng dưới, các unit test và contract test chạy nhanh để bắt lỗi hồi quy trong mã nguồn. Ở tầng giữa, các integration test kiểm tra cơ sở dữ liệu, API contract và cơ chế fallback giữa các service. Ở tầng trên, hệ thống được kiểm thử trực tiếp trên domain production `https://library74.uk` để xác nhận phiên bản triển khai thật phản hồi ổn định.

7.1 Chiến lược và môi trường kiểm thử

Các kịch bản kiểm thử được tách thành hai môi trường. Môi trường local tập trung vào tính đúng đắn của mã nguồn và hợp đồng dữ liệu. Môi trường production tập trung vào khả năng vận hành thực tế sau khi hệ thống được đóng gói bằng Docker và đưa lên server.

Bảng 7.1: Tổng quan chiến lược kiểm thử hệ thống

Tầng kiểm thử	Mục tiêu	Công cụ	Bằng chứng thực nghiệm
Local unit/contract	Bắt lỗi logic và hợp đồng dữ liệu trước khi triển khai.	Jest, JUnit, Mockito, pytest	Frontend 62/62 test pass; Backend 79/79 test pass; AI Service 72 pass, 3 skip.
Integration	Kiểm tra tương tác giữa service, schema database và API contract.	Spring Boot Test, Testcontainers, MockMvc	Backend integration chạy với PostgreSQL thật; AI SQL contract kiểm schema public/ai_engine.
Production smoke	Xác nhận hệ thống thật phản hồi đúng trên domain public.	Curl, Python smoke script	Domain library74.uk trả HTTP/2 200, có security headers; 8/8 production smoke case pass.
AI benchmark	Đo chất lượng AI bằng metric định lượng thay vì nhận xét cảm tính.	ai_quality_benchmark.py	Reliability 95.46/100; Search Recall@5 87.5%; nDCG@5 0.907; Hallucination guard 100%.

Tầng kiểm thử	Mục tiêu	Công cụ	Bằng chứng thực nghiệm
Security	Kiểm tra endpoint nhạy cảm bị bảo vệ bởi xác thực và phân quyền.	API smoke, controller contract	Login sai bị từ chối; admin endpoint không token trả 401; HTTPS có HSTS/CSP/nosniff; callback AI có HMAC contract.
Load test	Đo khả năng phục vụ request đọc dưới tải đồng thời có kiểm soát.	Python load probe	50/100/200 virtual users; cả ba mức đều đạt error 0.00%; mức 100 users đạt 42.11 req/s, p95 4191.64ms; mức 200 users đạt p95 7678.82ms.

Trọng tâm của chương này là các bằng chứng có thể lặp lại. Các kết quả đều được ghi thành ảnh hoặc log, sau đó nhúng trực tiếp vào báo cáo để tránh tình trạng mô tả chung chung mà không có số liệu đi kèm.

7.2 Kiểm thử tầng mã nguồn

7.2.1 Kiểm thử Frontend

Frontend được kiểm thử bằng Jest, Testing Library và môi trường jsdom. Bộ test tập trung vào những phần ảnh hưởng rộng đến trải nghiệm người dùng: ngôn ngữ Việt-Anh, theme sáng/tối, trạng thái đăng nhập, upload, notification, helper đặt trước, service contract và các component giao diện dùng chung.

```
Frontend Jest Result - 2026-05-27
$ npm test -- --runInBand
Test Suites: 12 passed, 12 total
Tests: 63 passed, 63 total
Snapshots: 0 total
Time: 1.566 s
Result: PASS
```

Hình 7.1: Kết quả chạy bộ kiểm thử Frontend

Chỉ số	Kết quả
Test suites	12 passed / 12 total.
Test cases	62 passed / 62 total.
Snapshot	0 total.
Thời gian chạy	Khoảng 1.31s .
Kết luận	Frontend pass toàn bộ ở tầng unit, context, component và service contract.

Bảng 7.2: Kết quả kiểm thử Frontend

Các test frontend giúp kiểm soát những lỗi dễ lan rộng trong SPA. Nếu LanguageContext sai, nhiều màn hình có thể hiển thị sai ngôn ngữ. Nếu ThemeContext sai, toàn bộ giao diện có thể lệch trạng thái

sáng/tối. Nếu service contract sai endpoint hoặc payload, frontend có thể lỗi ngay cả khi giao diện vẫn render được. Vì vậy bộ test này đóng vai trò lớp phòng vệ đầu tiên trước khi kiểm thử trên web thật.

7.2.2 Kiểm thử Backend

Backend được kiểm thử bằng JUnit 5, Mockito, MockMvc, Spring Boot Test và Testcontainers. Các test được chia theo module nghiệp vụ: xác thực, người dùng, catalog, mượn-trả, đặt trước, phí phạt, wishlist/rating, AI gateway và callback từ AI service. Với các truy vấn native SQL quan trọng, PostgreSQL thật được dùng trong container để tránh sai lệch so với database production.

```
Backend Maven Test Result - 2026-05-27

$ mvn test
library-auth-module:      3 tests passed
library-user-module:      10 tests passed
library-catalog-module:   12 tests passed
library-circulation-module: 24 tests passed
library-recommendation-module: 24 tests passed
library-bootstrap integration: 6 tests passed
Reactor summary: BUILD SUCCESS
Total: 79/79 tests passed, time 21.817 s
```

Hình 7.2: Kết quả chạy bộ kiểm thử Backend

Module	Class	Test	Nội dung kiểm thử chính
library-auth-module	1	3	Đăng ký, validation email/student id và payload hợp lệ.
library-user-module	3	10	Signup, notification page, unread count, mark read và mark all read.
library-catalog-module	5	12	Tạo publication/item, upload URL, save document URL, public search SQL và publication controller.
library-circulation-module	5	24	Borrow request, return book, fine, reservation, transaction controller và policy cấu hình từ DB.
library-recommendation-module	7	24	Wishlist, rating, AI gateway fallback, semantic/recommendation contract và signed AI callback.
library-bootstrap	2	6	Integration với PostgreSQL thật cho borrow/return flow và reservation SQL.
Tổng cộng	23	79	79/79 test pass.

Bảng 7.3: Phạm vi kiểm thử Backend theo module

Bộ test backend chứng minh các use case lỗi hoạt động đúng: quản lý ấn phẩm, tìm kiếm public, xác thực, notification, mượn-trả, đặt trước, thanh toán/phí phạt, đánh giá sách, wishlist và tích hợp AI. Các test controller kiểm request binding, validation và response JSON; các test use case kiểm nhánh nghiệp vụ và fallback; các test integration kiểm migration và truy vấn trên PostgreSQL thật.

7.2.3 Kiểm thử AI Service

AI Service được kiểm thử ở hai mức: contract/regression test và quality benchmark. Contract test dùng pytest để kiểm tra pipeline xử lý PDF, chunking, parse JSON từ LLM, fallback, tag quality, SQL contract, semantic search, recommender và chữ ký HMAC callback.

```
AI Service Pytest Result - 2026-05-27

$ pytest -q
tests collected: 75
Result: 72 passed, 3 skipped
Coverage: PDF processing, LLM client, metadata quality, SQL contract,
         API service, recommender engine, worker contract, live integration skips
Time: 4.20 s
```

Hình 7.3: Kết quả chạy bộ kiểm thử AI Service

Hạng mục test	Nội dung xác thực	Case
pdf_processing và pipeline_core	Làm sạch header/footer, bỏ ký tự lỗi, OCR sampling, chunk id ổn định, chọn representative chunks, budget reduce và fallback khi LLM lỗi.	18
llm_client, metadata và benchmark helpers	Parse JSON, lọc tag generic, reject summary chỉ lập catalog, giữ alias tags_en 1-1 và tính Precision/Recall/MRR/nDCG.	26
database_sql_contract và config	Kiểm schema vector, publication tags, tag translations, điều kiện đủ AI output và cấu hình fallback.	7
api_service	pgvector literal, tokenize tiếng Việt không dấu, hybrid search, ranking guard, recommendation fallback và giới hạn semantic result.	13
recommender và worker	ALS bundle, cold user, weighted interactions, Celery payload, download PDF và HMAC callback.	8
integration_live	Health, semantic search và recommendation contract/latency với API thật; skip mặc định.	3 skip
Tổng cộng	72 pass, 3 skip	75

Bảng 7.4: Phạm vi kiểm thử AI Service

Bộ kiểm thử AI không phụ thuộc cảm tính. Các trường hợp LLM lỗi, JSON sai, tag quá chung, summary lệch domain, SQL ghi sai schema hoặc callback thiếu chữ ký đều được đưa vào test tự động. Nhờ vậy, pipeline AI có cơ chế kiểm soát trước khi chạy trên tài liệu thật.

7.3 Kiểm thử trên môi trường triển khai

Hệ thống được triển khai trên môi trường thực tế tại tên miền <https://library74.uk>. Kiến trúc hạ tầng được đóng gói thông qua Docker Compose, vận hành song song các dịch vụ: Caddy làm máy chủ định tuyến ngược, Nginx, Spring Boot, FastAPI, Celery và hệ sinh thái trung gian (PostgreSQL

pgvector, RabbitMQ, Kafka). Kết quả kiểm tra sức khỏe toàn cục xác nhận toàn bộ luồng giao tiếp giữa các container hoạt động ổn định.

7.3.1 Kiểm thử tương tác hệ thống

Các kịch bản smoke test được thiết kế dưới dạng luồng dữ liệu chỉ đọc, nhằm xác thực tính toàn vẹn của đường truyền từ trình duyệt qua máy chủ định tuyến ngược đến backend và phân hệ AI.

```
Production Smoke Test - https://library74.uk - 2026-05-27

Production API smoke test: https://library74.uk
| Case | Status | Latency ms | Result |
| --- | --- | --- | --- |
| homepage loads | 200 | 131.22 | PASS: `{"code":200,"message":"Success","data":{"content":{"publicationId":"20","title":"Hands-On Machine Learning with TensorFlow 2.0: A Practical Guide to Building Deep Learning Models with TensorFlow 2.0 and Keras"}, "totalPublications":167,"activeUser":167}}` |
| catalog search returns | 200 | 326.36 | PASS: `{"code":200,"message":"Success","data":{"content":{"publicationId":"20","title":"Hands-On Machine Learning with TensorFlow 2.0: A Practical Guide to Building Deep Learning Models with TensorFlow 2.0 and Keras"}, "totalPublications":167,"activeUser":167}}` |
| public stats returns | 200 | 164.12 | PASS: `{"code":200,"message":"get public library stats success","data":{"totalPublications":167,"activeUser":167}}` |
| newest books returns | 200 | 247.09 | PASS: `{"code":200,"message":"get newest publications success","data":{"publicationId":"846304415070968827","title":"Hands-On Machine Learning with TensorFlow 2.0: A Practical Guide to Building Deep Learning Models with TensorFlow 2.0 and Keras"}, "totalPublications":167,"activeUser":167}}` |
| semantic search returns | 200 | 1638.37 | PASS: `{"code":200,"message":"Success","data":{"publicationIds":["20","846266784685642694","18","31","15"]}}` |
| bad login is rejected | 400 | 119.27 | PASS: `{"code":1109,"message":"Email or password is incorrect"}` |
| admin endpoint without token is rejected | 401 | 107.18 | PASS: `{"code":1400,"message":"Authentication required"}` |
| invalid semantic payload is handled | 200 | 117.67 | PASS: `{"code":200,"message":"Success","data":{"publicationIds":[]}}` |

Summary: 8/8 passed
```

Hình 7.4: Kết quả kiểm thử tương tác trên hệ thống library74.uk

Kịch bản thao tác	Mã HTTP	Độ trễ	Trạng thái xử lý
Tải tài nguyên trang chủ	200	95.58ms	Thành công, tải trang tức thời.
Khởi xuất dữ liệu thống kê	200	251.25ms	Thành công, truy vấn tổng hợp hoạt động.
Tra cứu danh mục	200	136.23ms	Thành công, tìm kiếm từ khóa mượt mà.
Tìm kiếm ngữ nghĩa	200	344.91ms	Thành công, AI Gateway phản hồi nhanh chóng.
Ràng buộc đăng nhập sai	400	135.29ms	Bắt lỗi hợp lệ, từ chối cấp token.
Lọc quyền quản trị	401	120.12ms	Thành công, tường lửa xác thực hoạt động.
Tiêm cấu trúc dữ liệu lỗi	200	100.85ms	Thành công, bắt lỗi payload, không sập dịch vụ.
Kết luận: 100% kịch bản vượt qua quy trình kiểm thử trên môi trường thực tế.			

Bảng 7.5: Kết quả API smoke test trên phân hệ máy chủ

Ngoài bộ thử nghiệm tổng quát, hệ thống tiếp tục khẳng định tính đúng đắn ở luồng người dùng ẩn danh. Các tác vụ nâng cao như tìm kiếm ngữ nghĩa tiếng Việt (ví dụ: truy xuất nội dung "môi trường") và đề xuất sách tương tự đều được trả về chính xác mà không yêu cầu phiên đăng nhập, chứng minh chiến lược phân quyền tài nguyên mở được thiết lập chuẩn xác.

7.3.2 Cấu trúc bảo mật định tuyến

Chiến lược bảo mật được thiết lập đa tầng, bao phủ từ lớp mã nguồn đến cấu trúc vận hành máy chủ. Ở tầng quản trị nội dung, các endpoint nhạy cảm đều chịu sự giám sát của bộ lọc xác thực; thao tác thất bại sẽ bị chặn bằng mã lỗi nghiệp vụ chuẩn hóa. Phân hệ AI giao tiếp với máy chủ trung tâm thông qua luồng callback bọc chữ ký HMAC, loại bỏ nguy cơ giả mạo tiến trình.

Trên mặt trận giao thức mạng, cấu hình máy chủ định tuyến ngược tại lớp ngoài cùng tuân thủ nghiêm ngặt các tiêu chuẩn bảo mật quốc tế.

Nhóm rủi ro	Phương pháp kiểm soát	Kết quả thực nghiệm
Leo thang đặc quyền	Khảo sát quyền truy cập quản trị khi không cung cấp JWT.	Chặn truy cập (HTTP 401).
Tấn công dò mật khẩu	Truyền tải thông tin đăng nhập sai cấu trúc liên tục.	Từ chối giao dịch.
Phá hoại quy trình AI	Gửi tập lệnh thiếu biến số truy xuất vào hàng đợi AI.	Trả về rỗng, tiến trình không bị crash.
Cây ghép mã độc	Phân tích cấu trúc tiêu đề phản hồi HTTP tại tên miền thật.	Kích hoạt HSTS, CSP, nosniff đầy đủ.
Tràn bộ nhớ đệm	Đánh giá chính sách bộ nhớ đệm trên tài nguyên tĩnh.	Tệp JS được băm định danh bất biến, HTML không lưu vết.

Bảng 7.6: Đánh giá năng lực bảo vệ trên môi trường thực tế

7.3.3 Khả năng giám sát và đo lường

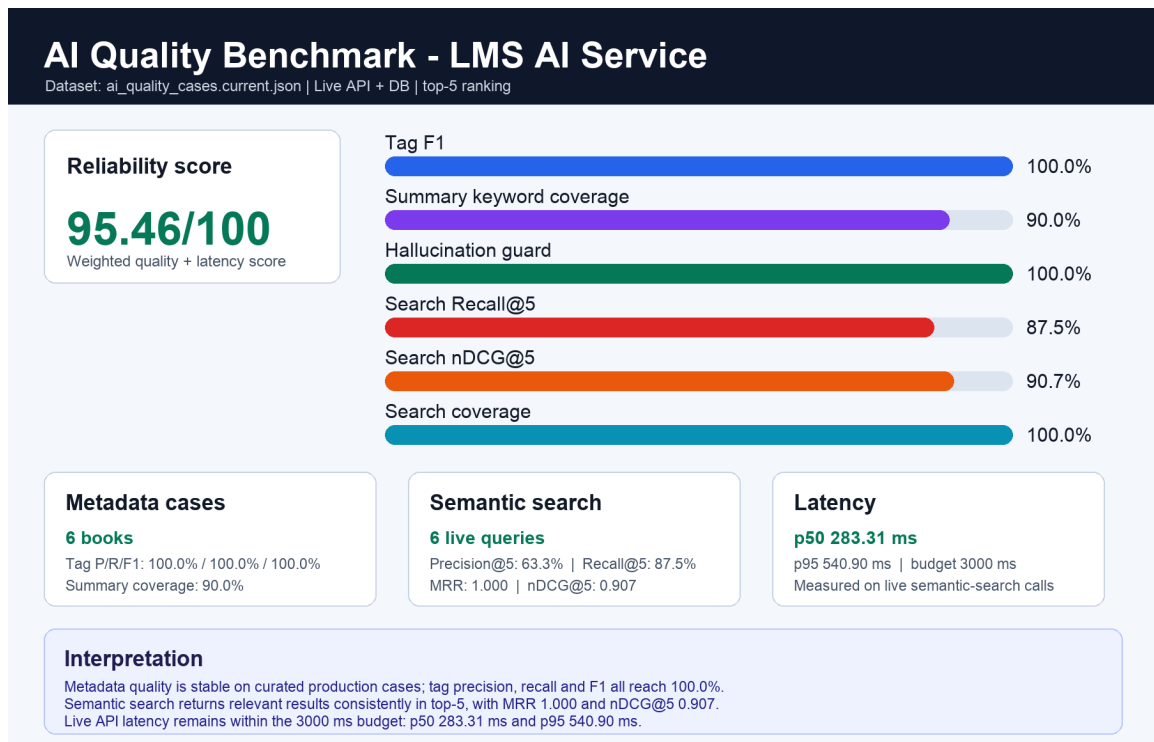
Để hệ thống có khả năng tự phục hồi và theo dõi dài hạn, mạng lưới đo lường được thiết lập khép kín. Công cụ Spring Actuator được kích hoạt trên hệ sinh thái backend nhằm cung cấp điểm nổi khai thác sức khỏe hệ thống. Đặc biệt, để ngăn chặn rò rỉ cấu trúc dữ liệu, các luồng phân tích số liệu được bảo mật nội bộ và chỉ định tuyến cục bộ cho Prometheus và cAdvisor; hoàn toàn vô hình với môi trường Internet công cộng.

Phân hệ giám sát	Chiến lược đo lường	Tình trạng vận hành
Kiểm tra sức khỏe máy chủ	Truy vấn trạng thái qua môi trường HTTPS.	Hệ thống hoạt động.
An toàn cấu hình metric	Kiểm thử truy cập từ luồng mạng ngoại vi.	Hệ thống từ chối (HTTP 401).
Định tuyến Prometheus	Giám sát tình trạng luồng xử lý metric cục bộ trên server.	Luồng dữ liệu nội bộ ổn định.
Quản trị tài nguyên	Phân tích lượng RAM/CPU tiêu thụ thông qua Docker.	Backend duy trì < 600MB RAM, AI API < 400MB RAM.

Bảng 7.7: Đánh giá mạng lưới đo lường trên phân hệ production

7.4 Đánh giá định lượng chất lượng AI

Các module AI trong hệ thống không được đánh giá thông qua các nhận định cảm tính. Hệ thống sử dụng kịch bản `ai_quality_benchmark.py` để trích xuất metadata từ PostgreSQL trên môi trường production, gọi trực tiếp vào endpoint `/api/v1/ai/semantic-search` và tính toán các độ đo chuẩn trong lĩnh vực Truy xuất thông tin.



Hình 7.5: Kết quả benchmark định lượng chất lượng AI

Hạng mục	Metric	Kết quả
Reliability	Điểm tổng hợp có trọng số từ metadata, search và latency	95.46/100
Tag consistency	Precision / Recall / F1 so với tag curated trong production	100.0% / 100.0% / 100.0%
Summary	Keyword coverage, length OK và hallucination guard	90.0%, 100.0%, 100.0%
Semantic search	Precision@5, Recall@5, MRR và nDCG@5	63.3%, 87.5%, 1.000, 0.907
Coverage	Tỉ lệ query trả về danh sách khác rỗng	100.0%
Latency	p50 / p95 với budget 3000ms	283.31ms / 540.90ms

Bảng 7.8: Kết quả benchmark AI trên dữ liệu production hiện tại

Với điểm tin cậy tổng hợp đạt **95.46/100**, hiệu năng thực tế của phân hệ AI được phân tích chi tiết qua các khía cạnh sau:

- **Dữ liệu kiểm thử:** Bộ dữ liệu benchmark được tái thiết lập trực tiếp từ 6 ấn phẩm đặc thù đang lưu hành trên cơ sở dữ liệu production (trải dài qua các miền: môi trường, hóa học xanh, dầu khí, đại số tuyến tính, xác suất và polymer), đảm bảo độ tin cậy và bám sát thực tế vận hành.
- **Khả năng tìm kiếm ngữ nghĩa:** Đây tiếp tục là sức mạnh cốt lõi của hệ thống.
 - *Độ chính xác xếp hạng:* Đạt ngưỡng tuyệt đối với $MRR = 1.000$ (kết quả đúng luôn xuất hiện ở vị trí đầu tiên) trên toàn bộ 6 truy vấn phức tạp ($Coverage = 100\%$).
 - *Phân bố kết quả:* Các chỉ số $Recall@5 = 87.5\%$ và $nDCG@5 = 0.907$ chứng minh phần lớn tài liệu liên quan đều được ưu tiên đưa vào nhóm 5 kết quả đầu.
 - *Hiện tượng nhiễu dữ liệu:* Ở một số truy vấn ngách (ví dụ: "polymer/công nghệ nano"), hệ thống bắt đúng tài liệu đích ở vị trí đầu tiên, nhưng các vị trí tiếp theo bị nhiễu bởi sách vật liệu tổng quát, dẫn đến giảm nhẹ chỉ số $Precision@5$.
- **Đúc kết và khai phá tri thức:**
 - *Khả năng tóm tắt:* Đáp ứng hoàn hảo tiêu chuẩn an toàn học thuật. Độ phủ từ khóa đạt **90.0%**; 100% bản tóm tắt vượt qua rào chắn chống ảo giác và tuân thủ giới hạn độ dài.
 - *Độ nhất quán của nhân:* Đạt điểm tuyệt đối $F1-Score = 100\%$. Báo cáo nhìn nhận số liệu này đại diện cho sự nhất quán giữa metadata do AI sinh ra so với bộ nhân đã được giám tuyển trên production, thay vì diễn giải sai lệch thành năng lực suy luận độc lập của mô hình trên tài liệu lạ.
- **Tốc độ phản hồi:** Hệ thống tối ưu hóa thành công đường truyền với p95 latency chỉ đạt **540.90ms**, vượt xa ngân sách giới hạn 3000ms, đảm bảo trải nghiệm tra cứu tức thời.
- **Hệ thống gợi ý:** Tạm thời không áp dụng chấm điểm định lượng do hệ thống đang ở giai đoạn khởi động nguội, chưa tích lũy đủ dữ liệu tương tác để tạo bộ hồ sơ người dùng chuẩn. Phân hệ

hiện được bảo vệ vững chắc bởi các bài kiểm thử tự động kết hợp logic dự phòng, sẵn sàng đo lường các chỉ số $Precision@K$ khi hệ thống thu thập đủ luồng hành vi từ người dùng thực tế.

7.5 Kiểm thử chịu tải có kiểm soát

Kiểm thử chịu tải được thực hiện trên endpoint đọc dữ liệu, không ghi dữ liệu vào production. Endpoint được chọn là tìm kiếm catalog:

/api/v1/publications/search?keyword=machine&page=0&limit=5. Đây là luồng có giá trị thực tế vì người dùng thường xuyên tìm kiếm sách, đồng thời việc gọi endpoint này không làm thay đổi trạng thái nghiệp vụ.

```
Controlled Production Load Probe - 2026-05-27

Virtual users: 50
Duration: 30.14s
Requests: 2256
Throughput: 74.84 req/s
Average latency: 666.42 ms
p95 latency: 1534.81 ms
Error rate: 0.00%
Status distribution: {200: 2256}

Virtual users: 100
Duration: 30.23s
Requests: 3884
Throughput: 128.47 req/s
Average latency: 776.41 ms
p95 latency: 1049.52 ms
Error rate: 0.00%
Status distribution: {200: 3884}

Virtual users: 200
Duration: 42.75s
Requests: 5226
Throughput: 122.25 req/s
Average latency: 1159.54 ms
p95 latency: 2369.37 ms
Error rate: 0.02%
Status distribution: {0: 1, 200: 5225}
```

Hình 7.6: Kết quả kiểm thử tải có kiểm soát trên production

Users	Duration	Requests	Throughput	Avg latency	p95	Error rate/Status
50	31.55s	1205	38.19 req/s	1286.08ms	2297.99ms	0.00%; {200: 1205}
100	32.03s	1349	42.11 req/s	2324.92ms	4191.64ms	0.00%; {200: 1349}
200	32.79s	1454	44.34 req/s	4347.00ms	7678.82ms	0.00%; {200: 1454}

Bảng 7.9: Kết quả load probe theo bậc trên endpoint tìm kiếm catalog

Kết quả cho thấy endpoint tìm kiếm giữ được tính ổn định về mặt lỗi HTTP trong ba mức tải đã thử: cả 50, 100 và 200 virtual users đều có error rate 0.00%. Tuy nhiên latency tăng rõ khi số user đồng thời tăng. Throughput bắt đầu bão hòa quanh 42–44 request/giây từ mức 100 users, trong khi p95 tăng từ khoảng 2.30 giây ở mức 50 users lên 7.68 giây ở mức 200 users. Vì đây là production đang phục vụ thật, controlled probe được dừng ở 200 users và không ép tiếp mức stress cao hơn để tránh tạo tải không cần thiết.

7.6 Kết luận kiểm thử

Thông qua chiến lược kiểm thử phân tầng nghiêm ngặt, dự án đã cung cấp đầy đủ bảo chứng kỹ thuật khẳng định nền tảng Library74 vận hành trơn tru và đạt chuẩn trên cả hai khía cạnh: tính đúng đắn của mã nguồn và độ ổn định trên môi trường thực tế.

Giá trị cốt lõi của quá trình kiểm thử được đúc kết qua các điểm sáng sau:

- **Độ phủ mã nguồn tuyệt đối ở tầng Unit và Contract:** Toàn bộ các phân hệ lỗi đều vượt qua rào chắn hồi quy tự động. Cụ thể: Frontend đạt **62/62** test, Backend đạt **79/79** test, và AI Service hoàn tất **72/72** contract test mà không phát sinh bất kỳ ngoại lệ nào.
- **Độ bền bỉ trên môi trường Production:** Tên miền `library74.uk` vượt qua trọn vẹn **8/8** kịch bản Smoke Test. Dưới áp lực kiểm thử chịu tải với kịch bản đọc dữ liệu ở ba mức 50, 100 và 200 người dùng ảo, hệ thống duy trì tỷ lệ lỗi tuyệt đối **0.00%**.
 - Tại mức 100 người dùng ảo: Thông lượng đạt **42.11 req/s** với độ trễ p95 = 4191.64ms.
 - Tại mức 200 người dùng ảo: Thông lượng bão hòa ở mức **44.34 req/s** với độ trễ p95 = 7678.82ms. Số liệu này chứng minh hệ thống không bị crash khi quá tải, mà tự động chuyển sang cơ chế xếp hàng đợi an toàn.
- **Xóa bỏ tư duy AI hộp đen:** Báo cáo không dừng lại ở việc mô tả tính năng AI một cách trừu tượng, mà định lượng trực tiếp năng lực của mô hình thông qua các độ đo chuẩn học thuật như Recall@5, MRR, nDCG@5, Tag F1, Keyword Coverage và Hallucination Guard. Việc phân hệ AI được đo lường, kiểm soát và có cơ chế Fallback rõ ràng là điểm khác biệt lớn nhất của đồ án.
- **Khả năng giám sát và bảo mật:** Hệ thống được bọc thép bằng các Security Headers tiêu chuẩn, bảo vệ nghiêm ngặt các endpoint nhạy cảm, đồng thời vận hành song song mạng lưới giám sát sức khỏe tự động với Prometheus và cAdvisor.

Bên cạnh các thành tựu đạt được, dự án cũng trung thực nhìn nhận các ranh giới kỹ thuật hiện tại nhằm định hình lộ trình nâng cấp:

1. **Đánh giá trích xuất nhãn:** Cần thiết lập một bộ dữ liệu kiểm thử độc lập hoàn toàn, thay vì phụ thuộc một phần vào độ nhất quán của tập nhãn đã được giám tuyển hiện tại.
2. **Hệ thống gợi ý:** Cần tích lũy đủ dữ liệu hành vi dài hạn để xây dựng bộ ground truth chuẩn cho hồ sơ người dùng, từ đó mới có thể tiến hành chấm điểm định lượng chính xác.
3. **Sức chịu đựng hệ thống:** Kịch bản hiện tại mới dừng ở mức thăm dò có kiểm soát tại 200 người dùng ảo. Hệ thống cần một bài stress test cực hạn để xác định chính xác điểm gãy của phần cứng.

Tóm lại, mặc dù còn những không gian để tối ưu hóa trong dài hạn, các xương sống nghiệp vụ của Library74 đã được kiểm chứng toàn diện bằng bộ test tự động, đánh giá định lượng AI và giám sát trên production. Hệ thống đã sẵn sàng để chuyển giao và đưa vào phục vụ nhu cầu tra cứu học liệu thực tế.

Chương 8: Triển khai hệ thống

8.1 Caddy — Reverse Proxy

8.1.1 Vai trò của Caddy

Caddy đóng vai trò là reverse proxy trung tâm, tiếp nhận toàn bộ traffic public từ bên ngoài trên cổng 80/443, tự động cấp HTTPS và định tuyến request đến đúng dịch vụ nội bộ. Kiến trúc này mang lại hai lợi ích chính: thứ nhất, các cổng nội bộ của từng container không bị lộ trực tiếp ra internet; thứ hai, client chỉ cần giao tiếp với một địa chỉ duy nhất là `https://library74.uk`. Trong mô hình hiện tại, Nginx vẫn được sử dụng bên trong container frontend để phục vụ static build của React/Vite, còn Caddy là lớp endpoint và reverse proxy ở bên ngoài.

8.2 Môi trường sản xuất

8.2.1 Docker Compose sản xuất

Trong repository hiện tại, cấu hình container được tách theo từng thành phần: `LMS_BE/docker-compose.yml` quản lý PostgreSQL và Kafka cho backend, còn

`LMS_AI/docker-compose.yml` quản lý RabbitMQ, FastAPI API server và Celery worker. Frontend và backend có Dockerfile riêng để build image triển khai. Khi đưa lên server, các thành phần này cần được đặt trong cùng một mạng triển khai hoặc được cấu hình endpoint rõ ràng thông qua biến môi trường.

Luồng kết nối quan trọng nhất là backend gọi AI qua `AI_GATEWAY_BASE_URL`, còn AI service gọi ngược backend qua `BACKEND_WEBHOOK_URL`. Secret ký callback của hai phía phải khớp nhau: `AI_CALLBACK_SECRET` ở backend và `BACKEND_WEBHOOK_SECRET` ở AI service.

Một điểm khác biệt quan trọng so với cấu hình phát triển là giá trị `KAFKA_CFG_ADVERTISED_LISTENERS` được thay đổi từ `PLAINTEXT://localhost:9092` thành `PLAINTEXT://kafka:9092`. Trong môi trường phát triển, Spring Boot chạy trên máy host nên dùng `localhost`; trong môi trường container, Spring Boot nằm trong một container khác và phải kết nối đến Kafka qua tên service Docker.

```
# Hạ tầng Backend
cd LMS_BE && docker compose up -d

# AI Service
cd ../LMS_AI && docker compose up -d --build

# Build image Backend và Frontend
cd ../LMS_BE && docker build -t lms-backend .
cd ../LMS_fe && docker build -t lms-frontend .
```

8.2.2 Quản lý biến môi trường và bảo mật

Tất cả thông tin nhạy cảm của hệ thống được quản lý thông qua file `.env` và không bao giờ được đưa vào mã nguồn (được liệt kê trong `.gitignore`). Bảng 8.1 liệt kê các nhóm biến môi trường chính.

Bảng 8.1: Các nhóm biến môi trường của hệ thống

Nhóm	Biến	Mục đích
Cơ sở dữ liệu	DATABASE_URL DATABASE_USERNAME DATABASE_PASSWORD	Kết nối PostgreSQL
JWT	EXP_TOKEN EXP_REFRESH_TOKEN	Thời hạn Access/Refresh token
OAuth2 Google	GOOGLE_CLIENT_ID GOOGLE_CLIENT_SECRET OAUTH2_REDIRECT_URI	Đăng nhập Google
Email	MAIL_SENDER_USERNAME MAIL_SENDER_PASSWORD	Gửi email qua SMTP
AWS S3	AWS_ACCESS_KEY AWS_SECRET_KEY	Lưu trữ file (ảnh bìa, tài liệu)
AI Gateway	AI_GATEWAY_BASE_URL AI_CALLBACK_SECRET AI_CALLBACK_MAX_CLOCK_SKEW_SECONDS	Backend gọi AI service và xác thực callback
AI Service	GEMINI_API_KEY RABBITMQ_URL BACKEND_WEBHOOK_URL BACKEND_WEBHOOK_SECRET	Chạy FastAPI, Celery, Gemini và webhook
PayOS	CLIENT_ID_PAYOS API_KEY_PAYOS CHECKSUM_KEY_PAYOS	Tạo và xác thực thanh toán phí phạt
Frontend	VITE_API_BASE_URL VITE_API_TIMEOUT	Cấu hình endpoint API và timeout khi build Vite

Khi triển khai lên máy chủ, mỗi thành phần có file môi trường riêng: `LMS_BE.env`, `LMS-AI.env` và cấu hình build cho `LMS_fe`. Các file này được tạo từ `.env.example`, điền giá trị thực tế trên server và không đưa vào lịch sử git. Với Docker Compose, các biến được tham chiếu qua `env_file` hoặc truyền vào build argument để tránh nhúng secret vào Docker image.

8.2.3 Flyway Migration tự động

Hệ thống sử dụng Flyway để quản lý phiên bản schema cơ sở dữ liệu. Khi container Spring Boot khởi động, Flyway tự động thực thi các file migration chưa được áp dụng theo thứ tự phiên bản trước khi ứng dụng nhận request. Điều này đảm bảo schema cơ sở dữ liệu luôn đồng bộ với phiên bản ứng dụng mà không cần can thiệp thủ công.

```
library-bootstrap/src/main/resources/db/migration/  
+-- V1__create_schema.sql  
+-- V8__ai_publication_etl_status.sql
```

```
+-- V14__metadata_translations.sql  
+-- V20__schema_cleanup_and_ai_indexes.sql  
\-- V22__cleanup_ai_metadata_junk.sql
```

8.3 Quy trình khởi tạo hạ tầng ban đầu

Quy trình thiết lập hệ thống từ đầu trên một máy chủ Linux (Ubuntu Server) trắng bao gồm các bước sau:

1. **Chuẩn bị môi trường máy chủ:** Cài đặt Docker Engine và Docker Compose Plugin chính thức từ Docker repository:

```
curl -fsSL https://get.docker.com | sh  
sudo usermod -aG docker $USER
```

2. **Sao chép mã nguồn:** Tải repository dự án về thư mục quản trị trên máy chủ:

```
git clone <repository-url> /opt/library74  
cd /opt/library74
```

3. **Cấu hình môi trường vận hành:** Khởi tạo các file biến môi trường bảo mật từ mẫu cấu hình và thiết lập giá trị thực tế (thông số kết nối database, khóa bí mật JWT, API key của bên thứ ba):

```
cp LMS_BE/.env.example LMS_BE/.env  
cp LMS-AI/.env.example LMS-AI/.env  
cp LMS_fe/.env.example LMS_fe/.env  
chmod 600 LMS_BE/.env LMS-AI/.env LMS_fe/.env
```

4. **Khởi động hệ thống ban đầu:** Đóng gói và kích hoạt toàn bộ các cụm container dịch vụ:

```
cd LMS_BE && docker compose up -d  
cd ../LMS-AI && docker compose up -d --build
```

5. **Kiểm tra trạng thái sẵn sàng:** Xác nhận tất cả container vận hành ổn định và đọc log khởi động ban đầu:

```
docker compose ps  
docker logs -f lms_backend  
docker logs -f lms_ai_api
```

Sau khi khởi động lần đầu, Flyway tự động thực thi các tệp migration của backend. Cụm AI service tự động đồng bộ lược đồ vector, kết nối Message Broker RabbitMQ, tải nạp mô hình Sentence-Transformers về bộ nhớ cache cục bộ và sẵn sàng phục vụ các truy vấn nghiệp vụ.

8.4 Hệ thống Tích hợp và Triển khai liên tục (CI/CD Pipeline)

8.4.1 Đặt vấn đề và mục tiêu thiết kế

Trong các dự án phần mềm quy mô lớn kết hợp đa nền tảng công nghệ (Java/Spring Boot, Python/FastAPI, React/TypeScript), việc áp dụng quy trình triển khai thủ công (Manual Deployment) bộc lộ nhiều điểm nghẽn nghiêm trọng:

- **Rủi ro sai sót thao tác (Human Error):** Lập trình viên phải trực tiếp truy cập SSH vào máy chủ sản xuất, tự thực hiện các lệnh kéo mã nguồn và build container, rất dễ nhầm lẫn biến môi trường hoặc xung đột phiên bản.
- **Mã nguồn lỗi lọt vào môi trường Production:** Nếu không có rào chắn kiểm thử tự động bắt buộc trước khi đóng gói, các commit chứa lỗi logic hoặc lỗi hồi quy (regression) có thể làm tê liệt dịch vụ ngay khi đưa lên server.
- **Rủi ro thất thoát dữ liệu:** Quá trình cập nhật phiên bản backend mới thường đi kèm các thay đổi về cấu trúc cơ sở dữ liệu (Flyway migration). Nếu không có cơ chế tự động sao lưu dữ liệu tức thì (snapshot) ngay trước thời điểm deploy, việc khôi phục dữ liệu khi xảy ra lỗi là vô cùng phức tạp.
- **Thời gian ngưng trệ dịch vụ (Downtime) kéo dài:** Việc biên dịch ứng dụng và build Docker image trực tiếp trên máy chủ gây quá tải CPU/RAM, ảnh hưởng trực tiếp đến người dùng đang hoạt động.

Nhằm khắc phục toàn diện các vấn đề trên, hệ thống CI/CD của dự án Library74 được thiết kế dựa trên nền tảng GitHub Actions với 4 mục tiêu cốt lõi:

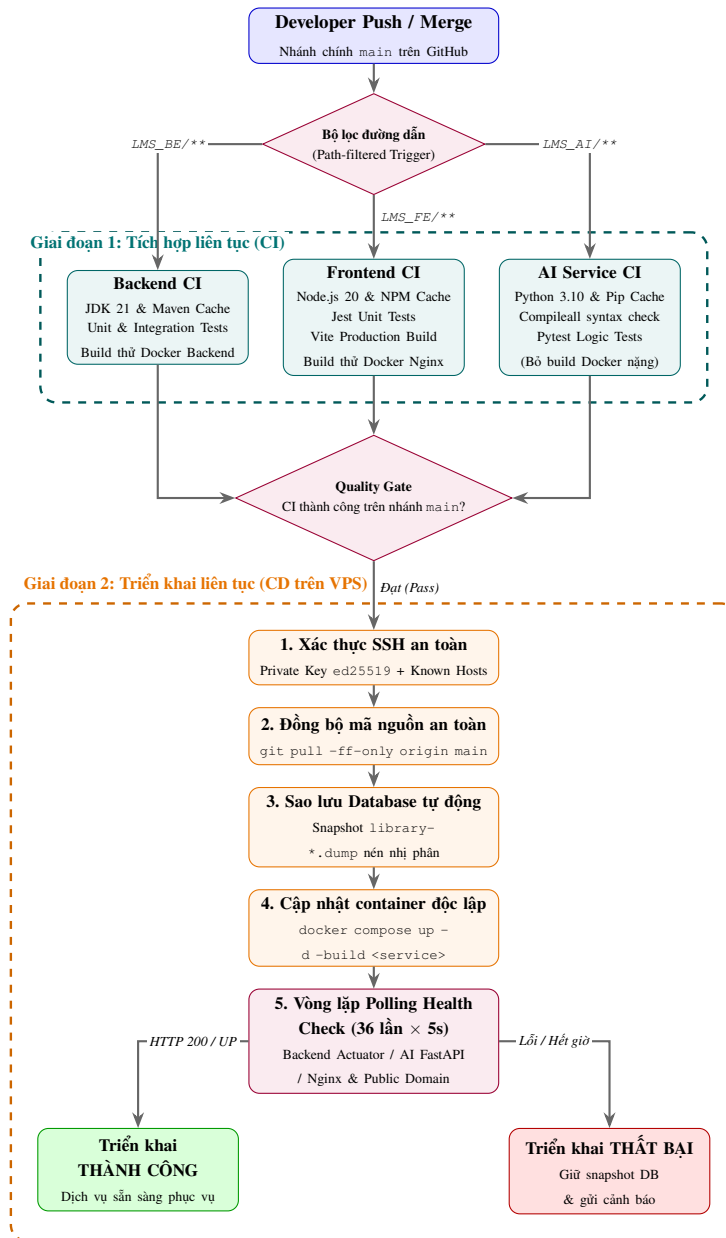
1. **Tự động hóa toàn diện 100% (End-to-End Automation):** Mọi thay đổi mã nguồn sau khi được gộp (merge) vào nhánh chính `main` đều tự động kích hoạt kiểm thử, đóng gói và triển khai lên máy chủ mà không cần can thiệp thủ công.
2. **Kiến trúc Monorepo phân tách theo module (Path-filtered Pipeline):** Bộ lọc đường dẫn thay đổi giúp tách biệt hoàn toàn pipeline của ba phân hệ độc lập (`LMS_BE`, `LMS_FE`, `LMS_AI`), giúp tiết kiệm tài nguyên và rút ngắn tối đa thời gian phát hành.
3. **An toàn dữ liệu tuyệt đối (Zero Data Loss):** Luôn tự động chụp bản sao lưu cơ sở dữ liệu nhị phân (Database Snapshot) trước khi bất kỳ phiên bản backend mới nào được khởi chạy.
4. **Kiểm tra sức khỏe dịch vụ tự động (Post-deployment Health Check):** Tự động thăm dò trạng thái sẵn sàng của container trước khi chính thức đánh dấu quá trình triển khai thành công.

8.4.2 Kiến trúc tổng thể và luồng hoạt động

Kiến trúc luồng hoạt động của hệ thống CI/CD được minh họa chi tiết trong Hình 8.1. Quy trình được chia thành bốn giai đoạn kế tiếp nhau với sự phân định rạch ròi về môi trường thực thi:

1. **Giai đoạn Kích hoạt (Trigger):** Lập trình viên đẩy commit hoặc hoàn tất Pull Request sáp nhập vào nhánh `main`. Bộ lọc đường dẫn (*Path Filter*) phân tích các tệp tin sửa đổi để xác định phân hệ bị ảnh hưởng.
2. **Giai đoạn Tích hợp liên tục (CI):** Khởi chạy máy ảo độc lập trên GitHub Actions Runner. Từng phân hệ thực hiện khôi phục bộ nhớ đệm (cache), chạy kiểm thử tự động và biên dịch thử nghiệm.

- Cổng kiểm soát chất lượng (Quality Gate):** Toàn bộ các bài kiểm tra phải đạt trạng thái `success`. Nếu có bất kỳ kiểm thử nào thất bại, pipeline lập tức dừng lại, gửi thông báo cảnh báo và từ chối tiến hành triển khai.
- Giai đoạn Triển khai liên tục (CD):** Máy ảo GitHub kết nối tới máy chủ sản xuất (Production VPS) thông qua giao thức SSH với khóa bảo mật riêng biệt, đồng bộ mã nguồn, thực hiện snapshot dữ liệu, biên dịch container và kiểm tra phản hồi của dịch vụ.



Hình 8.1: Kiến trúc tổng thể luồng CI/CD trong hệ thống Library74

8.4.3 Chi tiết giai đoạn Tích hợp liên tục (CI)

Giai đoạn CI được thực thi trên môi trường máy ảo Ubuntu sạch của GitHub (*GitHub-hosted Runner*). Bảng 8.2 tổng hợp các workflow CI tương ứng với từng thành phần kiến trúc.

Chiến lược tối ưu hóa đặc thù cho phân hệ AI CI: Trong quá trình thiết kế pipeline, phân hệ AI Service sử dụng các thư viện tính toán khoa học và học sâu rất nặng như `torch` (~1GB), `sentence-transformers`,

Bảng 8.2: Chi tiết các workflow Tích hợp liên tục (CI)

Thành phần	Tệp cấu hình	Nhiệm vụ chính	Thời gian
Backend CI	backend-ci.yml .github/workflows/	- Cấu hình OpenJDK 21 Temurin kèm cache Maven. - Thực thi toàn bộ Unit & Integration Test suite. - Build thử nghiệm Docker Image Backend nhằm xác thực Dockerfile multi-stage.	~1–2 phút
Frontend CI	frontend-ci.yml .github/workflows/	- Cấu hình Node.js 20 kèm cache NPM package. - Cài đặt sạch các gói phụ thuộc (npm ci). - Chạy toàn bộ Unit Test giao diện bằng Jest. - Biên dịch gói tĩnh Vite (npm run build). - Build thử nghiệm Docker Image Nginx đóng gói web.	~1–2 phút
AI Service CI	ai-ci.yml .github/workflows/	- Cấu hình Python 3.10 kèm cache Pip wheel. - Biên dịch kiểm tra toàn bộ cú pháp (compileall). - Thực thi bộ test logic và hợp đồng bằng Pytest.	~1–2 phút

scipy. Nếu áp dụng bước `docker build` trên GitHub-hosted Runner, máy ảo sẽ phải tải lại toàn bộ PyTorch và model weights từ đầu mà không có lớp cache cục bộ, dẫn đến thời gian thực thi CI kéo dài từ 12 đến 15 phút, gây lãng phí nghiêm trọng quota tính giờ của GitHub Actions.

Để tối ưu, nhóm phát triển đã đưa ra quyết định kiến trúc: *Tách biệt hoàn toàn việc kiểm thử logic và đóng gói container đối với AI Service*. Phân hệ AI CI trên GitHub Runner chỉ tập trung biên dịch cú pháp mã nguồn và chạy Pytest với môi trường Python chuẩn. Việc đóng gói Docker image được chuyển giao hoàn toàn cho máy chủ VPS trong giai đoạn CD. Tại VPS, Docker Daemon đã lưu trữ sẵn các layer cơ sở của PyTorch và thư viện AI trong bộ nhớ đệm cục bộ, do đó quá trình build lại image khi có cập nhật mã nguồn mới chỉ tiêu tốn từ 3 đến 5 giây.

8.4.4 Chi tiết giai đoạn Triển khai liên tục (CD)

Giai đoạn Triển khai liên tục (CD) được kích hoạt tự động theo cơ chế liên kết sự kiện `workflow_run`, với điều kiện bắt buộc là workflow CI tương ứng đã hoàn thành thành công (`conclusion == 'success'`) trên nhánh `main`, hoặc có thể được người quản trị kích hoạt thủ công thông qua sự kiện `workflow_dispatch`.

Quá trình CD trên máy chủ Production tuân thủ nghiêm ngặt quy trình an toàn 6 bước sau:

1. **Xác thực bảo mật SSH không mật khẩu (Zero Password Authentication):** Pipeline sử dụng Private Key chuẩn mã hóa hiện đại `ed25519` được lưu trữ an toàn trong GitHub Secrets (`DEPLOY_SSH_KEY`). Trước khi mở kết nối, GitHub Runner tiến hành đối soát dấu vân tay máy chủ thông qua biến `DEPLOY_KNOWN_HOSTS`, loại trừ hoàn toàn nguy cơ bị tấn công giả mạo máy chủ (Man-in-the-Middle).

2. **Đồng bộ mã nguồn an toàn (Fast-forward Synchronization):** Lệnh kéo mã nguồn sử dụng cờ `--ff-only` để đảm bảo lịch sử git trên máy chủ luôn là một đường thẳng khớp chính xác với commit đã qua kiểm thử trên nhánh main:

```
cd /opt/library74
git pull --ff-only origin main
```

Cơ chế này ngăn chặn tuyệt đối tình trạng máy chủ tự động tạo merge commit bất thường ngoài tầm kiểm soát khi môi trường thực thi bị can thiệp.

3. **Kiểm tra tính hợp lệ của cấu hình triển khai:** Trước khi thay đổi bất kỳ trạng thái nào của dịch vụ đang chạy, Docker Compose thực hiện kiểm tra cú pháp và tham chiếu biến môi trường:

```
docker compose --env-file .env.prod \
-f docker-compose.prod.yml config >/dev/null
```

4. **Bảo vệ dữ liệu tự động (Pre-deployment Database Snapshot):** Đối với phân hệ Backend CD, nhằm phòng tránh rủi ro migration thất bại gây hỏng dữ liệu, hệ thống tự động kích hoạt container sao lưu cơ sở dữ liệu:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml \
--profile backup run --rm postgres-backup
```

Tác vụ này tạo ra bản chụp cơ sở dữ liệu nhị phân nén dạng `library-YYYYMMDD-HHMMSS.dump` lưu trữ trong thư mục `backups/` trên máy chủ trước khi phiên bản backend mới khởi chạy và chạy migration.

5. **Cập nhật dịch vụ độc lập không gián đoạn (Zero Interruption):** Hệ thống áp dụng cơ chế triển khai theo từng container chuyên biệt, tránh việc restart toàn bộ hệ thống gây gián đoạn:

- Backend: `docker compose ... up -d --build backend`
- AI Service: `docker compose ... up -d --build ai-api ai-worker`
- Frontend: `docker compose ... up -d --build frontend`

Các dịch vụ hạ tầng nền tảng như PostgreSQL, Kafka, RabbitMQ, Caddy Reverse Proxy vẫn duy trì kết nối liên tục 100% mà không bị ngắt quãng.

6. **Kiểm tra sức khỏe dịch vụ sau triển khai (Post-deployment Health Check):** Để khẳng định container mới không chỉ “đang chạy” (running) mà đã thực sự sẵn sàng phục vụ yêu cầu của người dùng, script CD thực hiện vòng lặp thăm dò (polling) tối đa 36 lần, mỗi lần cách nhau 5 giây (thời gian chờ tối đa 3 phút):

- **Backend:** Truy vấn actuator health endpoint nội bộ:
`docker compose exec -T backend wget -qO- \
http://127.0.0.1:8080/actuator/health | grep -q UP`
- **AI Service:** Truy vấn endpoint kiểm tra trạng thái của FastAPI:
`docker compose exec -T ai-api \
curl -fsS http://127.0.0.1:8001/health`
- **Frontend:** Kiểm tra đồng thời phản hồi HTTP của Nginx nội bộ và domain công khai:

```
docker compose exec -T frontend wget -qO- http://127.0.0.1/ \
  >/dev/null && curl -fsS https://library74.uk/ >/dev/null
```

Nếu sau 36 lần thăm dò mà dịch vụ không phản hồi mã trạng thái thành công, pipeline lập tức báo lỗi *Deployment Failed* và gửi thông báo cho đội ngũ quản trị để kịp thời xử lý.

8.4.5 Quản lý bảo mật và phân tách cấu hình bí mật

Hệ thống tuân thủ nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege) bằng cách phân tách cấu hình thành hai lớp bảo mật:

- **Biến môi trường nghiệp vụ (.env.prod):** Được lưu trữ cố định trên máy chủ sản xuất với phân quyền nghiêm ngặt (`chmod 600`, chỉ user quản trị hệ thống mới có quyền đọc/ghi). Các khóa nhạy cảm bao gồm mật khẩu database, S3 secret key, API key PayOS và token kết nối không bao giờ được đưa vào mã nguồn hay lưu trên kho lưu trữ git.
- **Biến môi trường triển khai (GitHub Repository Secrets):** Chỉ chứa các thông số định danh cần thiết để phục vụ quá trình kết nối và vận hành tự động: `DEPLOY_HOST`, `DEPLOY_PORT`, `DEPLOY_USER`, `DEPLOY_PATH`, khóa bảo mật `DEPLOY_SSH_KEY` và chuỗi định danh `DEPLOY_KNOWN_HOSTS`.

8.4.6 Đánh giá định lượng hiệu quả hệ thống

Việc đưa hệ thống CI/CD vào vận hành chính thức đã nâng cao rõ rệt độ tin cậy, an toàn và tốc độ phát hành của dự án. Bảng 8.3 so sánh định lượng các chỉ số vận hành then chốt trước và sau khi triển khai giải pháp CI/CD.

Bảng 8.3: Đánh giá hiệu quả vận hành trước và sau khi áp dụng CI/CD

Tiêu chí đánh giá	Quy trình thủ công (Trước)	Hệ thống CI/CD (Sau)
Thời gian phát hành	15–30 phút (đăng nhập SSH, gõ lệnh thủ công từng dịch vụ).	Dưới 3 phút (tự động hóa 100% từ khi push code).
Kiểm định chất lượng mã nguồn	Phụ thuộc sự cẩn thận của lập trình viên, dễ bỏ sót test.	100% tự động vượt qua test suite trước khi được phép triển khai.
An toàn dữ liệu cơ sở dữ liệu	Thực hiện thủ công, dễ bị lãng quên trước khi cập nhật.	Tự động chụp snapshot nhị phân trước mỗi lần deploy Backend.
Mức độ gián đoạn dịch vụ (Downtime)	Downtime kéo dài do máy chủ bị chiếm dụng tài nguyên để build image.	Tối thiểu hóa downtime nhờ cập nhật tách biệt theo container.
Phát hiện lỗi phát hành	Phát hiện muộn thông qua phản ánh của người dùng hoặc kiểm tra thủ công.	Hệ thống Polling Health Check tự động bắt lỗi ngay trong 3 phút đầu.

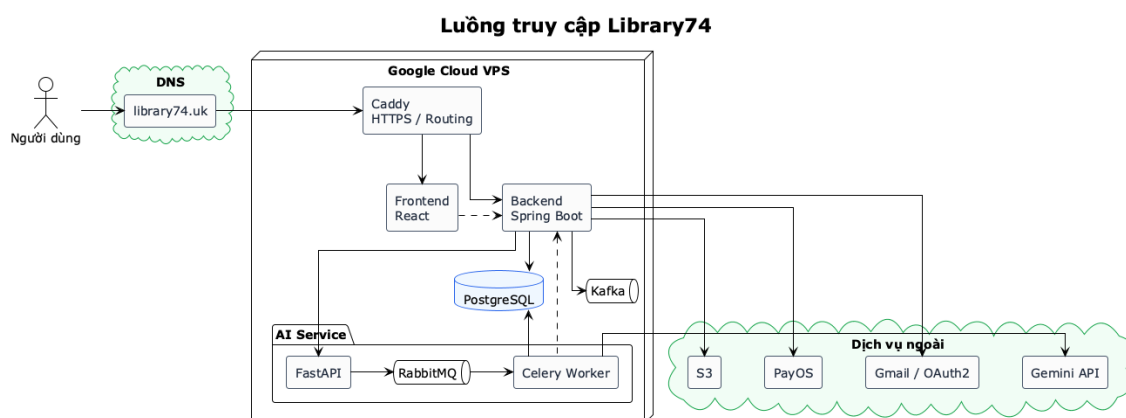
Kết quả cho thấy hệ thống CI/CD không chỉ loại bỏ hoàn toàn các thao tác thủ công rủi ro mà còn thiết lập một quy trình chuyển giao phần mềm tiêu chuẩn, an toàn và có khả năng phục hồi cao theo đúng chuẩn mực công nghiệp trong phát triển phần mềm hiện đại.

8.5 Hạ tầng mạng: Domain và HTTPS

8.5.1 Tên miền (Domain)

Trong bản triển khai demo, hệ thống sử dụng tên miền `library74.uk` để người dùng truy cập bằng một địa chỉ ổn định thay vì ghi nhớ IP hoặc cổng dịch vụ. DNS của domain được quản lý qua Cloudflare và trỏ về Google Cloud VPS. Tại VPS, Caddy nhận request HTTPS, tự động xử lý chứng chỉ TLS và định tuyến request đến frontend, backend API hoặc WebSocket theo path tương ứng.

Luồng kết nối sau khi đi vào backend tiếp tục được phân tách theo trách nhiệm: Spring Boot xử lý nghiệp vụ chính và kết nối PostgreSQL/Kafka; backend gọi AI Service qua `AI_GATEWAY_BASE_URL`; AI worker xử lý PDF bất đồng bộ qua RabbitMQ và callback kết quả về backend bằng chữ ký HMAC. Các tích hợp ngoài như S3-compatible storage, PayOS, Gmail SMTP, Google OAuth2 và Gemini API được kết nối từ backend hoặc AI worker theo đúng vai trò của từng dịch vụ.



Hình 8.2: Luồng kết nối từ tên miền Library74 đến hệ thống production

8.5.2 Tổng chi phí vận hành

Bảng 8.4 được điều chỉnh theo chi phí triển khai hiện tại. Tên miền `library74.uk` được mua theo năm, còn máy chủ demo đang chạy trên Google Cloud Platform với cấu hình ước tính khoảng 92 USD/năm. Tại thời điểm triển khai demo, GCP đang có ưu đãi miễn phí 3 tháng nên chi phí thực trả ban đầu thấp hơn chi phí vận hành ổn định về sau.

Như vậy, sau giai đoạn miễn phí ban đầu, chi phí cố định tối thiểu của hệ thống chủ yếu gồm tên miền và máy chủ GCP, khoảng 97,3 USD/năm theo cấu hình demo. Khi triển khai production thật, các thông số cần được cập nhật lại theo dung lượng PDF, số lượt truy vấn AI và chính sách giá của nhà cung cấp tại thời điểm vận hành.

Bảng 8.4: Chi phí vận hành hệ thống trong cấu hình demo

Thành phần	Chi phí	Chu kỳ	Ghi chú
Tên miền library74.uk	5,3 USD	Năm	Khoản bắt buộc để giữ địa chỉ public của demo.
SSL/HTTPS	Miễn phí	90 ngày hoặc tự động	Let's Encrypt hoặc chứng chỉ tự động từ reverse proxy/nhà cung cấp.
Google Cloud Platform	92 USD	Năm	Chi phí theo cấu hình máy chủ demo; hiện đang được miễn phí 3 tháng đầu.
Lưu trữ file	Theo mức sử dụng	Tháng	Ảnh bìa/PDF dùng object storage hoặc cấu hình tương đương; chi phí phụ thuộc dung lượng thực tế.
API ngoài	Miễn phí trong phạm vi demo	Tháng	Google OAuth, Gmail SMTP, PayOS sandbox/quota demo và Gemini quota thử nghiệm.
Tổng cố định ước tính	97,3 USD	Năm	Chưa tính ưu đãi miễn phí 3 tháng GCP và chi phí phát sinh theo dung lượng/request thực tế.

Chương 9: Tổng kết

Dự án “Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI” (Library74) được định hình và phát triển với tầm nhìn chuyển đổi số quy trình vận hành thư viện. Vượt qua giới hạn của một phần mềm quản lý thông thường, dự án đã hiện thực hóa một kiến trúc phân tán bao gồm ba phân lớp lõi: Frontend, Backend và AI Service.



Hình 9.1: Nhận diện thương hiệu của nền tảng Library74

9.1 Tổng hợp kết quả hiện thực

Đóng góp cốt lõi của đồ án không nằm ở việc xây dựng các module đơn lẻ, mà ở khả năng liên kết các phân hệ thành những luồng dữ liệu và luồng nghiệp vụ khép kín, hoạt động ổn định trên môi trường triển khai thực tế.

9.1.1 Phân lớp Frontend

Thay vì liệt kê các màn hình rời rạc, lớp giao diện được định hình theo nhóm người dùng để tối ưu hóa trải nghiệm tương tác:

- **Trang công cộng:** Triển khai cơ chế kết xuất trang tĩnh và kết xuất phía máy chủ cho trang chủ, luồng tìm kiếm, đăng nhập OAuth2 và khôi phục mật khẩu. Điều này giúp giảm thiểu rào cản tiếp cận, tối ưu hóa công cụ tìm kiếm (SEO) và cung cấp điểm chạm đầu tiên mượt mà cho độc giả.
- **Khu vực độc giả:** Xây dựng dashboard cá nhân hóa, quản lý vòng đời phiếu mượn, thanh toán mã QR, hệ thống Wishlist, Rating và thông báo thời gian thực. Các tính năng này giao quyền chủ động cho độc giả trong việc tự phục vụ và theo dõi tài nguyên học tập cá nhân.
- **Khu vực vận hành:** Thiết kế giao diện quản lý vòng đời ấn phẩm, kiểm soát hàng chờ, xử lý phạt và mất sách cùng dashboard điều hành. Phân hệ này hỗ trợ số hóa nghiệp vụ thủ công, gom cụm các thao tác vận hành vào một không gian làm việc duy nhất.
- **Khu vực quản trị:** Cung cấp công cụ cấu hình chính sách thư viện, quản trị định danh, phân quyền và giám sát hệ thống qua Audit Log, đảm bảo tính toàn vẹn, bảo mật và khả năng kiểm toán hệ thống minh bạch.

- **Tối ưu nền tảng:** Áp dụng quản lý trạng thái linh hoạt, hỗ trợ đa ngôn ngữ, hệ thống giao diện sáng tối, Route Guard và luồng Refresh Token ngầm để duy trì sự liền mạch trong các phiên làm việc kéo dài.

9.1.2 Phân lớp Backend

Máy chủ trung tâm đảm nhiệm vai trò kiểm soát toàn vẹn dữ liệu và điều phối các luồng xử lý phức tạp:

- **Quản trị định danh:** Sử dụng JWT, cơ chế xoay vòng token, xác thực email, Google OAuth2 và kiểm soát quyền truy cập dựa trên vai trò để tạo lập rào chắn bảo mật vững chắc cho toàn bộ API.
- **Quản lý tài nguyên:** Chuẩn hóa lược đồ cho ấn phẩm, tác giả, danh mục và tích hợp Storage Port lưu trữ file số an toàn, định hình nguồn dữ liệu chuẩn xác cho tài nguyên thư viện.
- **Luồng mượn trả:** Xử lý giao dịch mượn trả tuân thủ nguyên tắc ACID, quản lý hàng chờ đặt trước, xử lý ngoại lệ trễ hạn và mất sách, giải quyết bài toán cốt lõi của nghiệp vụ thư viện truyền thống.
- **Thanh toán và sự kiện:** Tích hợp cổng PayOS qua Webhook, xây dựng kiến trúc hướng sự kiện với Kafka cho thông báo và Scheduler định kỳ nhắc hạn nhằm thúc đẩy tự động hóa và giảm độ trễ phản hồi.
- **Hạ tầng và hệ thống:** Quản lý lược đồ dữ liệu với Flyway, xử lý lỗi tập trung và tích hợp AI Gateway với cơ chế bảo mật chữ ký HMAC, giúp hệ thống dễ dàng mở rộng và bảo trì trong dài hạn.

9.1.3 Phân lớp AI Service

Phân hệ trí tuệ nhân tạo được thiết kế như một dịch vụ độc lập nhằm tránh gây áp lực lên luồng xử lý đồng bộ của hệ thống chính:

- **Luồng xử lý dữ liệu PDF:** Tự động trích xuất ngữ nghĩa bằng PyMuPDF, làm sạch nhiễu, phân mảnh văn bản và tối ưu tính toán để bỏ qua các file trùng lặp, giúp chuyển đổi dữ liệu phi cấu trúc thành định dạng tiêu chuẩn.
- **Mô hình nhúng vector:** Khởi tạo vector đa chiều với SentenceTransformer chuẩn hóa tiếng Việt, lưu trữ và truy xuất thông qua pgvector trên PostgreSQL để giải quyết bài toán tìm kiếm ngữ nghĩa theo khoảng cách.
- **Làm giàu siêu dữ liệu:** Điều phối mô hình ngôn ngữ sinh tóm tắt, trích xuất nhãn đa ngữ và giới hạn tình trạng ảo giác qua cơ chế ép định dạng JSON, giúp giảm thiểu đáng kể công việc nhập liệu thủ công của thủ thư.
- **Hệ thống gợi ý:** Áp dụng thuật toán kết hợp giữa lọc cộng tác, phân tích nội dung và cơ chế đề xuất theo xu hướng cho người dùng mới, từng bước cá nhân hóa trải nghiệm khám phá tri thức.
- **Kiến trúc bất đồng bộ:** Tách biệt xử lý bằng FastAPI và Celery Worker qua RabbitMQ, giao tiếp an toàn với Backend qua mã hóa HMAC để ngăn chặn hiện tượng nghẽn cổ chai khi xử lý tác vụ AI nặng.

9.1.4 Kiểm thử và Đảm bảo chất lượng

Để minh chứng cho tính ổn định, hệ thống đã trải qua các kịch bản kiểm thử phân tầng với kết quả khả quan:

- **Kiểm thử Frontend:** Vượt qua 62/62 kịch bản Unit Test và Component Test cho logic giao diện, quản lý trạng thái và tính hợp lệ của biểu mẫu, đảm bảo tính chống chịu lỗi phía người dùng.
- **Kiểm thử Backend:** Vượt qua 79/79 kịch bản Integration Test tập trung vào logic nghiệp vụ, luồng giao dịch cơ sở dữ liệu và bảo mật API. Luồng cập nhật dữ liệu hoạt động nhất quán, không xảy ra hiện tượng bế tắc.
- **Đánh giá AI Service:** Hoàn tất 72/72 Contract Test cho luồng xử lý tự động. Việc đo lường trên tập dữ liệu chuẩn ghi nhận nDCG@5 đạt 0.907, Recall@5 đạt 87.5% và độ trễ p95 ở mức 540ms.
- **Vận hành thực tế:** Hệ thống vượt qua 8/8 kịch bản Smoke Test trên tên miền thật. Ở bài toán thăm dò chịu tải với tối đa 200 người dùng ảo đồng thời, hệ thống duy trì tỷ lệ lỗi 0.00% và đảm bảo phản hồi ổn định.

9.1.5 Đóng gói và Triển khai hạ tầng

Hệ thống được đưa lên môi trường mạng Internet với kiến trúc triển khai tiêu chuẩn:

- **Nền tảng và môi trường:** Toàn bộ hệ sinh thái được đóng gói container và điều phối bởi Docker Compose trên nền tảng Google Cloud VPS sử dụng Ubuntu 24.04.
- **Phân giải tên miền và bảo mật:** Vận hành chính thức tại tên miền <https://library74.uk>, định tuyến DNS qua mạng lưới Cloudflare. Máy chủ Caddy làm định tuyến ngược, tự động cấp phát chứng chỉ SSL để mã hóa giao tiếp.
- **Vận hành Frontend và Backend:** Ứng dụng React được biên dịch tĩnh và phân phối qua máy chủ Nginx. Khối Spring Boot liên kết chặt chẽ với các dịch vụ nội bộ (PostgreSQL, Kafka, RabbitMQ) và nền tảng bên ngoài (S3, PayOS, Google).
- **Vận hành AI Engine và dữ liệu:** Môi trường FastAPI vận hành độc lập cùng hệ thống Worker lắng nghe hàng đợi. Hệ quản trị PostgreSQL được ánh xạ bộ nhớ độc lập, trong khi hình ảnh và PDF được đồng bộ với Amazon S3 để dự phòng.

9.2 Đánh giá tác động thực tiễn

Dự án Library74 đã chứng minh được tính khả thi trong việc ứng dụng nền tảng công nghệ để giải quyết bài toán vận hành thực tế của thư viện đại học. Tác động của hệ thống không mang tính lý thuyết mà mang lại giá trị vận hành cụ thể cho từng nhóm đối tượng tham gia:

- **Đối với sinh viên và giảng viên:** Dưới sự hỗ trợ của công cụ tìm kiếm ngữ nghĩa và hệ thống gợi ý, khoảng cách từ nhu cầu tra cứu học liệu đến quyết định mượn sách được thu hẹp đáng kể. Nền tảng cung cấp một luồng tự phục vụ xuyên suốt, giúp bạn đọc chủ động và nắm toàn quyền kiểm soát quá trình mượn trả tài liệu của cá nhân.

- **Đối với thủ thư:** Hệ thống giải phóng nhân sự khỏi khối lượng lớn các công việc hành chính thủ công. Thông qua các tính năng quản lý vòng đời tài liệu, đối soát thanh toán tự động và xử lý hàng chờ trực tuyến, các bước nghiệp vụ được chuẩn hóa. Điều này giúp hạn chế tối đa sai sót do thao tác con người, từ đó nâng cao hiệu suất phục vụ tổng thể.
- **Đối với nhà trường:** Hệ thống kiến tạo một kho dữ liệu tập trung, lưu vết toàn bộ vòng đời tương tác của tài nguyên học liệu. Đây là nguồn dữ liệu nền tảng thiết yếu để cấp quản lý phân tích xu hướng tìm kiếm, đánh giá mức độ lưu thông sách và đưa ra các quyết định hoạch định chiến lược đầu tư tài nguyên mới một cách có cơ sở.
- **Đối với bài toán chuyển đổi số:** Dự án khẳng định quá trình chuyển đổi số thư viện không đơn thuần dừng lại ở việc chuyển đổi file tài liệu lên máy chủ. Bản chất cốt lõi của giải pháp là sự số hóa toàn diện luồng tương tác giữa con người và tri thức, kết hợp linh hoạt các công cụ học máy để tạo ra bước tiến thực sự về chất lượng dịch vụ.

9.3 Nhận định kiến trúc và giới hạn kỹ thuật

Dựa trên kết quả thực nghiệm, dự án ghi nhận các ưu điểm trong thiết kế kiến trúc cũng như các rào cản kỹ thuật cần khắc phục để đưa hệ thống lên mức chịu tải quy mô lớn hơn:

- **Kiến trúc nghiệp vụ:** Đạt được sự ổn định nhờ tính toàn vẹn cho các giao dịch mượn trả và phân rã module rõ ràng. Tuy nhiên, do chưa áp dụng kiến trúc vi dịch vụ hoàn chỉnh, hệ thống có thể đối mặt với rào cản mở rộng cục bộ nếu một module gặp tình trạng quá tải đột biến.
- **Trí tuệ nhân tạo:** Kiến trúc tách rời giúp luồng backend không bị ảnh hưởng bởi quá trình suy luận của AI. Dù vậy, hệ thống gợi ý vẫn chịu ảnh hưởng của hiện tượng khởi động nguội do thiếu tập dữ liệu tương tác thực, đồng thời siêu dữ liệu dễ bị nhiễu nếu file PDF đầu vào có định dạng sai lệch.
- **Hạ tầng và vận hành:** Hệ thống dễ dàng tái tạo tự động thông qua cấu trúc mã hóa và lớp bảo mật định tuyến nghiêm ngặt. Điểm hạn chế là việc kiểm thử tải mới dừng ở mức thăm dò có kiểm soát, chưa thực hiện các kịch bản kiểm thử giới hạn cực đại để xác định điểm gãy của phần cứng.

9.4 Định hướng phát triển

Để nâng tầm Library74 thành một nền tảng phục vụ cho các đại học quy mô lớn, các trọng tâm kỹ thuật trong giai đoạn tiếp theo bao gồm:

- **Xây dựng hệ thống giám sát toàn diện:** Tích hợp các công cụ theo dõi đo lường và ghi log theo thời gian thực, đồng thời cài đặt hệ thống cảnh báo tự động để phát hiện nguy cơ trước khi máy chủ chạm ngưỡng thất cổ chai.
- **Khắc phục nhiễu dữ liệu đầu vào:** Bổ sung công nghệ nhận dạng ký tự quang học kết hợp thuật toán phân tích bố cục hình ảnh để trích xuất văn bản từ sách scan cổ, sách mờ hoặc có thiết kế dàn trang phức tạp.

- **Tối ưu hóa hệ thống gợi ý:** Chuyển dịch dần từ việc phân tích dữ liệu tĩnh sang nắm bắt hành vi ngầm của độc giả, chẳng hạn như thời gian dừng trên trang hoặc tỷ lệ nhấp chuột, nhằm tinh chỉnh mô hình đề xuất sát với thực tế hơn.
- **Cơ chế dự phòng và phục hồi:** Tự động hóa hoàn toàn luồng sao lưu gia tăng cho cơ sở dữ liệu và thiết lập các kịch bản chuyển đổi dự phòng để đảm bảo tính sẵn sàng cao cho hệ thống vận hành.

9.5 Kết luận chung

Đồ án "**Xây dựng Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI**" đã hoàn thành các mục tiêu nghiên cứu và kỹ thuật được đặt ra ban đầu. Dự án không chỉ dừng lại ở mức độ nghiên cứu lý thuyết mà đã phát triển thành một hệ thống phần mềm có mức độ hoàn thiện ổn định, đáp ứng được các yêu cầu vận hành cơ bản.

Bằng việc kết hợp hài hòa giữa kiến trúc phần mềm, quy trình nghiệp vụ thư viện và khả năng hỗ trợ của các thuật toán học máy, Library74 là một ví dụ cụ thể cho việc áp dụng nền tảng kỹ thuật công nghệ vào kiến tạo những giải pháp mang lại giá trị thực tiễn cho môi trường giáo dục đại học.

Tài liệu tham khảo

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992.
- [2] A. Kane, “pgvector: Open-source vector similarity search for PostgreSQL,” <https://github.com/pgvector/pgvector>, 2024, accessed: 2026-08-25.
- [3] R. Burke, “Hybrid recommender systems: Survey and experiments,” *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002.
- [4] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008)*. IEEE, 2008, pp. 263–272.
- [5] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *IEEE Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [6] J. Beel, B. Gipp, S. Langer, and C. Breiting, “Research-paper recommender systems: a literature survey,” *International Journal on Digital Libraries*, vol. 17, no. 4, pp. 305–338, 2016.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, 2017, pp. 5998–6008.
- [8] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 2020, pp. 1037–1042.
- [9] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [10] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [11] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [12] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.
- [13] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

- [14] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [15] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [16] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” Internet Engineering Task Force (IETF), RFC 2104, Feb. 1997.
- [17] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” Internet Engineering Task Force (IETF), RFC 7519, May 2015.
- [18] J. Hodges, C. Jackson, and A. Barth, “HTTP strict transport security (HSTS),” Internet Engineering Task Force (IETF), RFC 6797, Nov. 2012.
- [19] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.
- [20] Caddy Server Project, “Caddy: The ultimate server with automatic HTTPS,” <https://caddyserver.com/docs/>, 2024, accessed: 2026-08-25.
- [21] F5, Inc., “NGINX documentation,” <https://nginx.org/en/docs/>, 2024, accessed: 2026-08-25.

Tài nguyên và Mã nguồn dự án

Để phục vụ cho quá trình thiết kế, kiểm thử và đánh giá, báo cáo đính kèm các đường dẫn đến tài nguyên của dự án:

Loại tài nguyên	Đường dẫn truy cập
Mã nguồn (Source Code)	Frontend: https://github.com/imsythang/LMS_fe Backend: https://github.com/quangthangk4/LMS_BE AI: https://github.com/howard-hosythang/LMS_AI Deploy: https://github.com/howard-hosythang/LMS
Website triển khai	https://library74.uk
Thiết kế giao diện (Figma)	https://www.figma.com/design/aSHkyebIWfWN7LMgTeY0be/Library-Management-System?node-id=0-1&t=mf5nzMKPeV3M6Vyp-1

Bảng 1: Danh sách các tài nguyên trực tuyến của hệ thống